Design and Analysis of Algorithms CC342

☺ Correctness, Recursion and Recurrences☹

Lectures 5-6

Dr. Salman Masih

Assistant Professor

Department of Computer Science

University of Management and Technology, Sialkot, Pakistan

Outline

1. Main Idea of Divide-and-Conquer
2. Linear Search
3. Binary Search







a problem to be solved

Divide: Break into non-overlapping subproblems of the same type



Divide: Break into non-overlapping subproblems of the same type



Divide: Break into non-overlapping subproblems of the same type

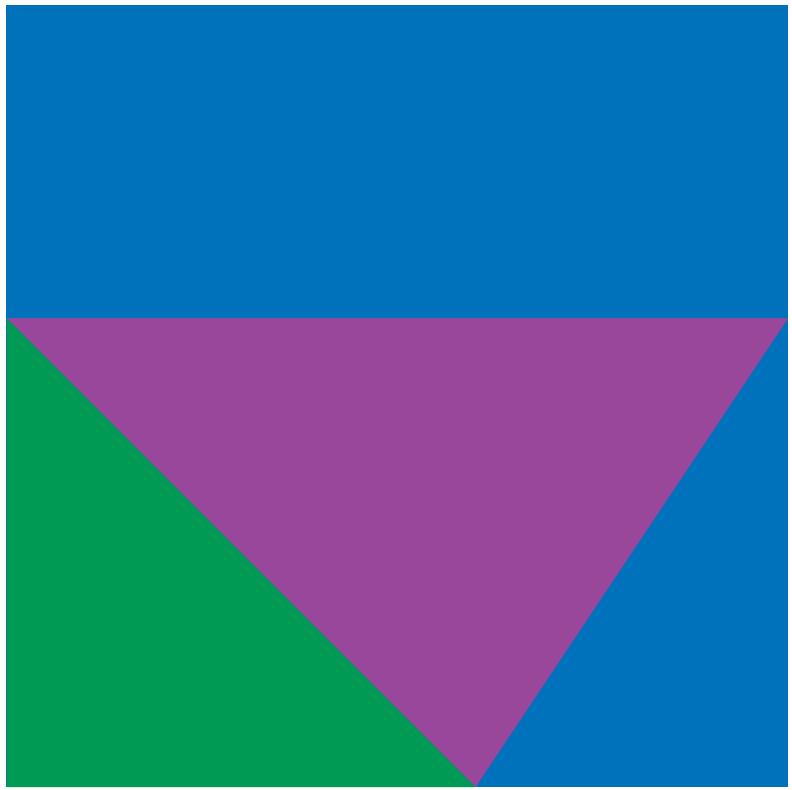


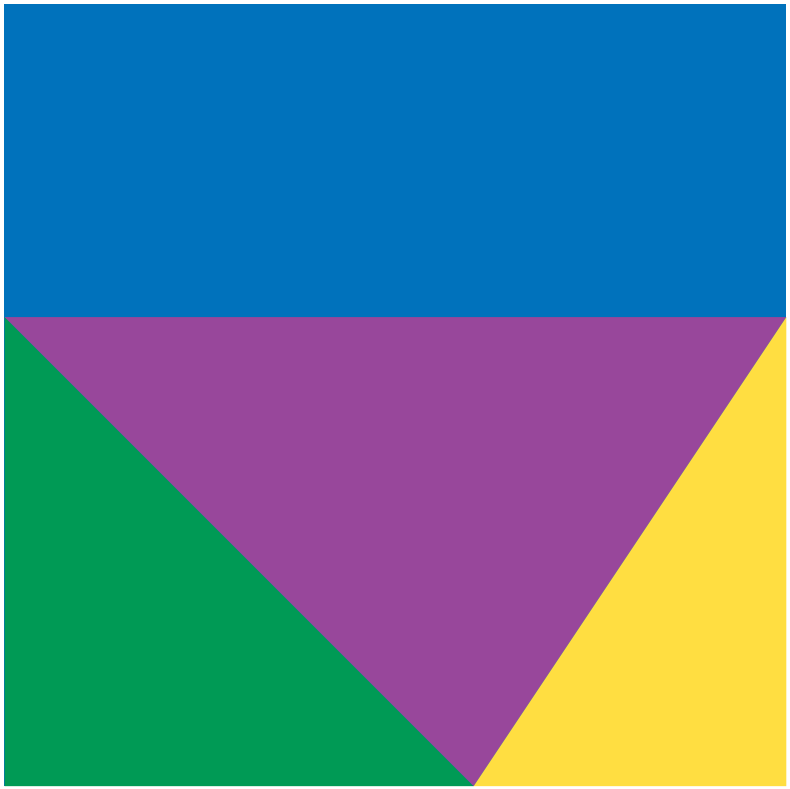
Divide: Break into non-overlapping subproblems of the same type

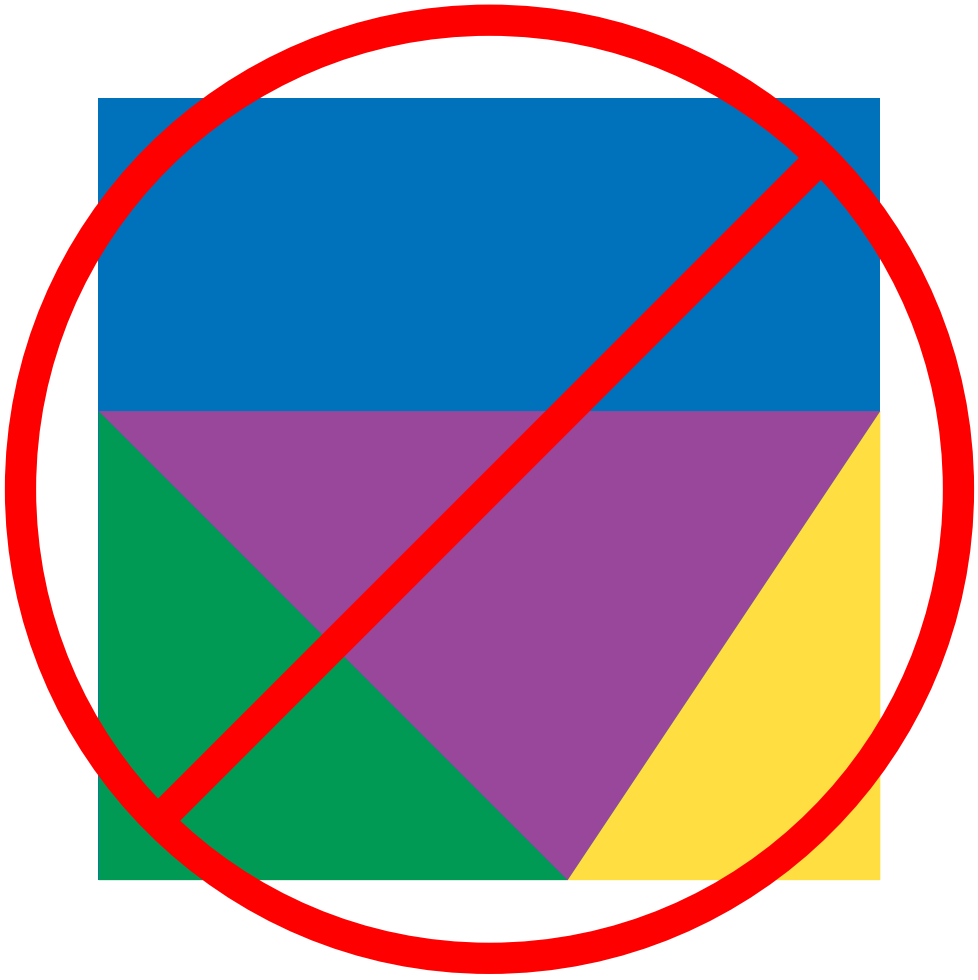












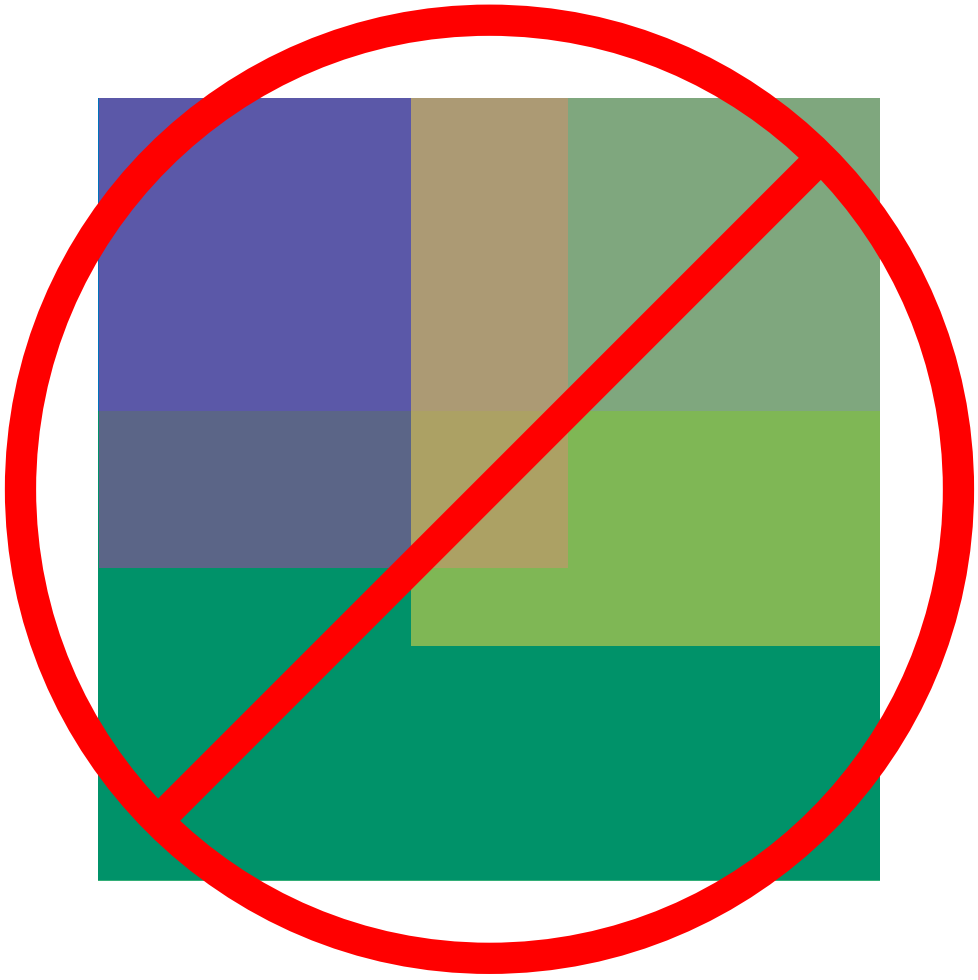
not the
same type











overlapping

Divide: break apart



Divide: break apart



Conquer: solve subproblems



Conquer: solve subproblems



Conquer: solve subproblems



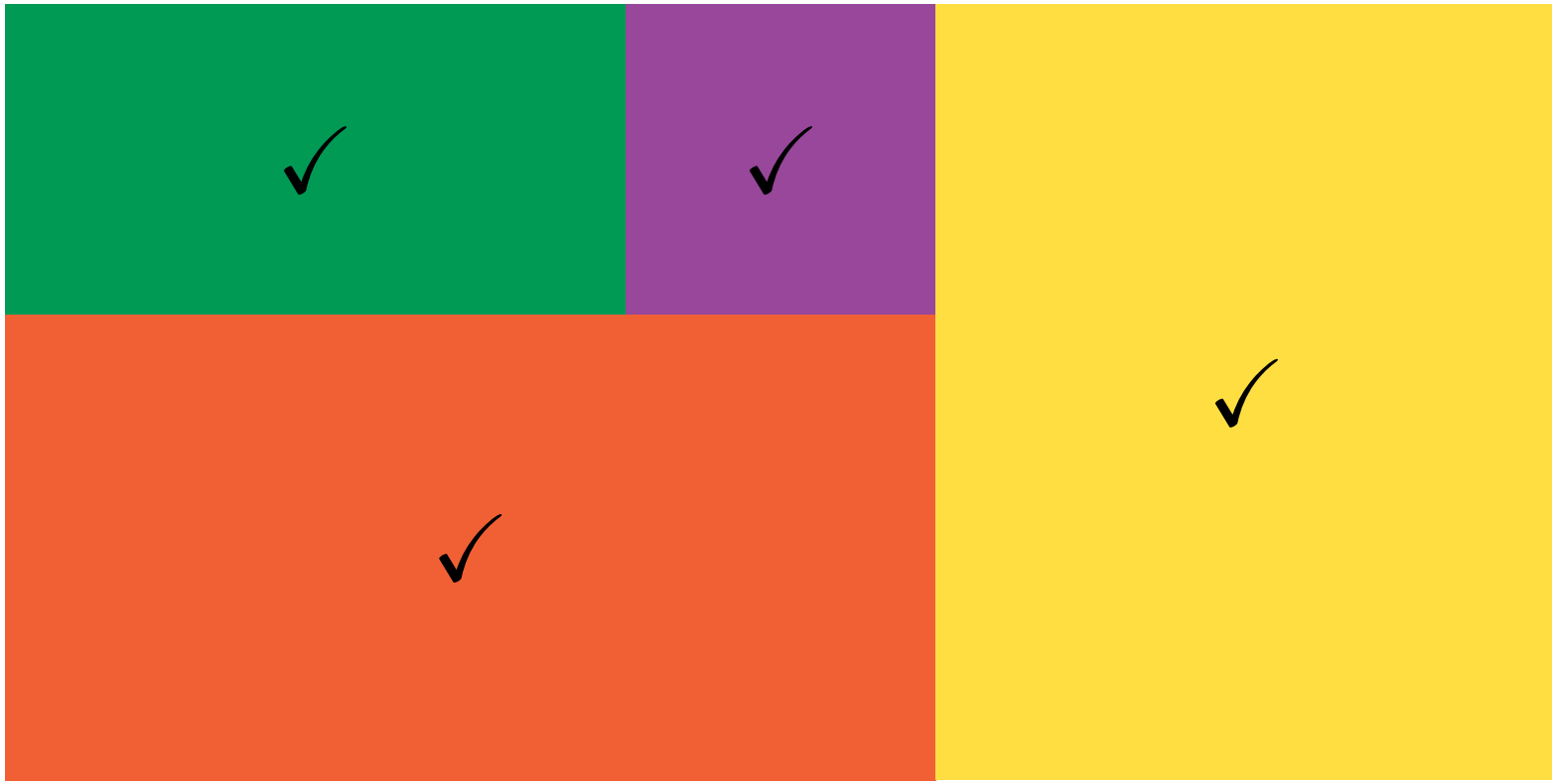
Conquer: solve subproblems



Conquer: solve subproblems



Conquer: combine



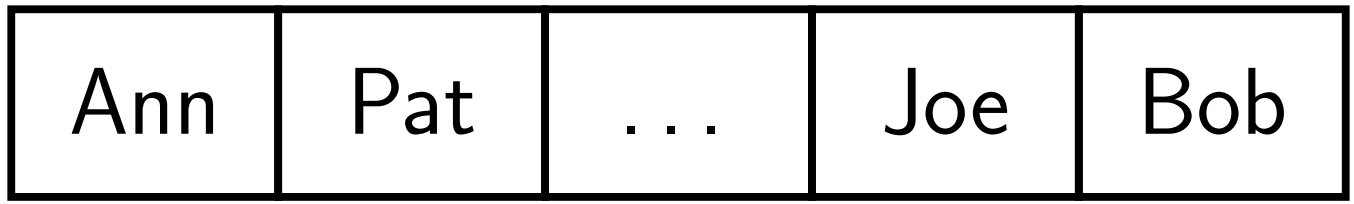


1. Break into non-overlapping subproblems of the same type
2. Solve subproblems
3. Combine results

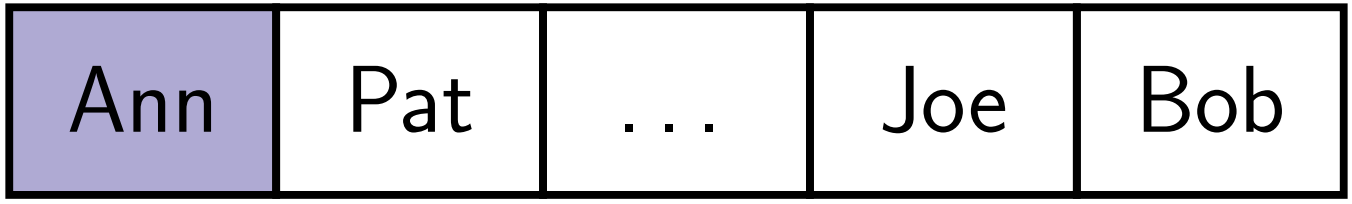
Outline

1. Main Idea of Divide-and-Conquer
2. Linear Search
3. Binary Search

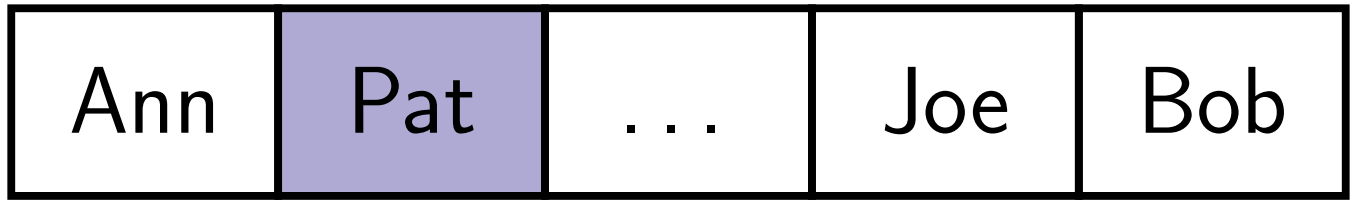
Linear Search in Array



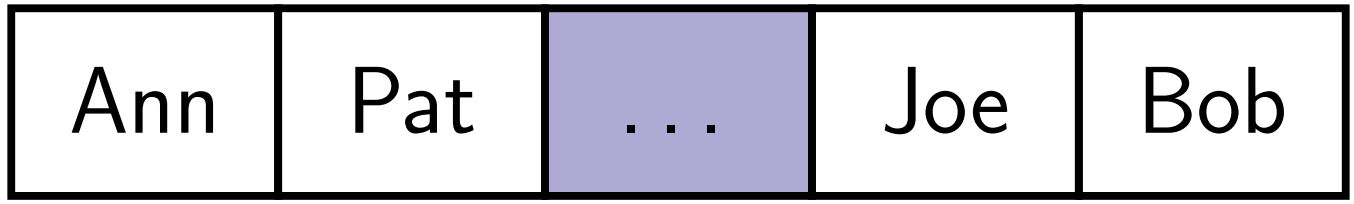
Linear Search in Array



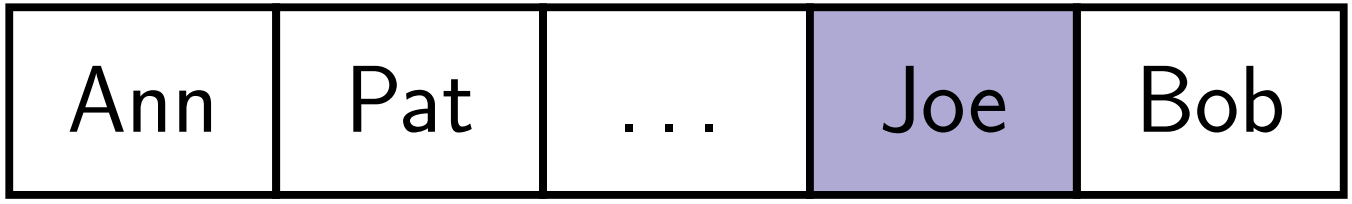
Linear Search in Array



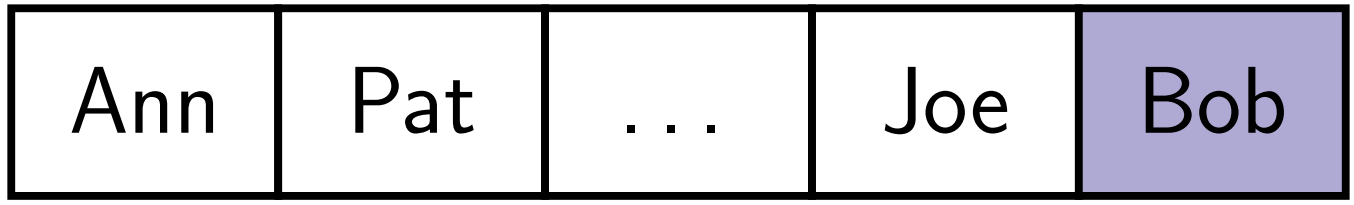
Linear Search in Array



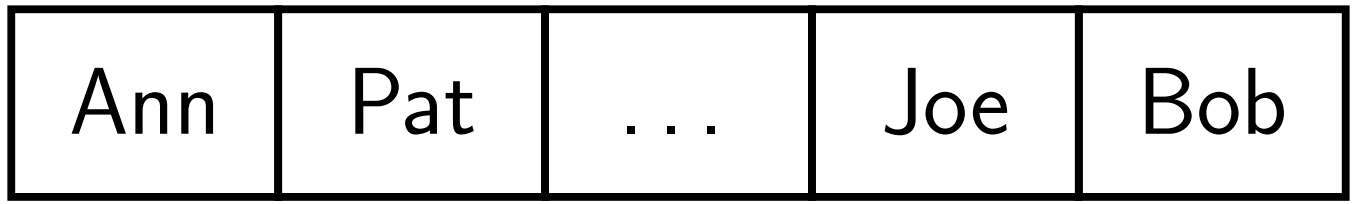
Linear Search in Array



Linear Search in Array



Linear Search in Array



Real-life Example

english	**french**	**italian**	**german**	**spanish**
house	maison	casa	Haus	casa
car	voiture	auto	Auto	auto
table	table	tavola	Tabelle	mesa

Searching in an array

Input: An array A with n elements.
A key k.

Output: An index, i, where $A[i] = k$.
If there is no such i, then
NOT_FOUND.

Recursive Solution

`LinearSearch`(A, low, $high$, key)

Recursive Solution

LinearSearch(A, low, $high$, key)

```
if high < low:
   return NOT_FOUND
if A[low] = key:
   return low
```

Recursive Solution

LinearSearch(A, low, $high$, key)

```
if high < low:
   return NOT_FOUND
if A[low] = key:
   return low
return LinearSearch(A, low + 1, high, key)
```

Recursive Solution

LinearSearch(A, low, $high$, key)

```
if high < low:
   return NOT_FOUND
if A[low] = key:
   return low
return LinearSearch(A, low + 1, high, key)
```

Definition

A recurrence relation is an equation recursively defining a sequence of values.

Definition

A recurrence relation is an equation recursively defining a sequence of values.

Fibonacci recurrence relation

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Definition

A recurrence relation is an equation recursively defining a sequence of values.

Fibonacci recurrence relation

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

$$0, 1, 1, 2, 3, 5, 8, \ldots$$

LinearSearch(A, low, $high$, key)

```
if high < low:
  return NOT_FOUND
if A[low] = key:
  return low
return LinearSearch(A, low + 1, high, key)
```

LinearSearch(A, low, $high$, key)

```
if high < low:
  return NOT_FOUND
if A[low] = key:
  return low
return LinearSearch(A, low + 1, high, key)
```

Recurrence defining worst-case time:

$$T(n) = T(n-1) + c$$

LinearSearch(A, low, $high$, key)

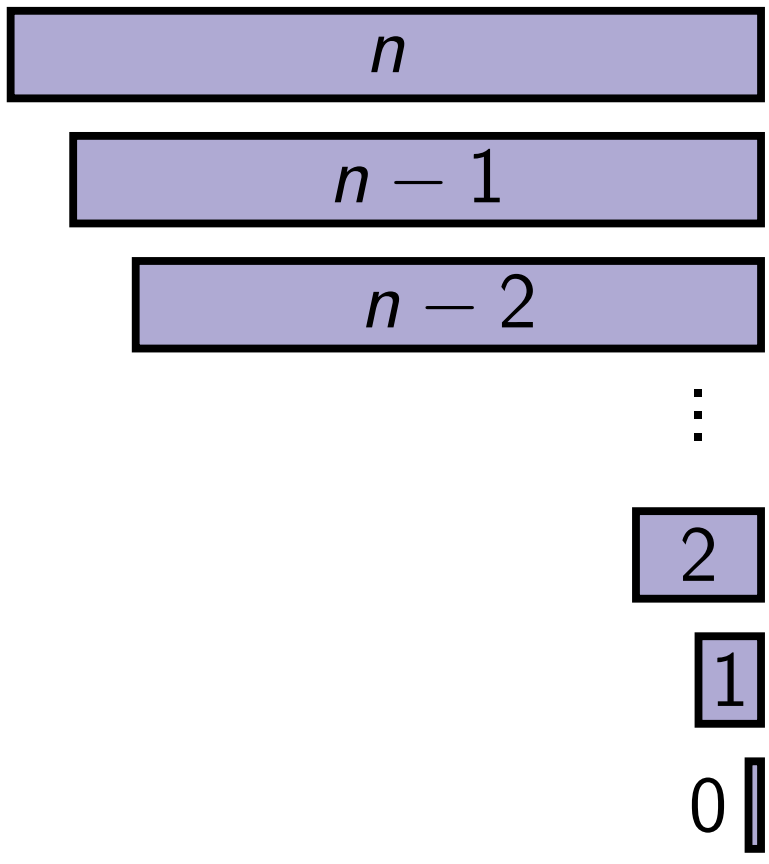
```
if high < low:
  return NOT_FOUND
if A[low] = key:
  return low
return LinearSearch(A, low + 1, high, key)
```

Recurrence defining worst-case time:

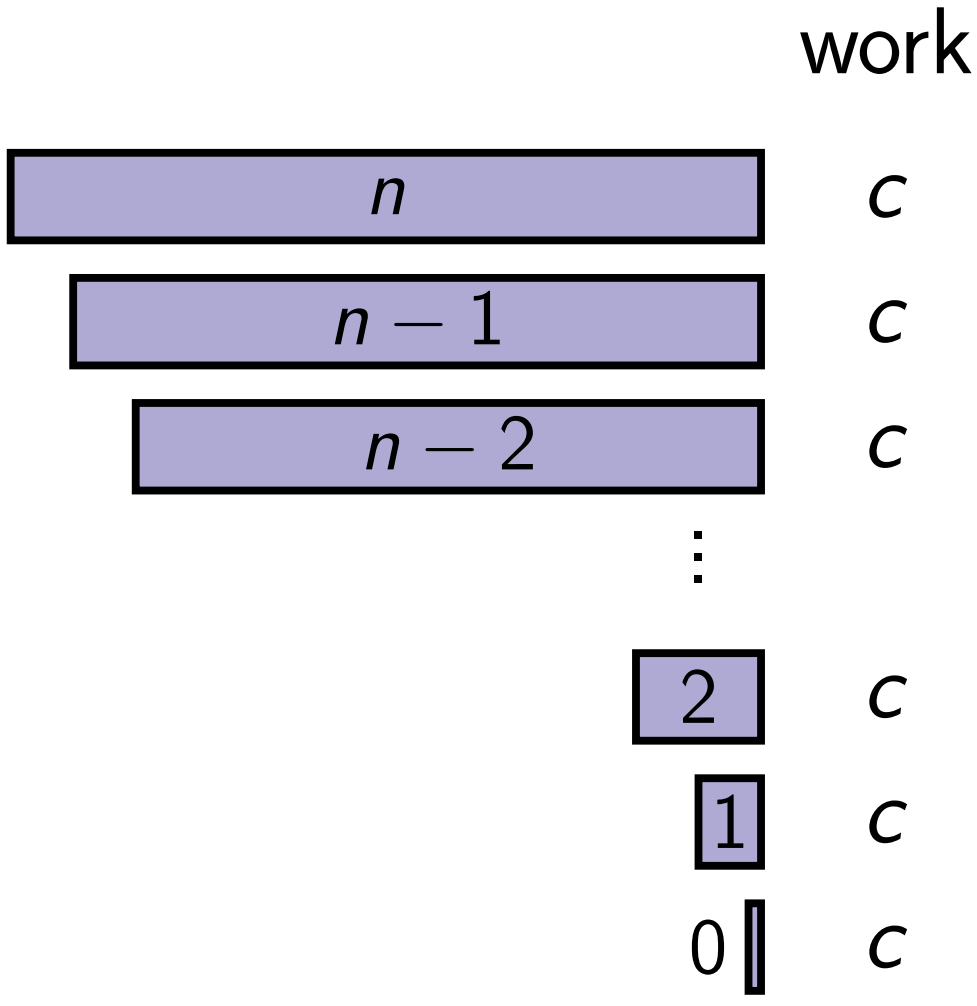
$$T(n) = T(n-1) + c$$

$$T(0) = c$$

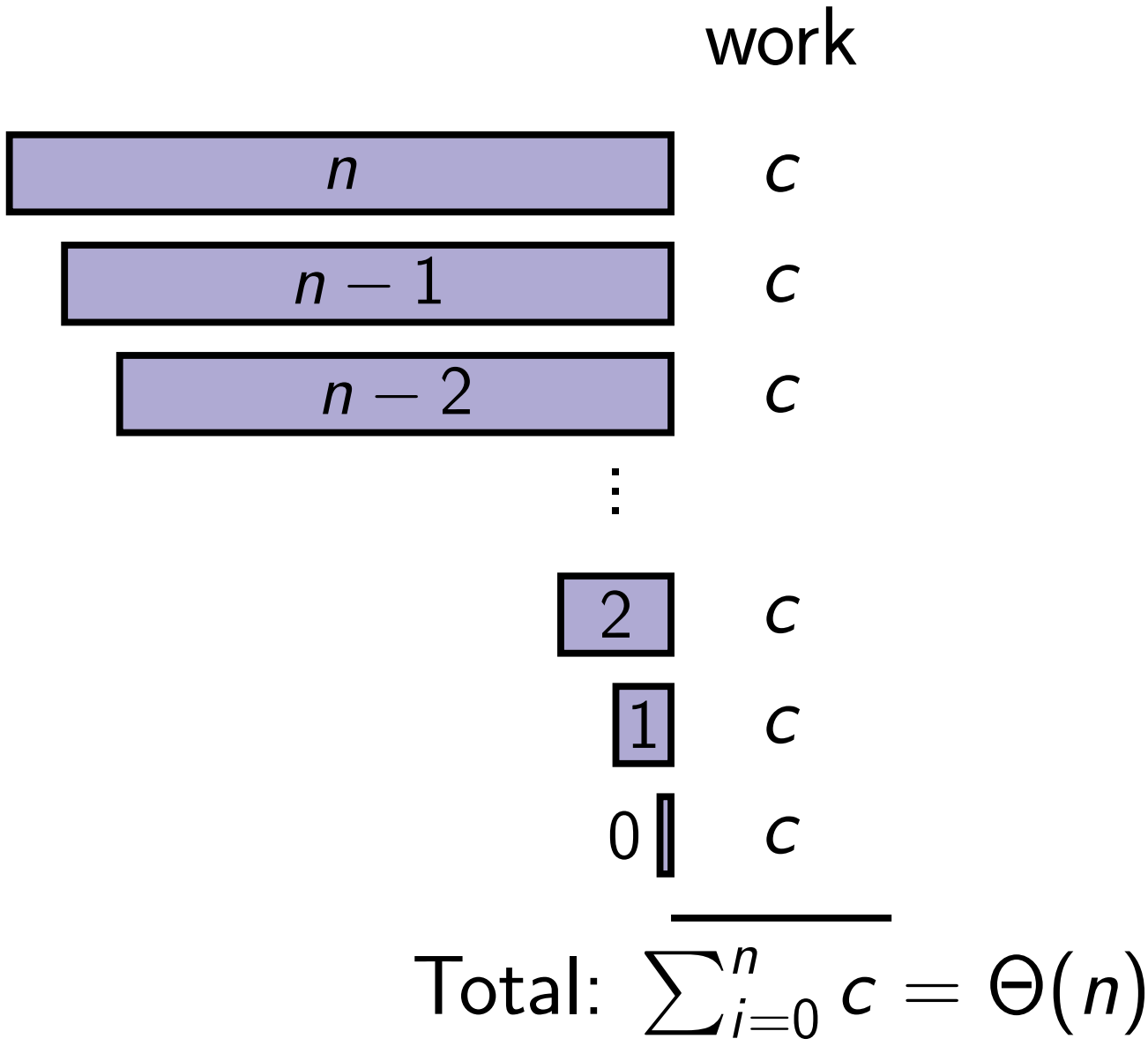
Runtime of Linear Search



Runtime of Linear Search



Runtime of Linear Search



Iterative Version

`LinearSearchIt`(A, *low*, *high*, *key*)

```
for i from low to high:
    if A[i] = key:
        return i
return NOT_FOUND
```

Summary

- Create a recursive solution

Summary

- Create a recursive solution
- Define a corresponding recurrence relation, T

Summary

- Create a recursive solution
- Define a corresponding recurrence relation, T
- Determine $T(n)$: worst-case runtime

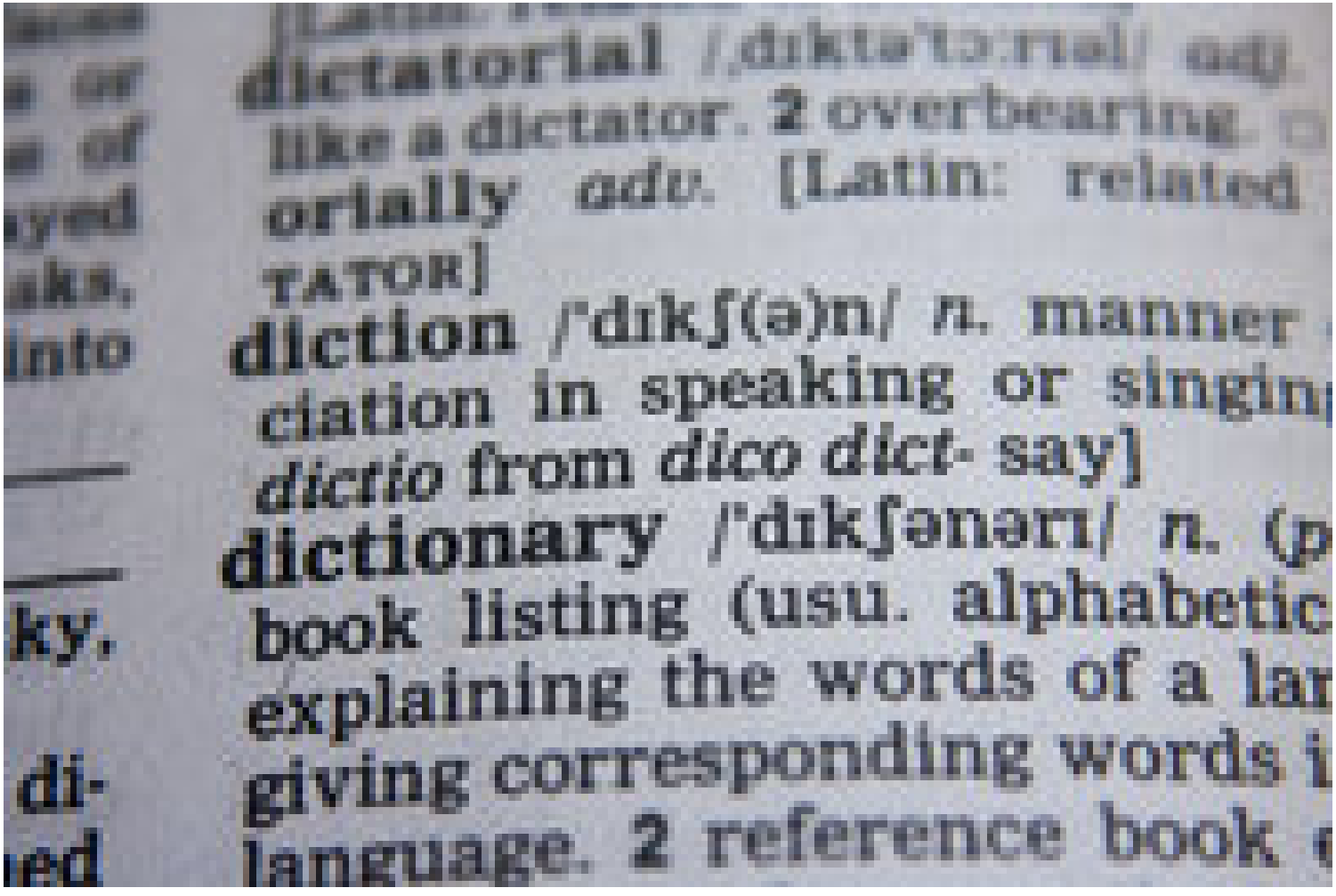
Summary

- Create a recursive solution
- Define a corresponding recurrence relation, T
- Determine $T(n)$: worst-case runtime
- Optionally, create iterative solution

Outline

1. Main Idea of Divide-and-Conquer
2. Linear Search
3. Binary Search

Searching Sorted Data



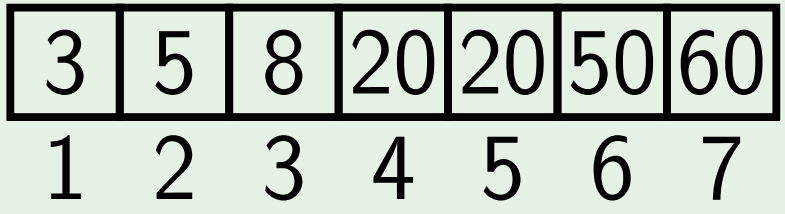
Searching in a sorted array

Input: A sorted array $A[low \ldots high]$
$(\forall low \leq i < high : A[i] \leq A[i+1])$.
A key k.

Output: An index, i, $(low \leq i \leq high)$ where
$A[i] = k$.
Otherwise, the greatest index i,
where $A[i] < k$.
Otherwise $(k < A[low])$, the result is
$low - 1$.

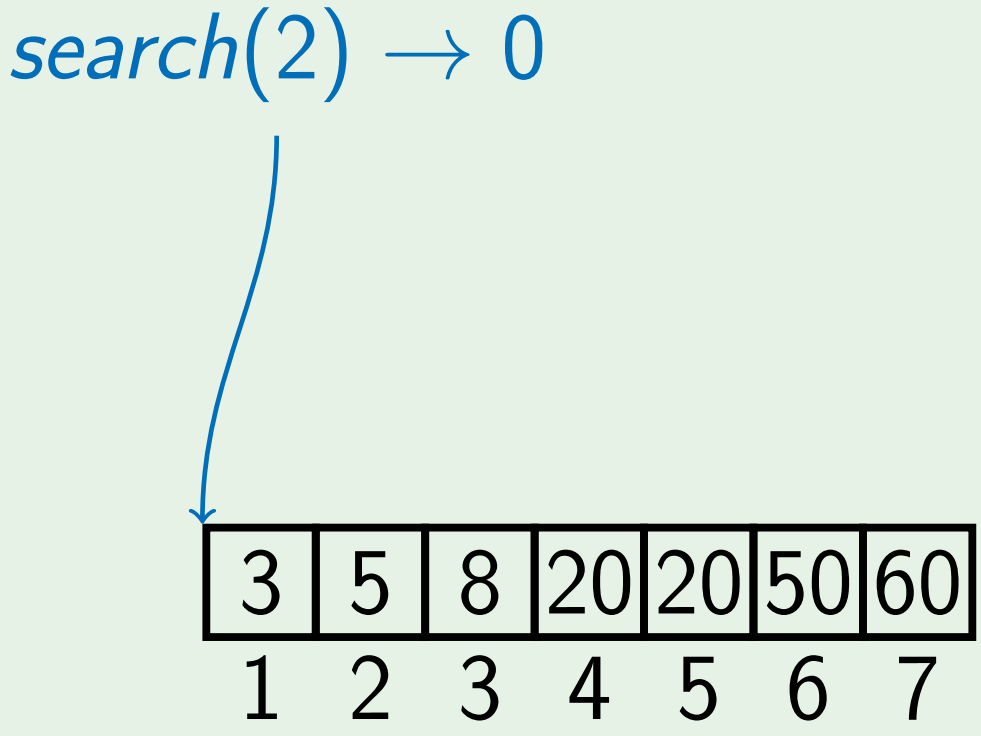
Searching in a Sorted Array

Example



Searching in a Sorted Array

Example

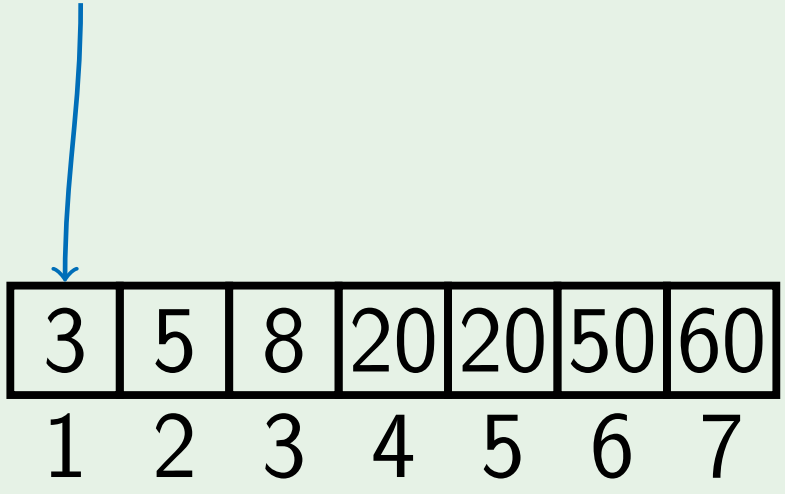


Searching in a Sorted Array

Example

$search(2) \rightarrow 0$

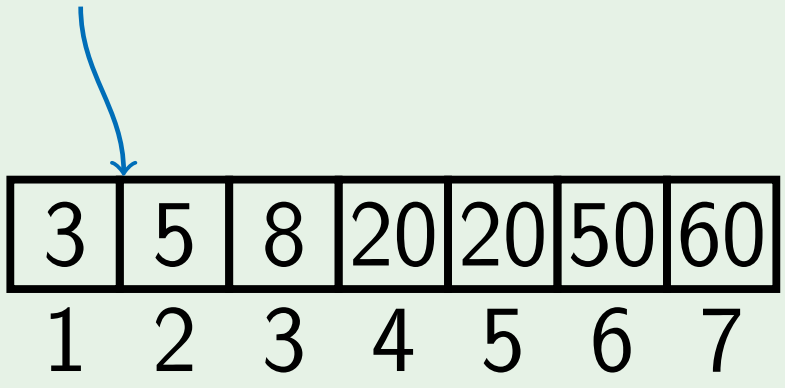
$search(3) \rightarrow 1$



Searching in a Sorted Array

Example

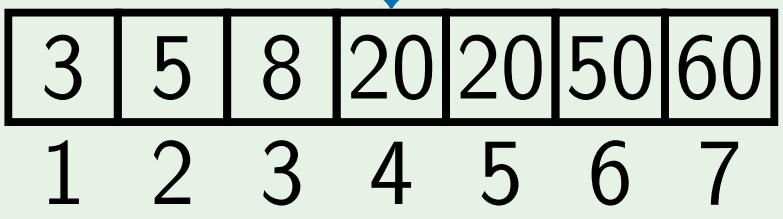
$search(2) \rightarrow 0$
$search(3) \rightarrow 1$
$search(4) \rightarrow 1$



Searching in a Sorted Array

Example

$search(2) \rightarrow 0$ $search(20) \rightarrow 4$
$search(3) \rightarrow 1$
$search(4) \rightarrow 1$



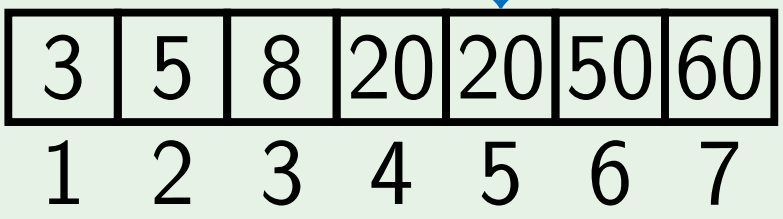
Searching in a Sorted Array

Example

$search(2) \to 0$ $search(20) \to 4$

$search(3) \to 1$ $search(20) \to 5$

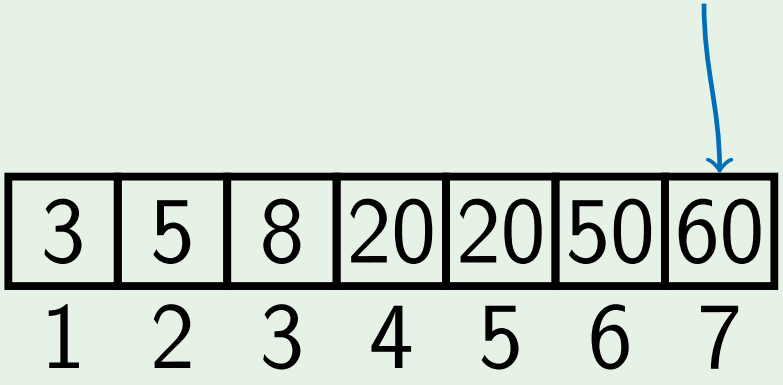
$search(4) \to 1$



Searching in a Sorted Array

Example

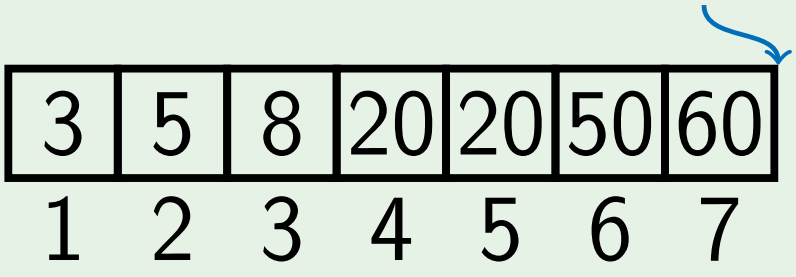
$search(2) \to 0$ $search(20) \to 4$
$search(3) \to 1$ $search(20) \to 5$
$search(4) \to 1$ $search(60) \to 7$



Searching in a Sorted Array

Example

$search(2) \to 0$ $search(20) \to 4$
$search(3) \to 1$ $search(20) \to 5$
$search(4) \to 1$ $search(60) \to 7$
$search(70) \to 7$



`BinarySearch`(A, low, $high$, key)

BinarySearch(A, *low*, *high*, *key*)

```
if high < low:
   return low − 1
```

BinarySearch(A, low, $high$, key)

if $high < low$:
  return $low - 1$
$mid \leftarrow \left\lfloor low + \frac{high - low}{2} \right\rfloor$

BinarySearch(A, low, $high$, key)

```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
```

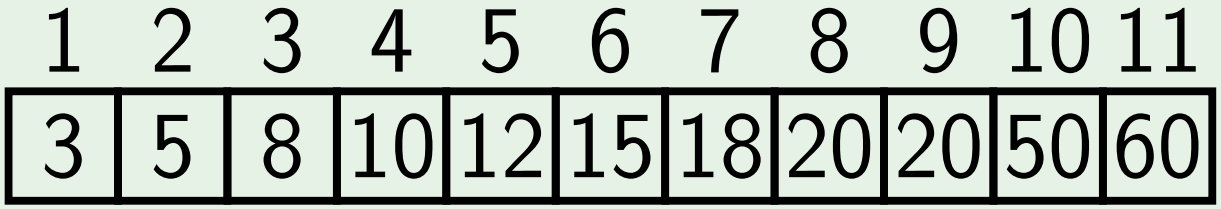
BinarySearch(A, low, $high$, key)

```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
else if key < A[mid]:
    return BinarySearch(A, low, mid − 1, key)
```

BinarySearch(A, low, $high$, key)

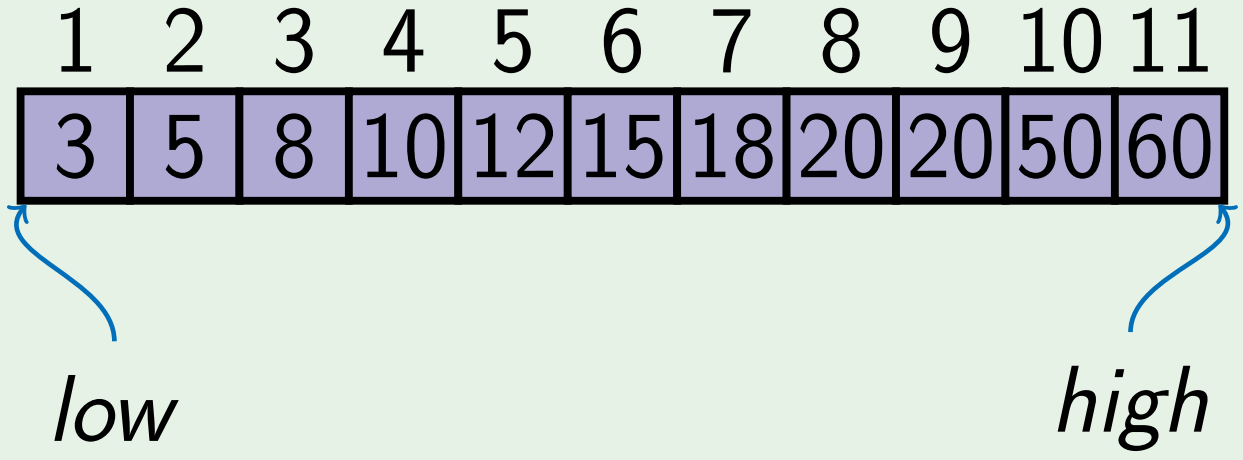
```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
else if key < A[mid]:
    return BinarySearch(A, low, mid − 1, key)
else:
    return BinarySearch(A, mid + 1, high, key)
```

Example: Searching for the key 50



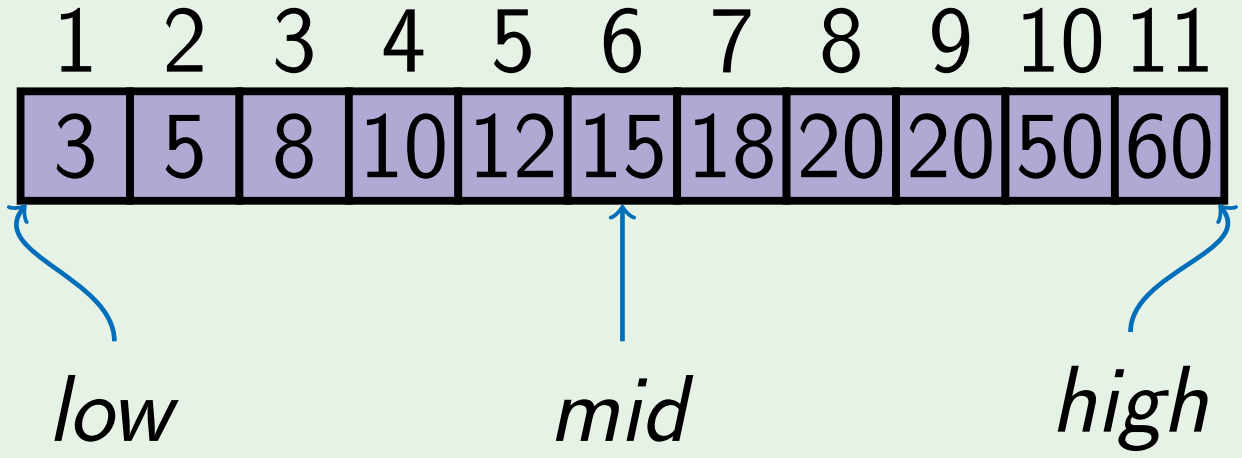
Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$



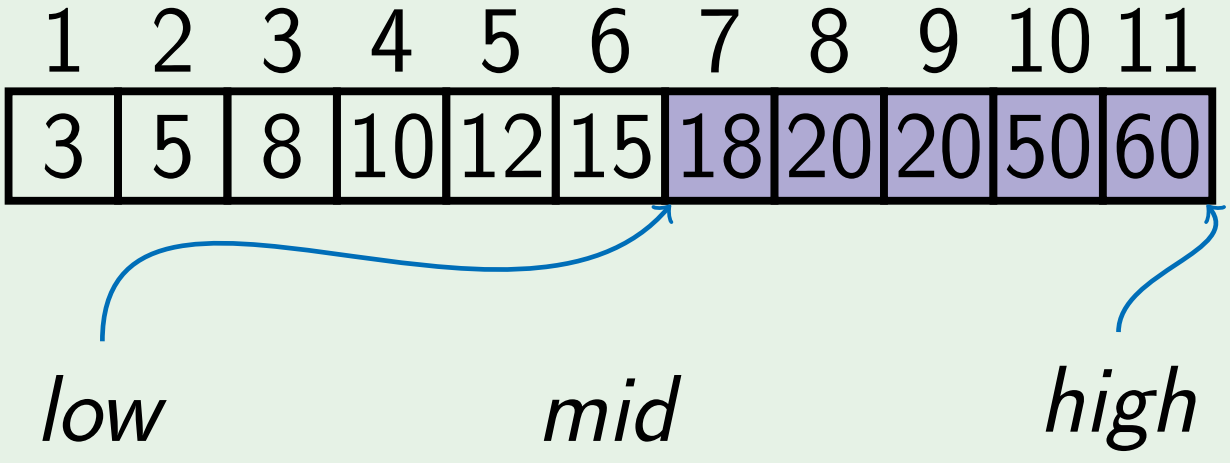
Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$



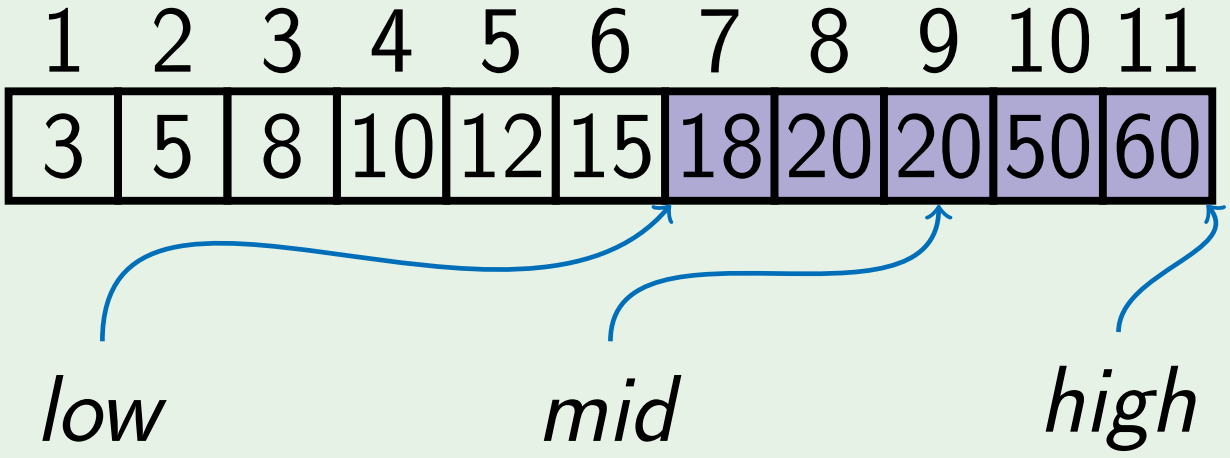
Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$



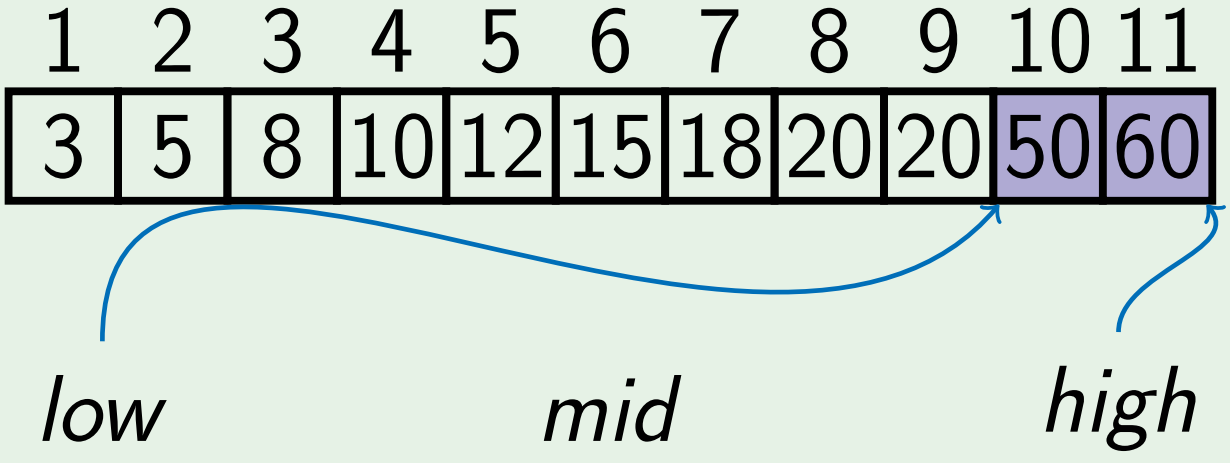
Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$



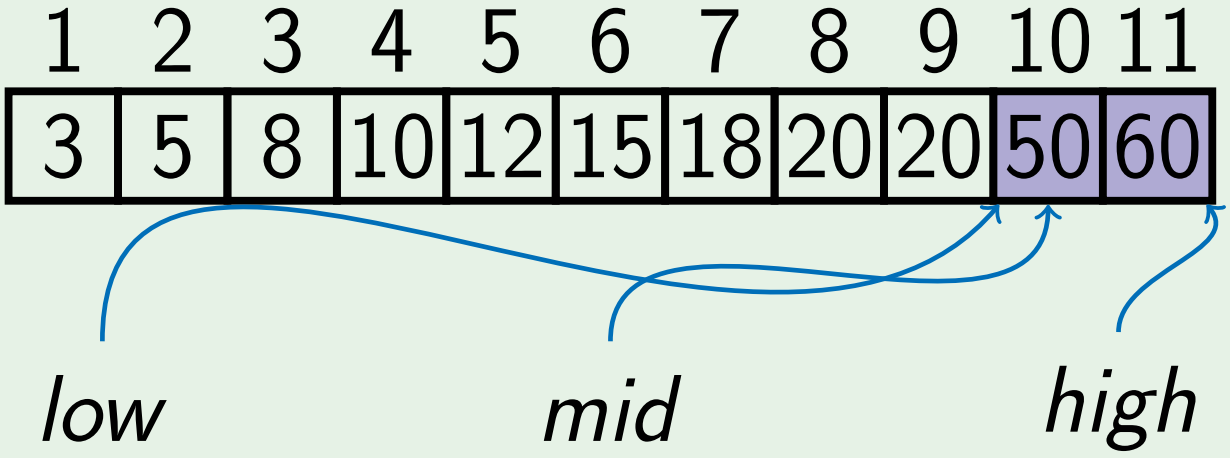
Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$
$\texttt{BinarySearch}(A, 10, 11, 50)$



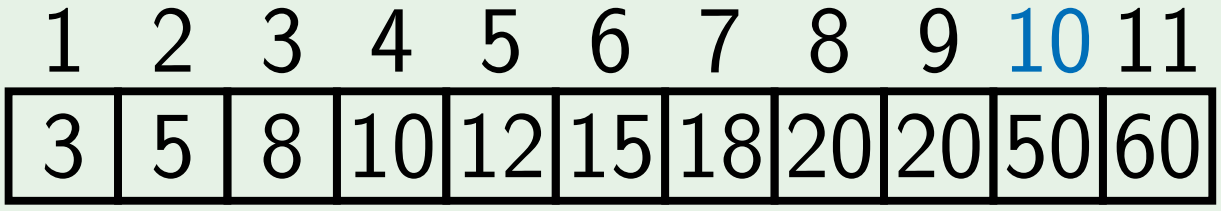
Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$
$\texttt{BinarySearch}(A, 10, 11, 50)$



Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$
$\texttt{BinarySearch}(A, 10, 11, 50) \rightarrow 10$



Summary

Summary

- Break problem into non-overlapping subproblems of the same type.

Summary

- Break problem into non-overlapping subproblems of the same type.
- Recursively solve those subproblems.

Summary

- Break problem into non-overlapping subproblems of the same type.
- Recursively solve those subproblems.
- Combine results of subproblems.

BinarySearch(A, low, $high$, key)

```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
else if key < A[mid]:
    return BinarySearch(A, low, mid − 1, key)
else:
    return BinarySearch(A, mid + 1, high, key)
```

Binary Search Recurrence Relation

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$$

Binary Search Recurrence Relation

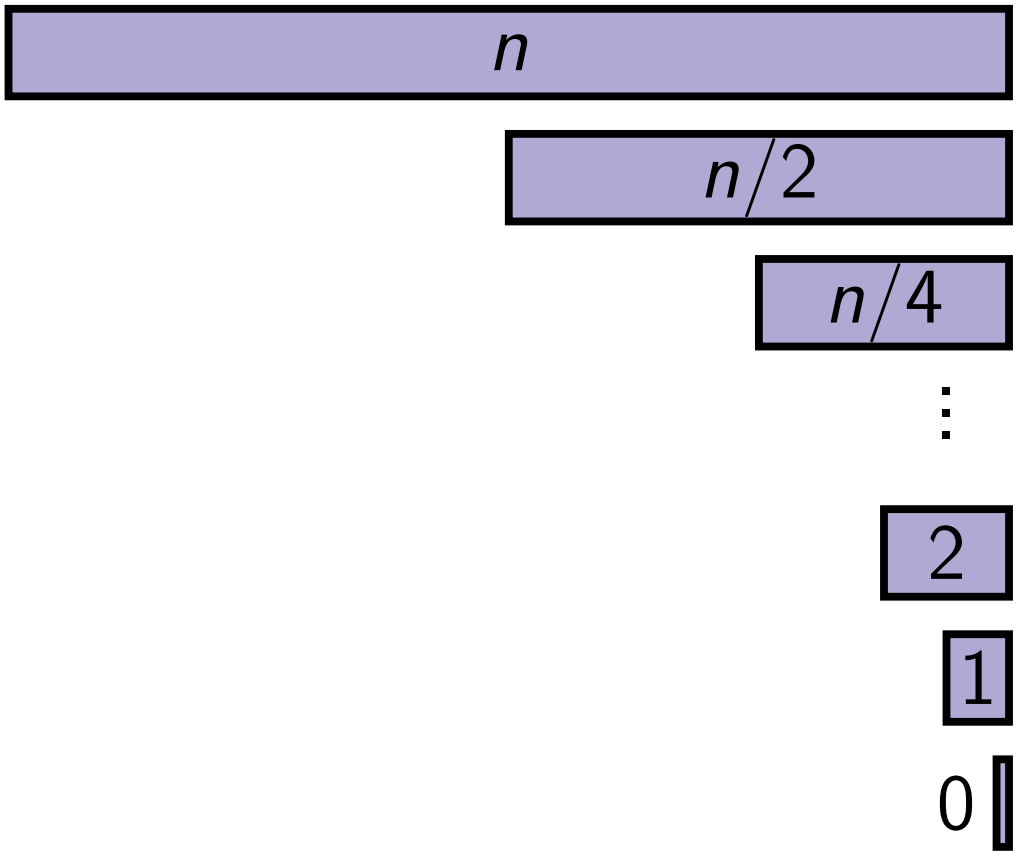
$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$$

Binary Search Recurrence Relation

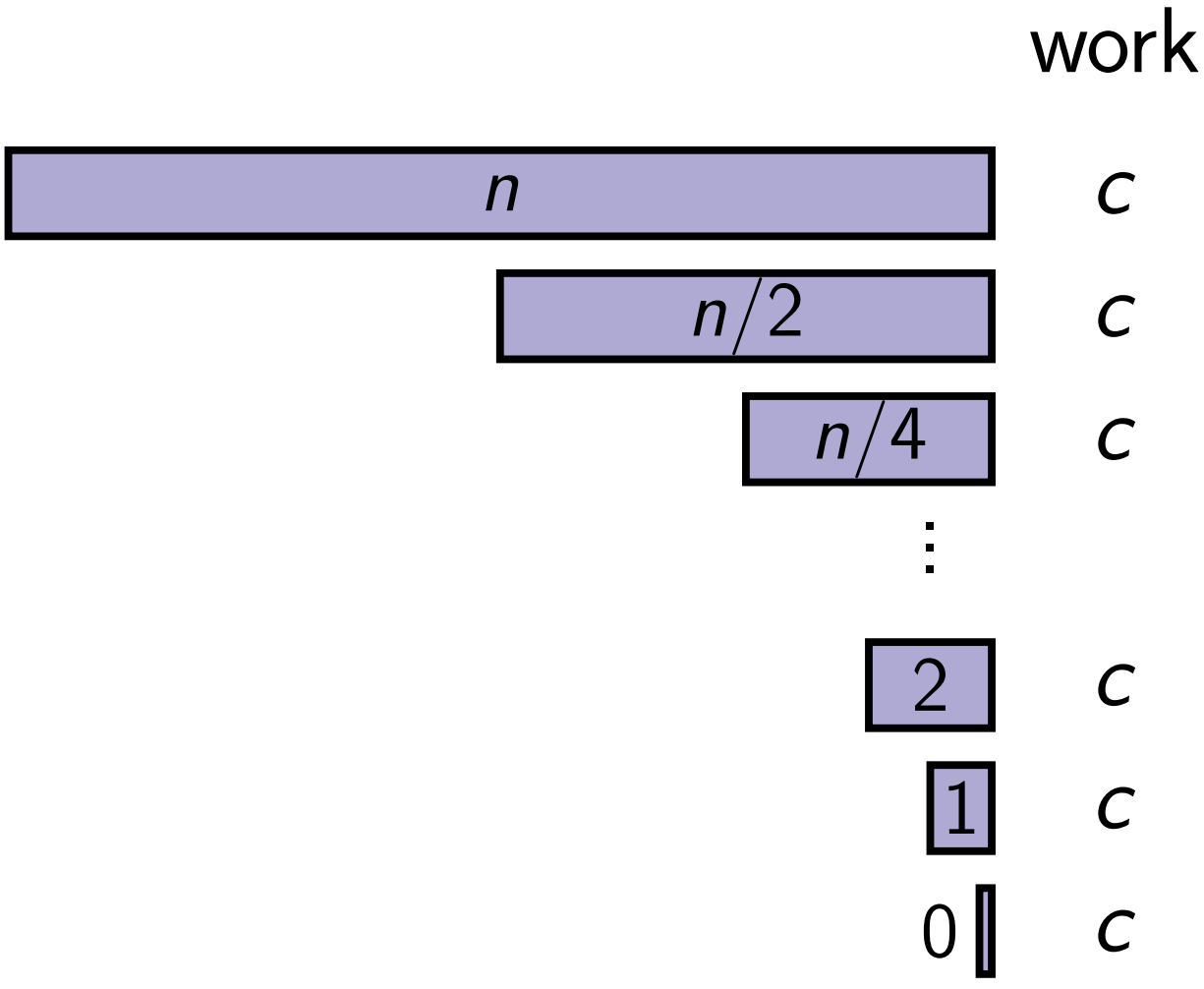
$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$$

$$T(0) = c$$

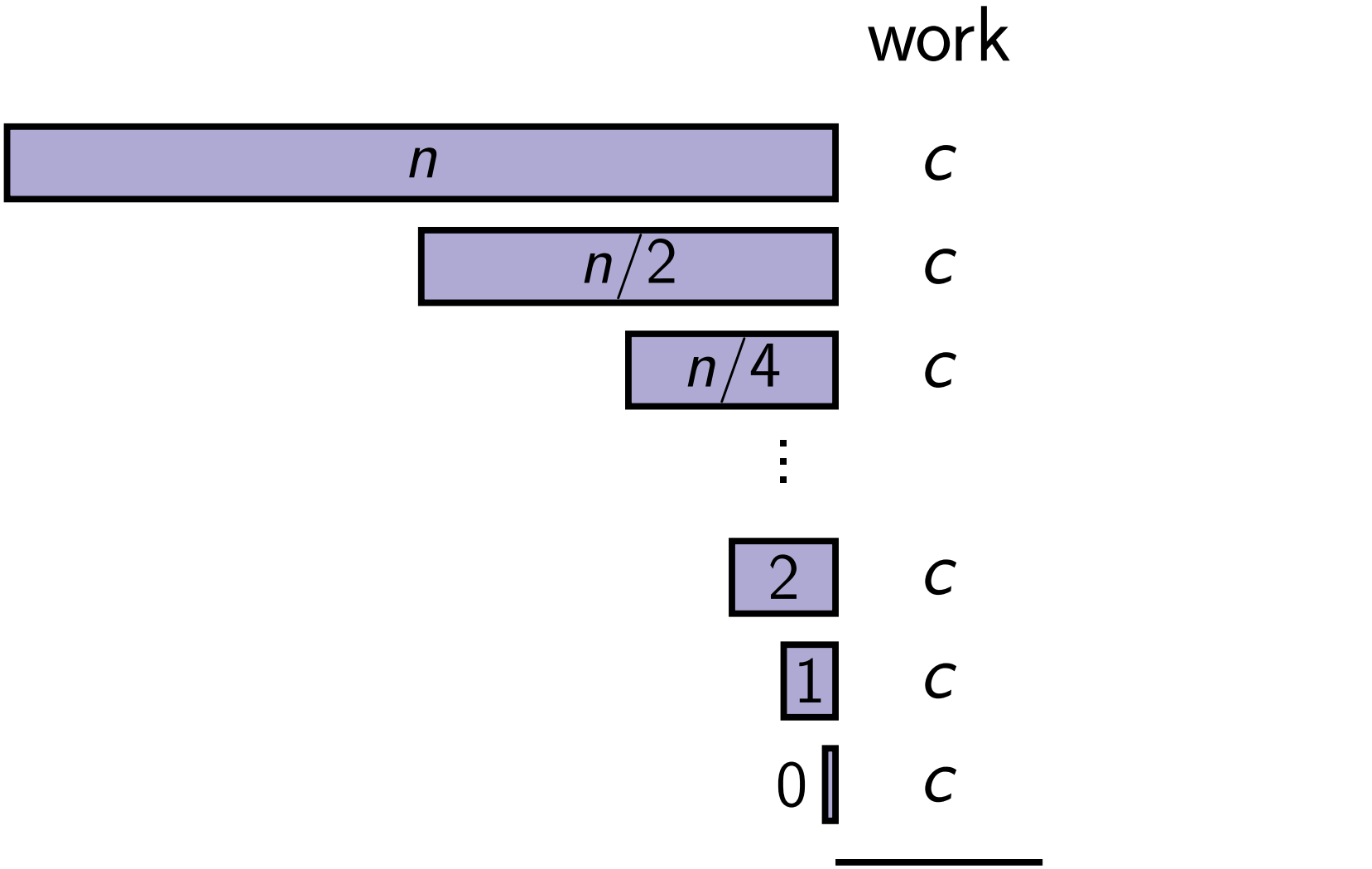
Runtime of Binary Search



Runtime of Binary Search



Runtime of Binary Search



Total: $\sum_{i=0}^{\log_2 n} c = \Theta(\log_2 n)$

Iterative Version

`BinarySearchIt`$(A, low, high, key)$

`while` $low \leq high$:

$mid \leftarrow \left\lfloor low + \frac{high - low}{2} \right\rfloor$

Iterative Version

`BinarySearchIt`$(A, low, high, key)$

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
```

Iterative Version

`BinarySearchIt`$(A, low, high, key)$

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
    else if key < A[mid]:
        high = mid − 1
```

Iterative Version

`BinarySearchIt`$(A, low, high, key)$

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
    else if key < A[mid]:
        high = mid − 1
    else:
        low = mid + 1
```

Iterative Version

BinarySearchIt($A, low, high, key$)

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
    else if key < A[mid]:
        high = mid − 1
    else:
        low = mid + 1
return low − 1
```

Real-life Example

english	**french**	**italian**	**german**	**spanish**
house	maison	casa	Haus	casa
chair	chaise	sedia	Sessel	silla
pimple	bouton	foruncolo	Pickel	espenilla

Real-life Example

english (sorted)	**french** (sorted)	**italian** (sorted)	**german** (sorted)	**spanish** (sorted)
chair	chaise	casa	Haus	casa
house	bouton	foruncolo	Pickel	espenilla
pimple	maison	sedia	Sessel	silla

Real-life Example

english	**french**	**italian**	**german**	**spanish**
house	maison	casa	Haus	casa
chair	chaise	sedia	Sessel	silla
pimple	bouton	foruncolo	Pickel	espenilla

english
sorted

2
1
3

spanish
sorted

1
3
2

Real-life Example

english	**french**	**italian**	**german**	**spanish**
house	maison	casa	Haus	casa
chair	chaise	sedia	Sessel	silla
pimple	bouton	foruncolo	Pickel	espenilla



english
sorted

2
1
3

spanish
sorted

1
3
2

Real-life Example

english	**french**	**italian**	**german**	**spanish**
house	maison	casa	Haus	casa
chair	chaise	sedia	Sessel	silla
pimple	bouton	foruncolo	Pickel	espenilla



english
sorted

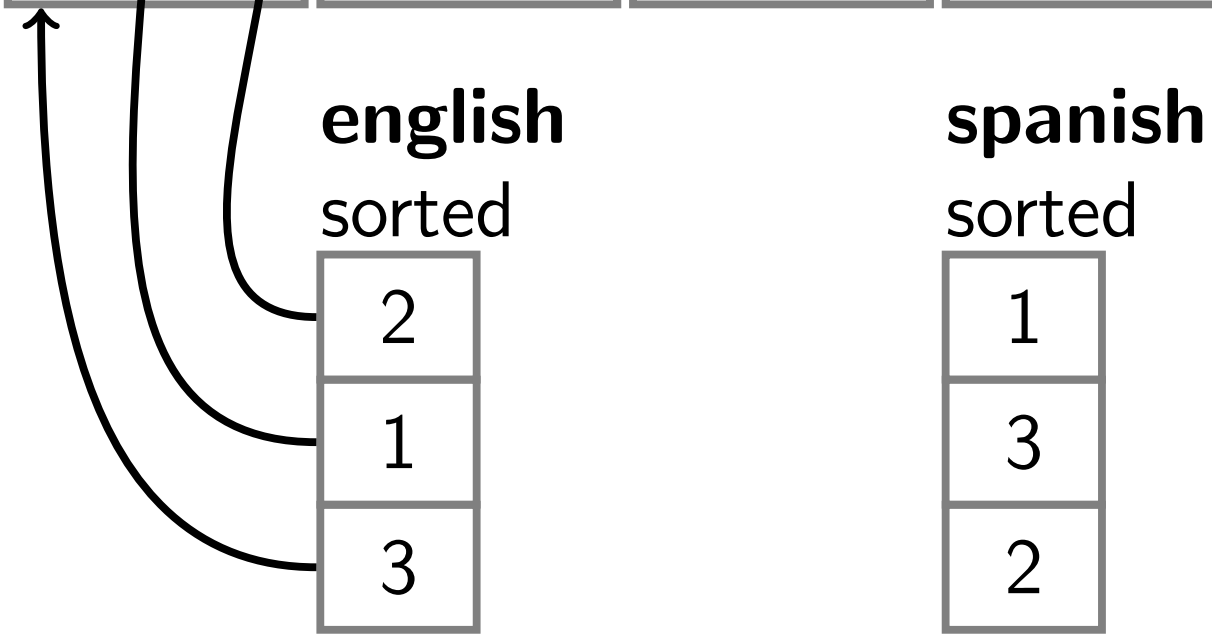
2
1
3

spanish
sorted

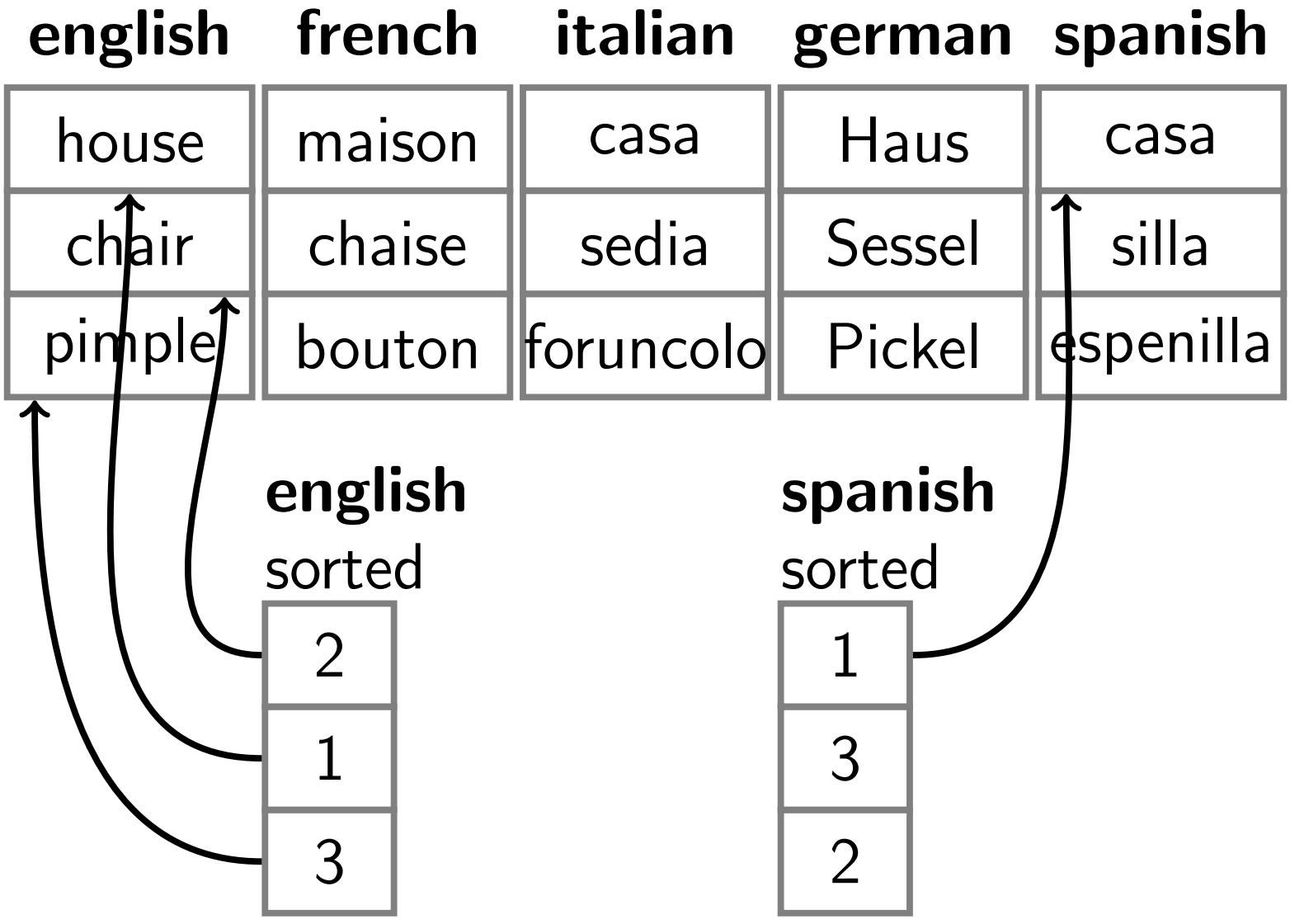
1
3
2

Real-life Example

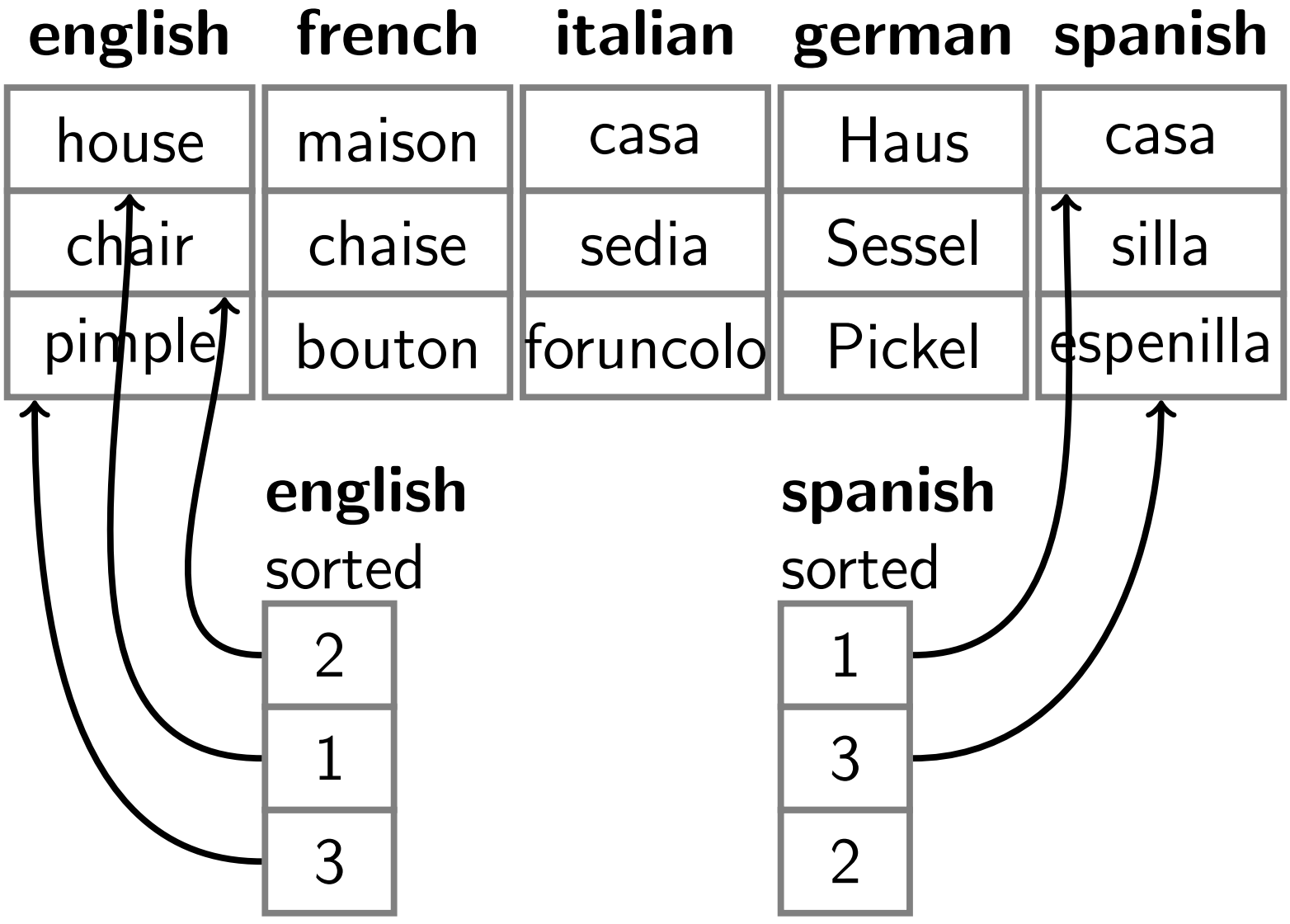
english	**french**	**italian**	**german**	**spanish**
house	maison	casa	Haus	casa
chair	chaise	sedia	Sessel	silla
pimple	bouton	foruncolo	Pickel	espenilla



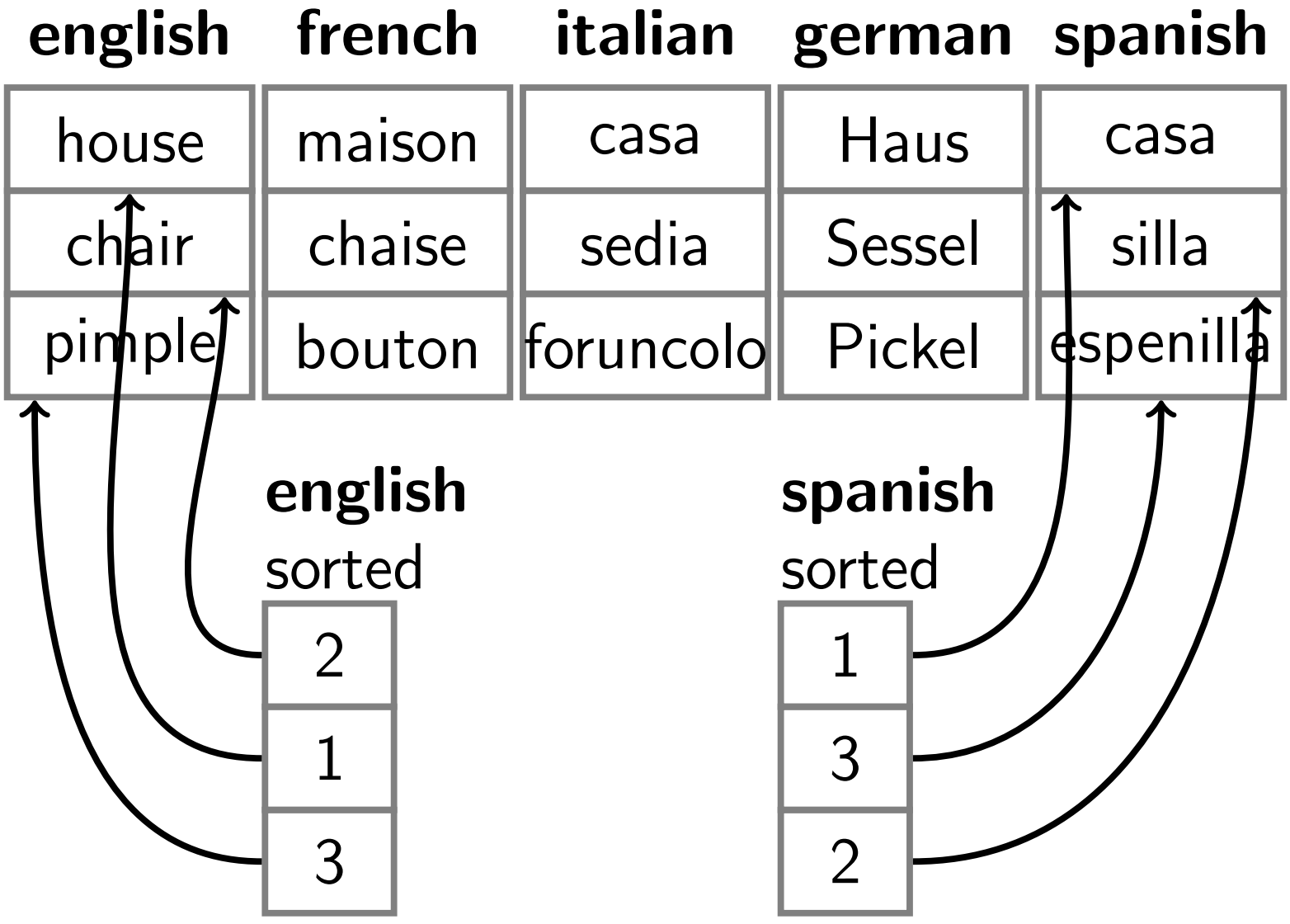
Real-life Example



Real-life Example



Real-life Example



Summary

Summary

The runtime of binary search is $\Theta(\log n)$.

Outline

1. Problem Overview
2. Naïve Algorithm
3. Naïve Divide and Conquer Algorithm
4. Faster Divide and Conquer

Uses of multiplying polynomials

- Error-correcting codes
- Large-integer multiplication
- Generating functions
- Convolution in signal processing

Multiplying Polynomials

Example

Multiplying Polynomials

Example

$$A(x) = 3x^2 + 2x + 5$$

Multiplying Polynomials

Example

$$A(x) = 3x^2 + 2x + 5$$
$$B(x) = 5x^2 + x + 2$$

Multiplying Polynomials

Example

$$A(x) = 3x^2 + 2x + 5$$
$$B(x) = 5x^2 + x + 2$$
$$A(x)B(x) = 15x^4 + 13x^3 + 33x^2 + 9x + 10$$

Multiplying polynomials

Input: Two $n-1$ degree polynomials:

$$a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

$$b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \cdots + b_1 x + b_0$$

Output:

Multiplying polynomials

Input: Two $n-1$ degree polynomials:

$$a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

$$b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \cdots + b_1 x + b_0$$

Output: The product polynomial:

$$c_{2n-2}x^{2n-2} + c_{2n-3}x^{2n-3} + \cdots + c_1 x + c_0$$

Multiplying polynomials

Input: Two $n-1$ degree polynomials:

$$a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

$$b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \cdots + b_1 x + b_0$$

Output: The product polynomial:

$$c_{2n-2}x^{2n-2} + c_{2n-3}x^{2n-3} + \cdots + c_1 x + c_0$$

where:

$$c_{2n-2} = a_{n-1}b_{n-1}$$

Multiplying polynomials

Input: Two $n-1$ degree polynomials:

$$a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

$$b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \cdots + b_1 x + b_0$$

Output: The product polynomial:

$$c_{2n-2}x^{2n-2} + c_{2n-3}x^{2n-3} + \cdots + c_1 x + c_0$$

where:

$$c_{2n-2} = a_{n-1}b_{n-1}$$

$$c_{2n-3} = a_{n-1}b_{n-2} + a_{n-2}b_{n-1}$$

Multiplying polynomials

Input: Two $n-1$ degree polynomials:

$$a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

$$b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \cdots + b_1 x + b_0$$

Output: The product polynomial:

$$c_{2n-2}x^{2n-2} + c_{2n-3}x^{2n-3} + \cdots + c_1 x + c_0$$

where:

$$c_{2n-2} = a_{n-1}b_{n-1}$$

$$c_{2n-3} = a_{n-1}b_{n-2} + a_{n-2}b_{n-1}$$

...

$$c_2 = a_2 b_0 + a_1 b_1 + a_0 b_2$$

Multiplying polynomials

Input: Two $n-1$ degree polynomials:

$$a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

$$b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \cdots + b_1 x + b_0$$

Output: The product polynomial:

$$c_{2n-2}x^{2n-2} + c_{2n-3}x^{2n-3} + \cdots + c_1 x + c_0$$

where:

$$c_{2n-2} = a_{n-1}b_{n-1}$$

$$c_{2n-3} = a_{n-1}b_{n-2} + a_{n-2}b_{n-1}$$

...

$$c_2 = a_2 b_0 + a_1 b_1 + a_0 b_2$$

$$c_1 = a_1 b_0 + a_0 b_1$$

Multiplying polynomials

Input: Two $n-1$ degree polynomials:

$$a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

$$b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \cdots + b_1 x + b_0$$

Output: The product polynomial:

$$c_{2n-2}x^{2n-2} + c_{2n-3}x^{2n-3} + \cdots + c_1 x + c_0$$

where:

$$c_{2n-2} = a_{n-1}b_{n-1}$$

$$c_{2n-3} = a_{n-1}b_{n-2} + a_{n-2}b_{n-1}$$

...

$$c_2 = a_2 b_0 + a_1 b_1 + a_0 b_2$$

$$c_1 = a_1 b_0 + a_0 b_1$$

$$c_0 = a_0 b_0$$

Multiplying Polynomials

Example

Input: $n = 3, A = (3, 2, 5), B = (5, 1, 2)$

Multiplying Polynomials

Example

Input: $n = 3, A = (3, 2, 5), B = (5, 1, 2)$

$$A(x) = 3x^2 + 2x + 5$$

Multiplying Polynomials

Example

Input: $n = 3, A = (3, 2, 5), B = (5, 1, 2)$

$$A(x) = 3x^2 + 2x + 5$$
$$B(x) = 5x^2 + x + 2$$

Multiplying Polynomials

Example

Input: $n = 3, A = (3, 2, 5), B = (5, 1, 2)$

$$A(x) = 3x^2 + 2x + 5$$
$$B(x) = 5x^2 + x + 2$$
$$A(x)B(x) = 15x^4 + 13x^3 + 33x^2 + 9x + 10$$

Multiplying Polynomials

Example

Input: $n = 3, A = (3, 2, 5), B = (5, 1, 2)$

$$A(x) = 3x^2 + 2x + 5$$
$$B(x) = 5x^2 + x + 2$$
$$A(x)B(x) = 15x^4 + 13x^3 + 33x^2 + 9x + 10$$

Output: $C = (15, 13, 33, 9, 10)$

Outline

1. Problem Overview
2. Naïve Algorithm
3. Naïve Divide and Conquer Algorithm
4. Faster Divide and Conquer

MultPoly(A, B, n)

$product \leftarrow Array[2n-1]$

```
for i from 0 to 2n − 2:
```

$\quad product[i] \leftarrow 0$

MultPoly(A, B, n)

$product \leftarrow Array[2n-1]$
`for` i `from` 0 `to` $2n-2$`:`
  $product[i] \leftarrow 0$
`for` i `from` 0 `to` $n-1$`:`
  `for` j `from` 0 `to` $n-1$`:`
    $product[i+j] \leftarrow product[i+j] + A[i] \times B[j]$

MultPoly(A, B, n)

$product \leftarrow Array[2n-1]$
`for` i `from` 0 `to` $2n-2$`:`
  $product[i] \leftarrow 0$
`for` i `from` 0 `to` $n-1$`:`
  `for` j `from` 0 `to` $n-1$`:`
    $product[i+j] \leftarrow product[i+j] + A[i] \times B[j]$
`return` $product$

MultPoly(A, B, n)

```
product ← Array[2n − 1]
for i from 0 to 2n − 2:
  product[i] ← 0
for i from 0 to n − 1:
  for j from 0 to n − 1:
    product[i + j] ← product[i + j] + A[i] × B[j]
return product
```

Runtime: $O(n^2)$

Outline

1. Problem Overview
2. Naïve Algorithm
3. Naïve Divide and Conquer Algorithm
4. Faster Divide and Conquer

Multiplying Polynomials

- Let $A(x) = D_1(x)x^{\frac{n}{2}} + D_0(x)$ where
  $D_1(x) = a_{n-1}x^{\frac{n}{2}-1} + a_{n-2}x^{\frac{n}{2}-2} + \ldots + a_{\frac{n}{2}}$
  $D_0(x) = a_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + a_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + a_0$

Multiplying Polynomials

- Let $A(x) = D_1(x)x^{\frac{n}{2}} + D_0(x)$ where
  $D_1(x) = a_{n-1}x^{\frac{n}{2}-1} + a_{n-2}x^{\frac{n}{2}-2} + \ldots + a_{\frac{n}{2}}$
  $D_0(x) = a_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + a_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + a_0$
- Let $B(x) = E_1(x)x^{\frac{n}{2}} + E_0(x)$ where
  $E_1(x) = b_{n-1}x^{\frac{n}{2}-1} + b_{n-2}x^{\frac{n}{2}-2} + \ldots + b_{\frac{n}{2}}$
  $E_0(x) = b_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + b_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + b_0$

Multiplying Polynomials

- Let $A(x) = D_1(x)x^{\frac{n}{2}} + D_0(x)$ where
  $D_1(x) = a_{n-1}x^{\frac{n}{2}-1} + a_{n-2}x^{\frac{n}{2}-2} + \ldots + a_{\frac{n}{2}}$
  $D_0(x) = a_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + a_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + a_0$
- Let $B(x) = E_1(x)x^{\frac{n}{2}} + E_0(x)$ where
  $E_1(x) = b_{n-1}x^{\frac{n}{2}-1} + b_{n-2}x^{\frac{n}{2}-2} + \ldots + b_{\frac{n}{2}}$
  $E_0(x) = b_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + b_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + b_0$
- $AB = (D_1x^{\frac{n}{2}} + D_0)(E_1x^{\frac{n}{2}} + E_0)$
  $= (D_1E_1)x^n + (D_1E_0 + D_0E_1)x^{\frac{n}{2}} + D_0E_0$

Multiplying Polynomials

- Let $A(x) = D_1(x)x^{\frac{n}{2}} + D_0(x)$ where
  $D_1(x) = a_{n-1}x^{\frac{n}{2}-1} + a_{n-2}x^{\frac{n}{2}-2} + \ldots + a_{\frac{n}{2}}$
  $D_0(x) = a_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + a_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + a_0$
- Let $B(x) = E_1(x)x^{\frac{n}{2}} + E_0(x)$ where
  $E_1(x) = b_{n-1}x^{\frac{n}{2}-1} + b_{n-2}x^{\frac{n}{2}-2} + \ldots + b_{\frac{n}{2}}$
  $E_0(x) = b_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + b_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + b_0$
- $AB = (D_1x^{\frac{n}{2}} + D_0)(E_1x^{\frac{n}{2}} + E_0)$
  $= (D_1E_1)x^n + (D_1E_0 + D_0E_1)x^{\frac{n}{2}} + D_0E_0$
- Calculate D_1E_1, D_1E_0, D_0E_1, and D_0E_0

Multiplying Polynomials

- Let $A(x) = D_1(x)x^{\frac{n}{2}} + D_0(x)$ where
  $D_1(x) = a_{n-1}x^{\frac{n}{2}-1} + a_{n-2}x^{\frac{n}{2}-2} + \ldots + a_{\frac{n}{2}}$
  $D_0(x) = a_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + a_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + a_0$
- Let $B(x) = E_1(x)x^{\frac{n}{2}} + E_0(x)$ where
  $E_1(x) = b_{n-1}x^{\frac{n}{2}-1} + b_{n-2}x^{\frac{n}{2}-2} + \ldots + b_{\frac{n}{2}}$
  $E_0(x) = b_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + b_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + b_0$
- $AB = (D_1x^{\frac{n}{2}} + D_0)(E_1x^{\frac{n}{2}} + E_0)$
  $= (D_1E_1)x^n + (D_1E_0 + D_0E_1)x^{\frac{n}{2}} + D_0E_0$
- Calculate D_1E_1, D_1E_0, D_0E_1, and D_0E_0

Multiplying Polynomials

- Let $A(x) = D_1(x)x^{\frac{n}{2}} + D_0(x)$ where
  $D_1(x) = a_{n-1}x^{\frac{n}{2}-1} + a_{n-2}x^{\frac{n}{2}-2} + \ldots + a_{\frac{n}{2}}$
  $D_0(x) = a_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + a_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + a_0$
- Let $B(x) = E_1(x)x^{\frac{n}{2}} + E_0(x)$ where
  $E_1(x) = b_{n-1}x^{\frac{n}{2}-1} + b_{n-2}x^{\frac{n}{2}-2} + \ldots + b_{\frac{n}{2}}$
  $E_0(x) = b_{\frac{n}{2}-1}x^{\frac{n}{2}-1} + b_{\frac{n}{2}-2}x^{\frac{n}{2}-2} + \ldots + b_0$
- $AB = (D_1x^{\frac{n}{2}} + D_0)(E_1x^{\frac{n}{2}} + E_0)$
  $= (D_1E_1)x^n + (D_1E_0 + D_0E_1)x^{\frac{n}{2}} + D_0E_0$
- Calculate D_1E_1, D_1E_0, D_0E_1, and D_0E_0

Recurrence: $T(n) = 4T(\frac{n}{2}) + kn$.

Polynomial Mult: Divide & Conquer

$A(x) = 4x^3 + 3x^2 + 2x + 1$

$B(x) = x^3 + 2x^2 + 3x + 4$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1 E_1 = 4x^2 + 11x + 6$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$AB =$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$(\qquad\qquad\qquad\qquad\qquad\qquad)x^2 +$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$(12x^2 + 25x + 12 \qquad)x^2 +$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$(12x^2 + 25x + 12 + 2x^2 + 5x + 2)x^2 +$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$AB = (4x^2 + 11x + 6)x^4 +$$
$$(12x^2 + 25x + 12 + 2x^2 + 5x + 2)x^2 +$$
$$6x^2 + 11x + 4$$

Polynomial Mult: Divide & Conquer

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_1E_0 = 12x^2 + 25x + 12$$

$$D_0E_1 = 2x^2 + 5x + 2 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$\begin{aligned} AB = &(4x^2 + 11x + 6)x^4 + \\ &(12x^2 + 25x + 12 + 2x^2 + 5x + 2)x^2 + \\ &6x^2 + 11x + 4 \\ =&4x^6 + 11x^5 + 20x^4 + 30x^3 + 20x^2 + 11x + 4 \end{aligned}$$

Function Mult2(A, B, n, a_l, b_l)

Function Mult2(A, B, n, a_l, b_l)

$R =$ array$[0..2n-1]$

Function Mult2(A, B, n, a_l, b_l)

$R =$ `array`$[0..2n-1]$

`if` $n = 1$:

$\quad R[0] = A[a_l] * B[b_l]$; `return` R

Function Mult2(A, B, n, a_l, b_l)

$R =$ array$[0..2n-1]$

if $n = 1$:

$R[0] = A[a_l] * B[b_l]$; return R

$R[0..n-2]$ = Mult2$(A, B, \frac{n}{2}, a_l, b_l)$

Function Mult2(A, B, n, a_l, b_l)

$R =$ array$[0..2n-1]$

if $n = 1$:

$R[0] = A[a_l] * B[b_l]$; return R

$R[0..n-2]$ = Mult2$(A, B, \frac{n}{2}, a_l, b_l)$

$R[n..2n-2]$ = Mult2$(A, B, \frac{n}{2}, a_l + \frac{n}{2}, b_l + \frac{n}{2})$

Function Mult2(A, B, n, a_l, b_l)

$R =$ array$[0..2n-1]$

if $n = 1$:

$R[0] = A[a_l] * B[b_l]$; return R

$R[0..n-2]$ = Mult2$(A, B, \frac{n}{2}, a_l, b_l)$

$R[n..2n-2]$ = Mult2$(A, B, \frac{n}{2}, a_l + \frac{n}{2}, b_l + \frac{n}{2})$

D_0E_1 = Mult2$(A, B, \frac{n}{2}, a_l, b_l + \frac{n}{2})$

Function Mult2(A, B, n, a_l, b_l)

$R =$ array$[0..2n-1]$

if $n = 1$:

$R[0] = A[a_l] * B[b_l]$; return R

$R[0..n-2]$ = Mult2$(A, B, \frac{n}{2}, a_l, b_l)$

$R[n..2n-2]$ = Mult2$(A, B, \frac{n}{2}, a_l + \frac{n}{2}, b_l + \frac{n}{2})$

D_0E_1 = Mult2$(A, B, \frac{n}{2}, a_l, b_l + \frac{n}{2})$

D_1E_0 = Mult2$(A, B, \frac{n}{2}, a_l + \frac{n}{2}, b_l)$

$R[\frac{n}{2}...n + \frac{n}{2} - 2]$ += $D_1E0 + D_0E_1$

Function Mult2(A, B, n, a_l, b_l)

$R =$ `array[0..`$2n-1$`]`

`if` $n = 1$`:`

$\quad R[0] = A[a_l] * B[b_l]$ `; return` R

R`[0..`$n-2$`] = Mult2(`$A, B, \frac{n}{2}, a_l, b_l$`)`

R`[`n`..`$2n-2$`] = Mult2(`$A, B, \frac{n}{2}, a_l + \frac{n}{2}, b_l + \frac{n}{2}$`)`

D_0E_1 `= Mult2(`$A, B, \frac{n}{2}, a_l, b_l + \frac{n}{2}$`)`

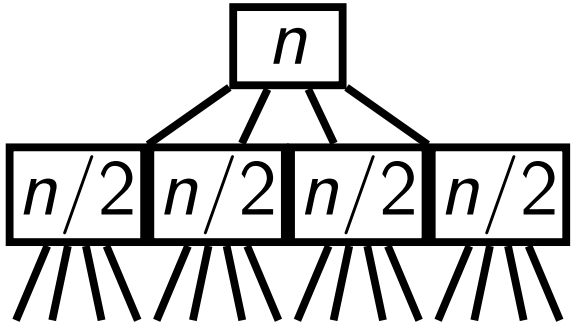
D_1E_0 `= Mult2(`$A, B, \frac{n}{2}, a_l + \frac{n}{2}, b_l$`)`

$R[\frac{n}{2} \ldots n + \frac{n}{2} - 2]$ `+=` $D_1E0 + D_0E_1$

`return` R


n

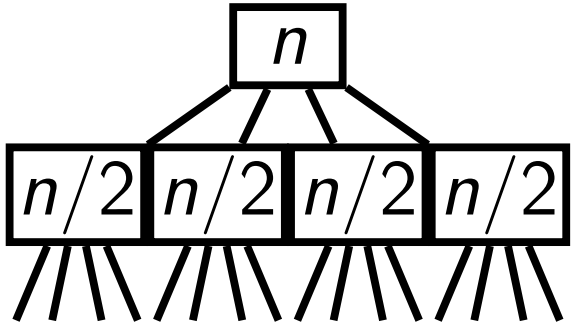
level

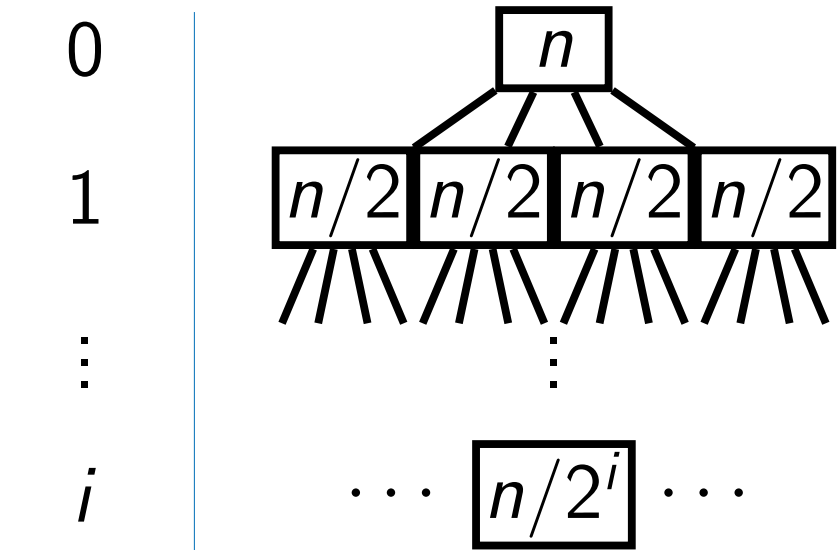


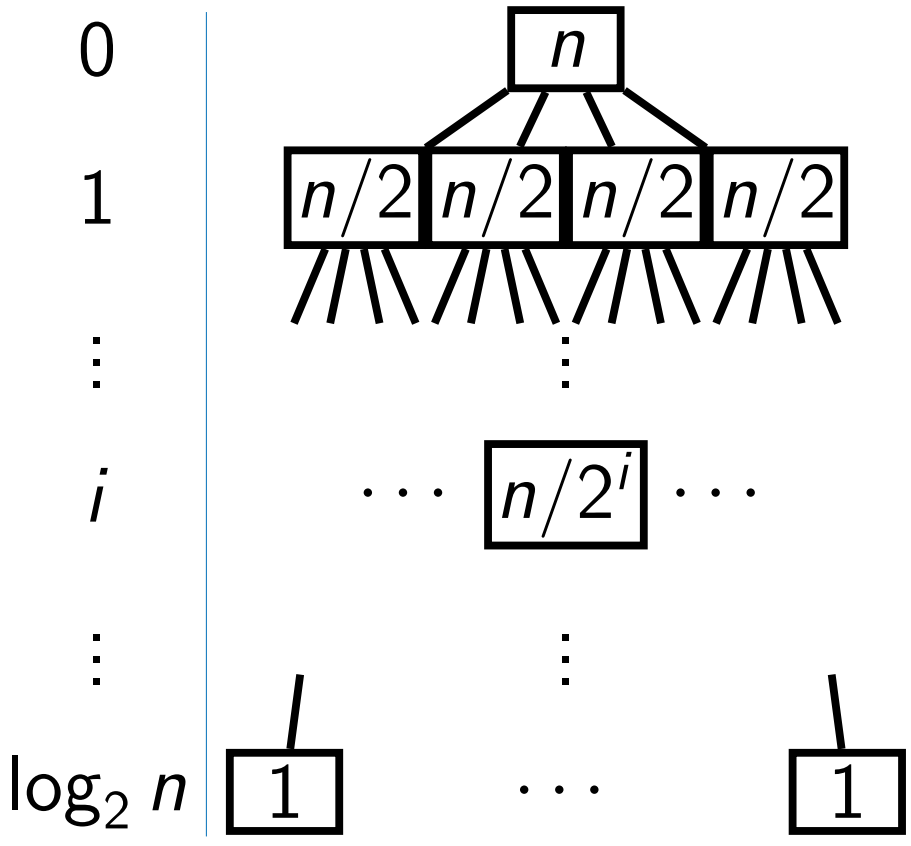
level

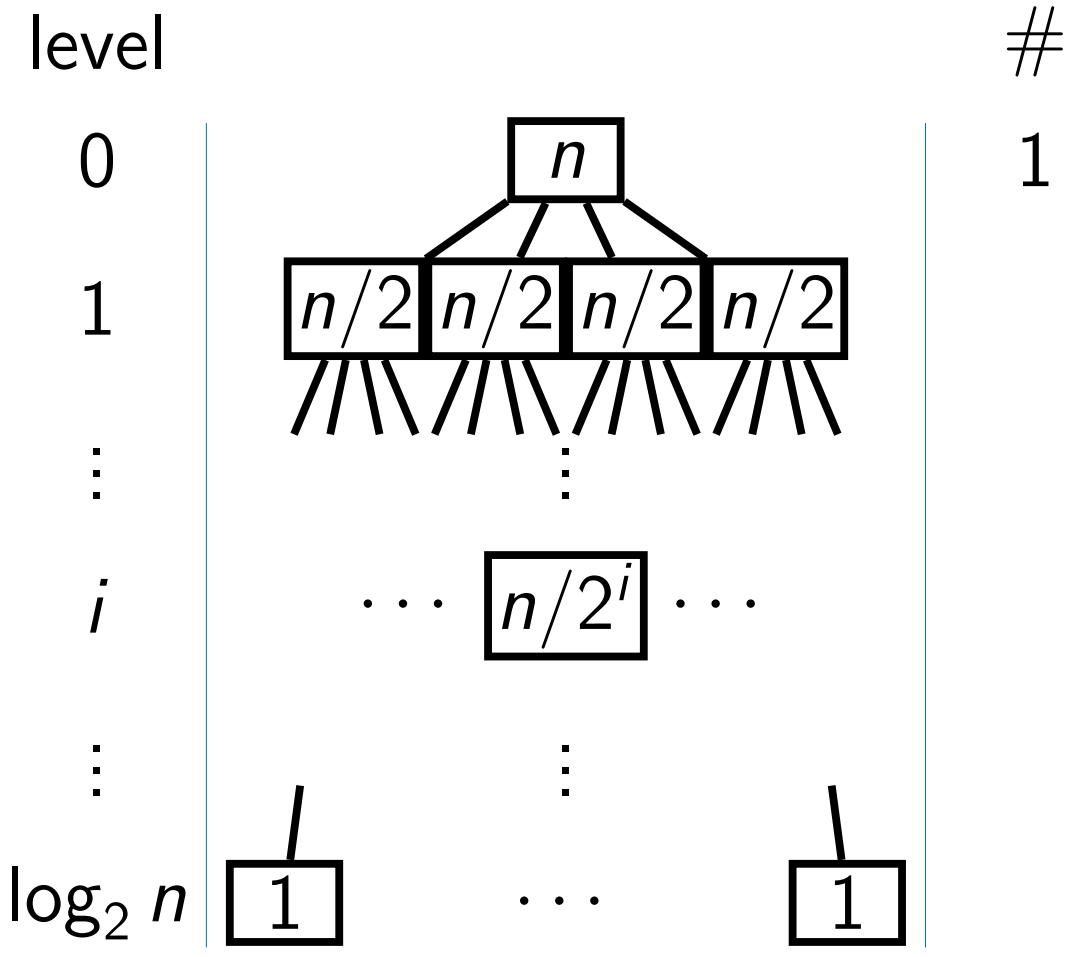
0

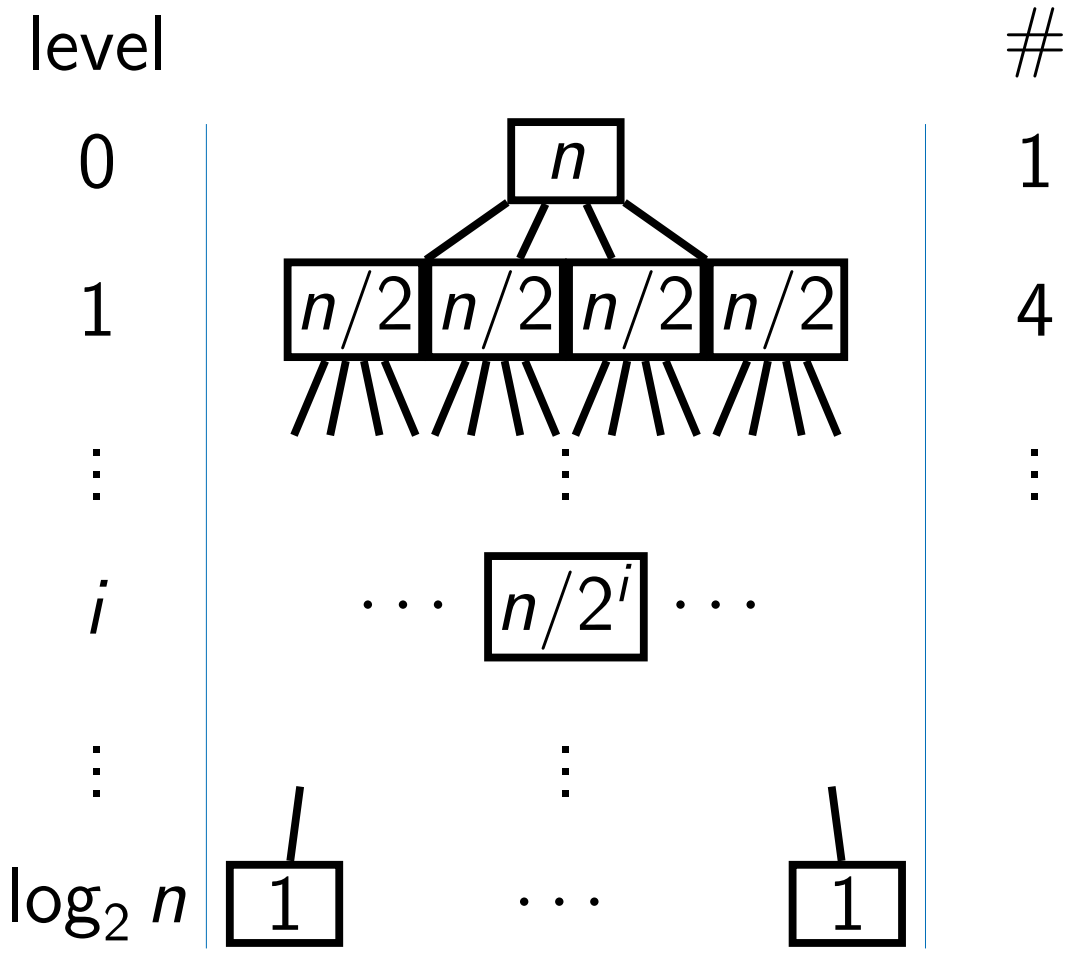
1

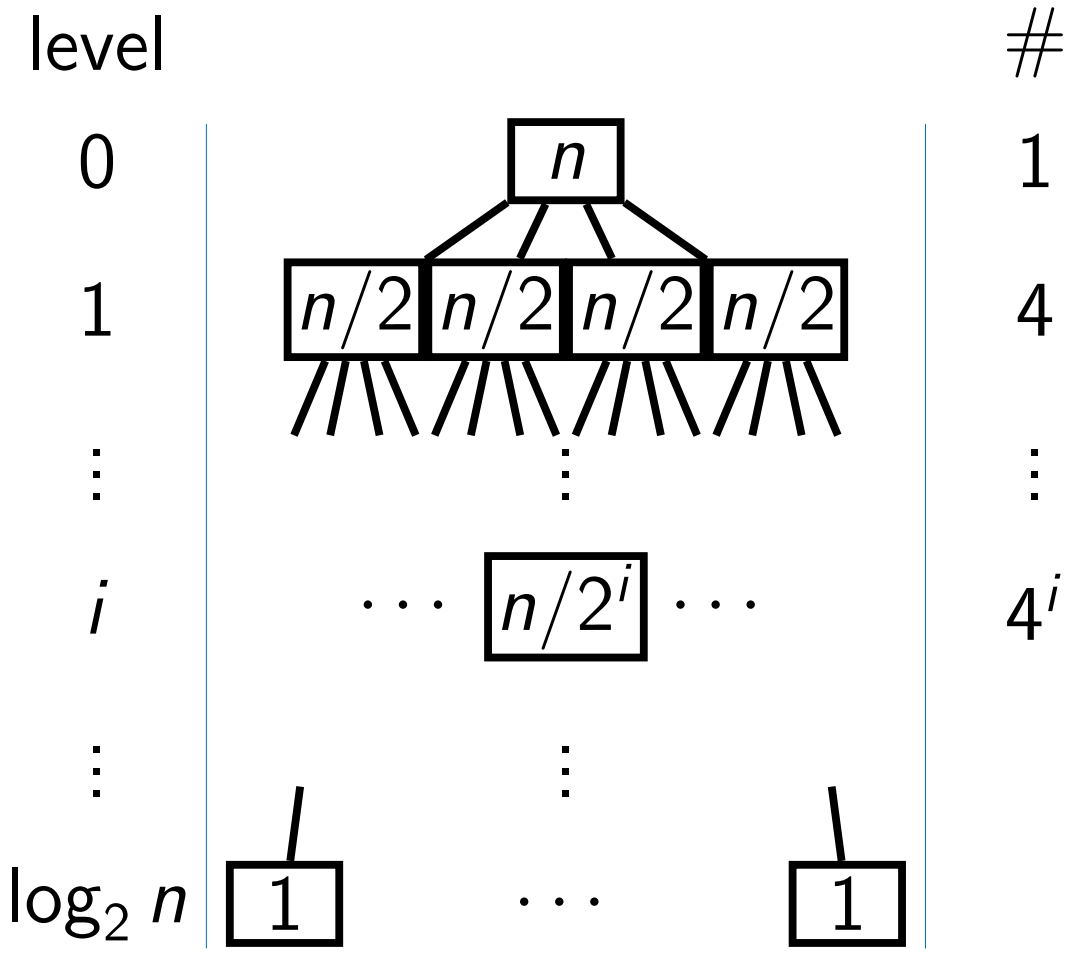


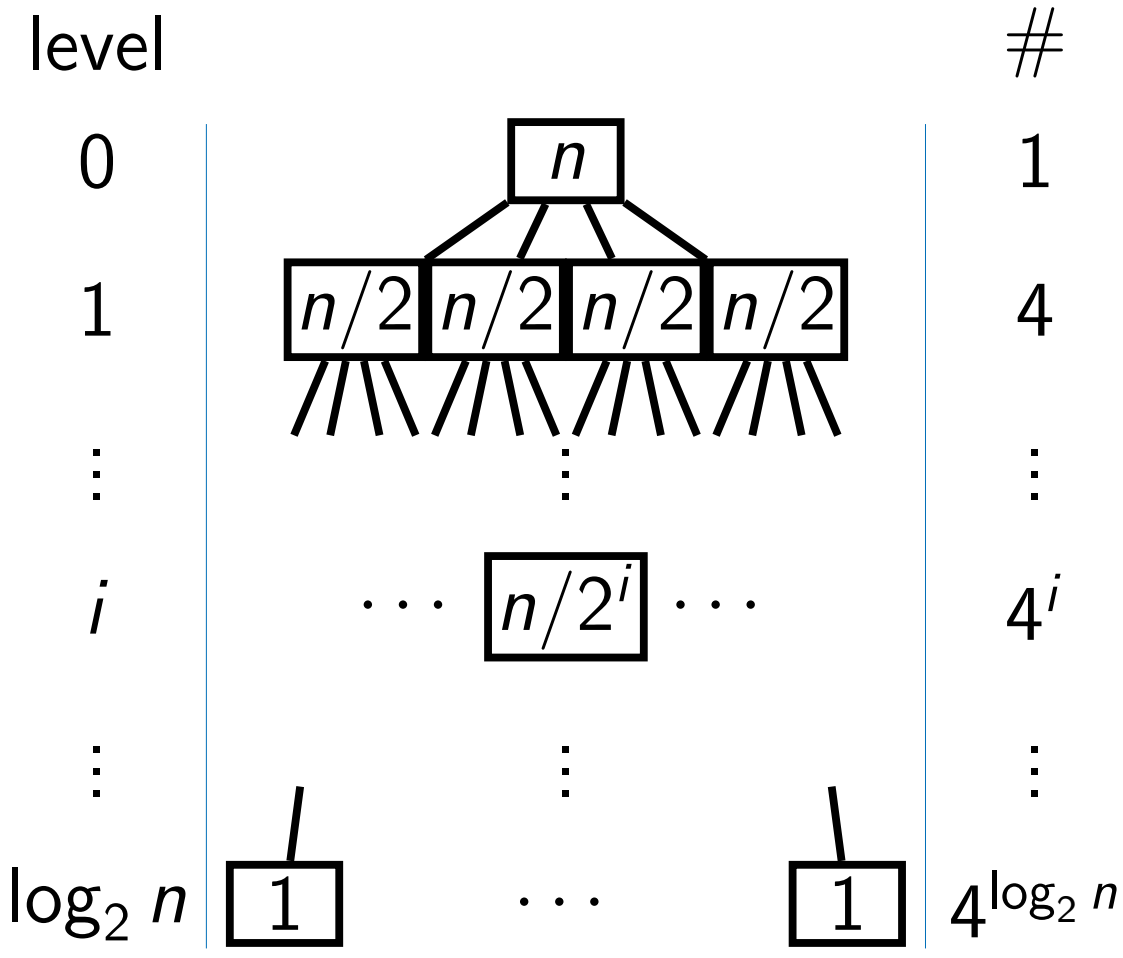

level
0
1
⋮
i
n
n/2
n/2
n/2
n/2
⋮
· · ·
n/2^i
· · ·

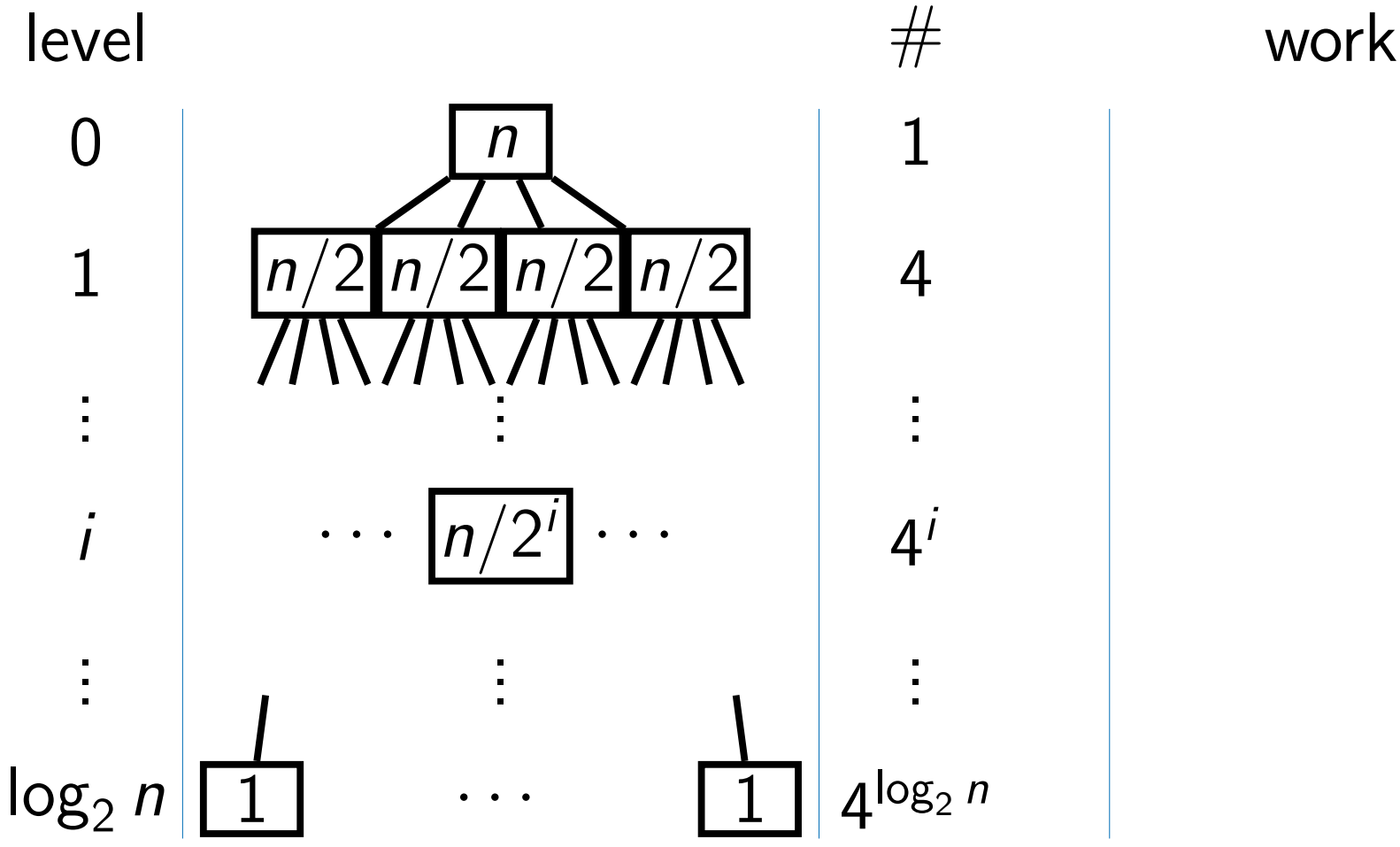

level
0
1
i
log2 n
n
n/2
n/2
n/2
n/2
n/2^i
1
1

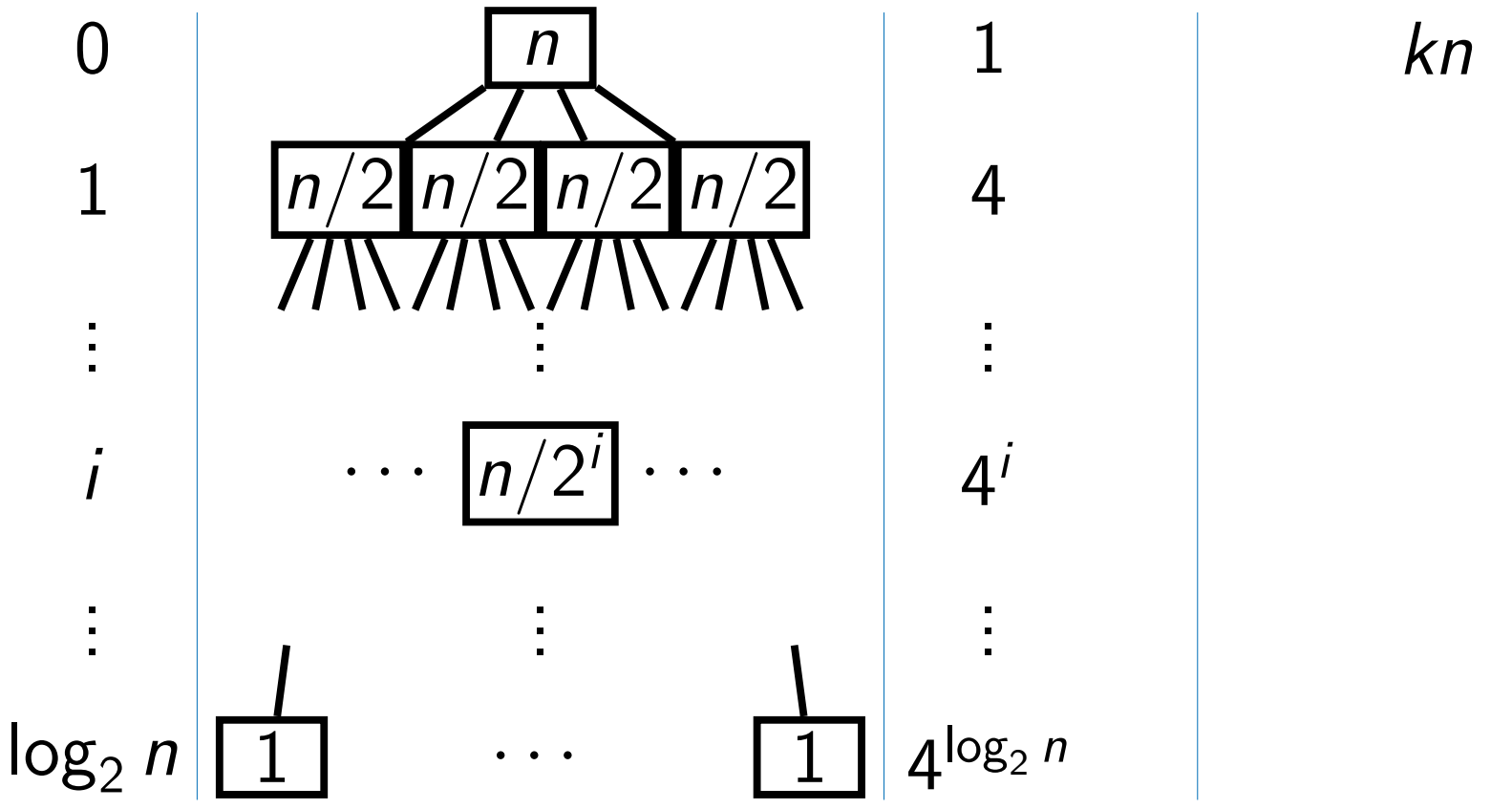

level
#
0
1
i
$\log_2 n$
n
$n/2$
$n/2$
$n/2$
$n/2$
$n/2^i$
1
1
1

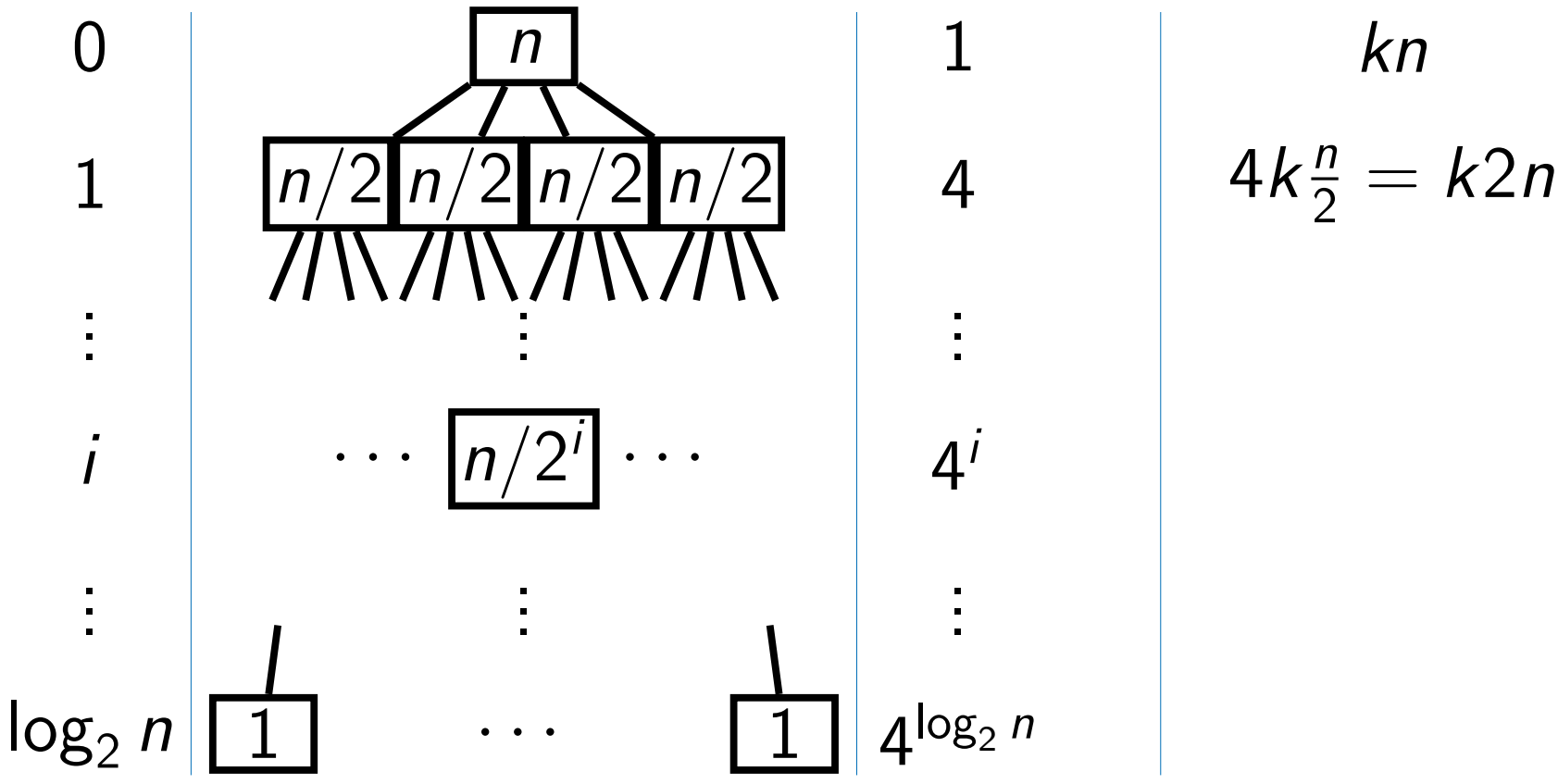

level
#
0
1
n
1
n/2
n/2
n/2
n/2
4
i
n/2^i
log2 n
1
1

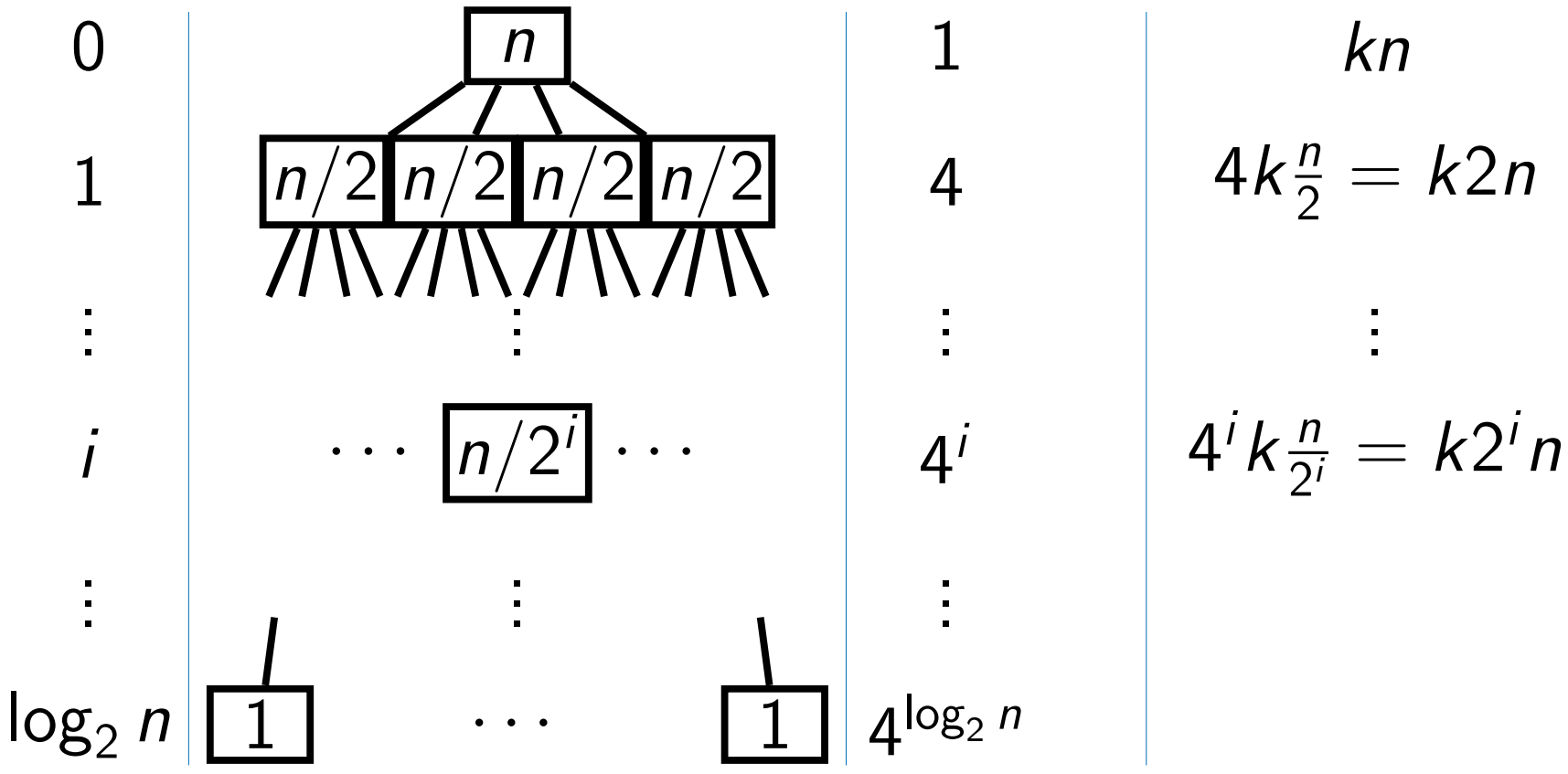

level
#
0
1
n
$n/2$
$n/2$
$n/2$
$n/2$
1
4
i
$\cdots$
$n/2^i$
$\cdots$
4^i
$\log_2 n$
1
$\cdots$
1

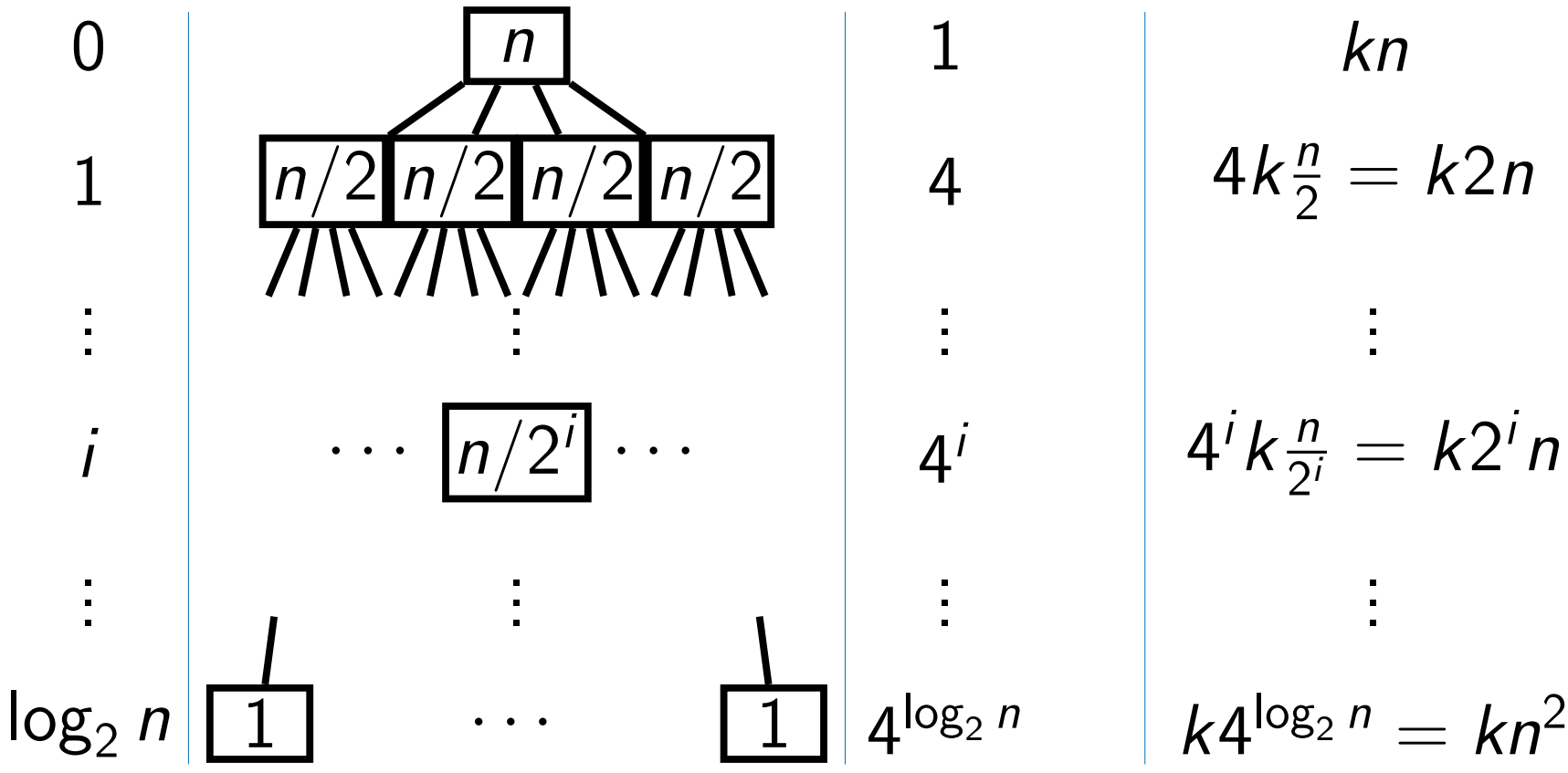

level
#
0
1
n
n/2
n/2
n/2
n/2
1
4
i
n/2^i
4^i
log_2 n
1
1
4^log_2 n

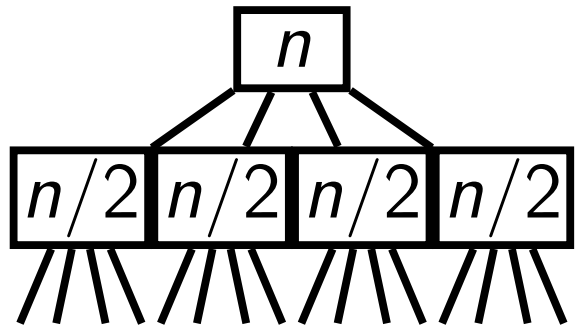

level
#
work
0
1
i
$\log_2 n$
n
$n/2$ $n/2$ $n/2$ $n/2$
$\cdots$ $n/2^i$ $\cdots$
1
$\cdots$
1
1
4
4^i
$4^{\log_2 n}$


level
#
work
0
1
kn
n
1
n/2
n/2
n/2
n/2
4
i
n/2^i
4^i
log_2 n
1
1
4^log_2 n


level
#
work
0
1
$\log_2 n$
n
$n/2$ $n/2$ $n/2$ $n/2$
$n/2^i$
1
1
$\cdots$
$\cdots$
$\cdots$
$\cdots$
$\cdots$
$\cdots$
$\cdots$
1
4
4^i
$4^{\log_2 n}$
kn
$4k\frac{n}{2} = k2n$


level
#
work
0
1
kn
1
4
$4k\frac{n}{2} = k2n$
i
4^i
$4^i k\frac{n}{2^i} = k2^i n$
$\log_2 n$
$4^{\log_2 n}$
n
$n/2$
$n/2$
$n/2$
$n/2$
$n/2^i$
1
1


level
#
work
0
1
kn
1
4
$4k\frac{n}{2} = k2n$
i
4^i
$4^i k\frac{n}{2^i} = k2^i n$
$\log_2 n$
$4^{\log_2 n}$
$k4^{\log_2 n} = kn^2$
n
$n/2$
$n/2$
$n/2$
$n/2$
$\cdots$
$n/2^i$
$\cdots$
1
$\cdots$
1



level	#	work
0	1	kn
1	4	$4k\frac{n}{2} = k2n$
$\vdots$	$\vdots$	$\vdots$
i	4^i	$4^i k\frac{n}{2^i} = k2^i n$
$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	$4^{\log_2 n}$	$k4^{\log_2 n} = kn^2$

Total: $\sum_{i=0}^{\log_2 n} 4^i k\frac{n}{2^i} = \Theta(n^2)$

Outline

1. Problem Overview
2. Naïve Algorithm
3. Naïve Divide and Conquer Algorithm
4. Faster Divide and Conquer

Karatsuba approach

Karatsuba approach

$$A(x) = a_1 x + a_0$$

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

$$C(x) = a_1 b_1 x^2 + (a_1 b_0 + a_0 b_1) x + a_0 b_0$$

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

$$C(x) = a_1 b_1 x^2 + (a_1 b_0 + a_0 b_1)x + a_0 b_0$$

Needs 4 multiplications

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

$$C(x) = a_1 b_1 x^2 + (a_1 b_0 + a_0 b_1) x + a_0 b_0$$

Needs 4 multiplications

Rewrite as:

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

$$C(x) = a_1 b_1 x^2 + (a_1 b_0 + a_0 b_1)x + a_0 b_0$$

Needs 4 multiplications

Rewrite as:

$$C(x) = a_1 b_1 x^2 + ((a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0)x + a_0 b_0$$

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

$$C(x) = a_1 b_1 x^2 + (a_1 b_0 + a_0 b_1)x + a_0 b_0$$

Needs 4 multiplications

Rewrite as:

$$C(x) = a_1 b_1 x^2 + ((a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0)x + a_0 b_0$$

Needs 3 multiplications

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

$$C(x) = a_1 b_1 x^2 + (a_1 b_0 + a_0 b_1)x + a_0 b_0$$

Needs 4 multiplications

Rewrite as:

$$C(x) = a_1 b_1 x^2 + ((a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0)x + a_0 b_0$$

Needs 3 multiplications

Karatsuba approach

$$A(x) = a_1 x + a_0$$

$$B(x) = b_1 x + b_0$$

$$C(x) = a_1 b_1 x^2 + (a_1 b_0 + a_0 b_1)x + a_0 b_0$$

Needs 4 multiplications

Rewrite as:

$$C(x) = a_1 b_1 x^2 + ((a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0)x + a_0 b_0$$

Needs 3 multiplications

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) =$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)(4x + 6)$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$\begin{aligned}(D_1 + D_0)(E_1 + E_0) &= (6x + 4)(4x + 6) \\ &= 24x^2 + 52x + 24\end{aligned}$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)(4x + 6)$$

$$= 24x^2 + 52x + 24$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)(4x + 6)$$

$$= 24x^2 + 52x + 24$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$($$

$$)x^2 +$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)(4x + 6)$$

$$= 24x^2 + 52x + 24$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$(24x^2 + 52x + 24$$

$$)x^2 +$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)(4x + 6)$$

$$= 24x^2 + 52x + 24$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$(24x^2 + 52x + 24 - (4x^2 + 11x + 6)$$

$$)x^2 +$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)(4x + 6)$$

$$= 24x^2 + 52x + 24$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$(24x^2 + 52x + 24 - (4x^2 + 11x + 6)$$

$$- (6x^2 + 11x + 4))x^2 +$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$(D_1 + D_0)(E_1 + E_0) = (6x + 4)(4x + 6)$$

$$= 24x^2 + 52x + 24$$

$$AB = (4x^2 + 11x + 6)x^4 +$$

$$(24x^2 + 52x + 24 - (4x^2 + 11x + 6)$$

$$- (6x^2 + 11x + 4))x^2 +$$

$$6x^2 + 11x + 4$$

Karatsuba Example

$$A(x) = 4x^3 + 3x^2 + 2x + 1$$

$$B(x) = x^3 + 2x^2 + 3x + 4$$

$$D_1(x) = 4x + 3 \qquad D_0(x) = 2x + 1$$

$$E_1(x) = x + 2 \qquad E_0(x) = 3x + 4$$

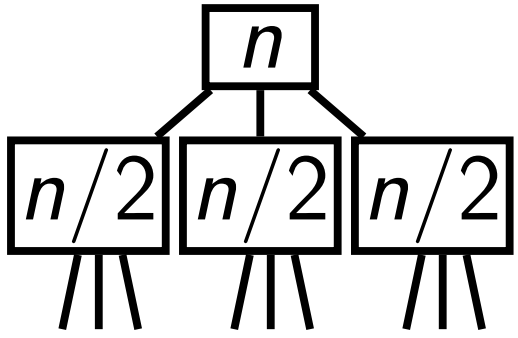
$$D_1E_1 = 4x^2 + 11x + 6 \qquad D_0E_0 = 6x^2 + 11x + 4$$

$$\begin{aligned}(D_1 + D_0)(E_1 + E_0) &= (6x + 4)(4x + 6) \\ &= 24x^2 + 52x + 24\end{aligned}$$

$$\begin{aligned}AB = &(4x^2 + 11x + 6)x^4 + \\ &(24x^2 + 52x + 24 - (4x^2 + 11x + 6) \\ &\qquad - (6x^2 + 11x + 4))x^2 + \\ &6x^2 + 11x + 4 \\ =&4x^6 + 11x^5 + 20x^4 + 30x^3 + 20x^2 + 11x + 4\end{aligned}$$


n

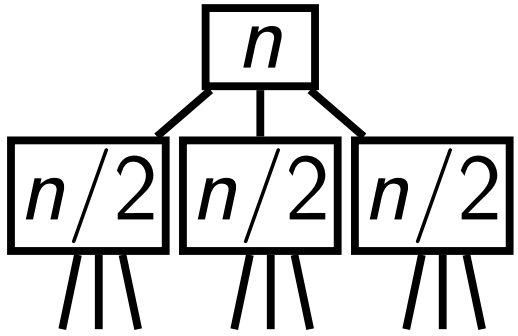
level

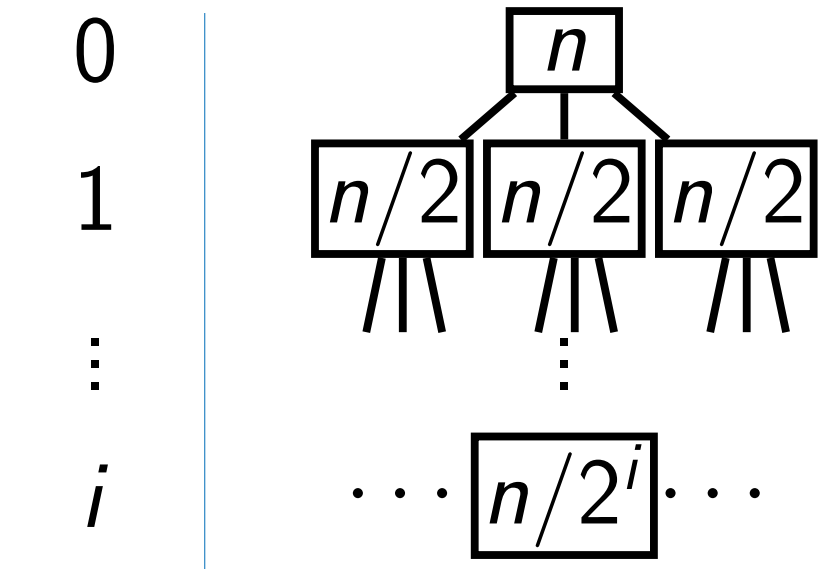


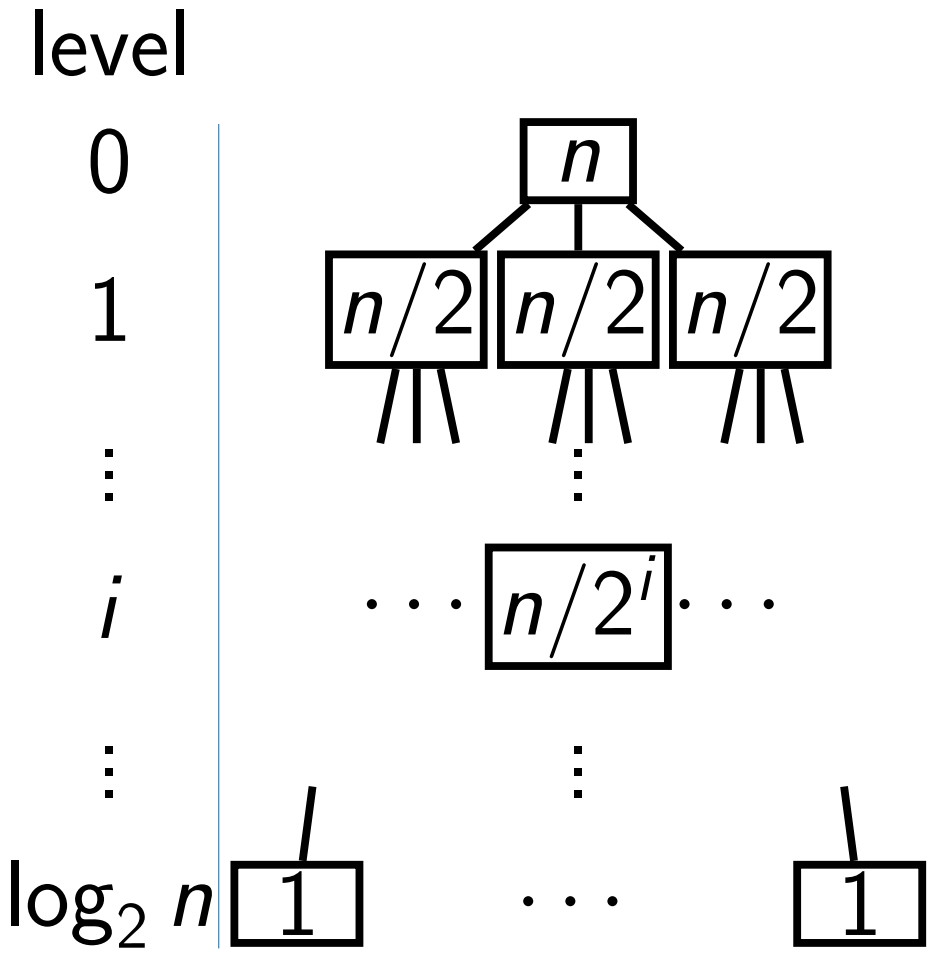
level

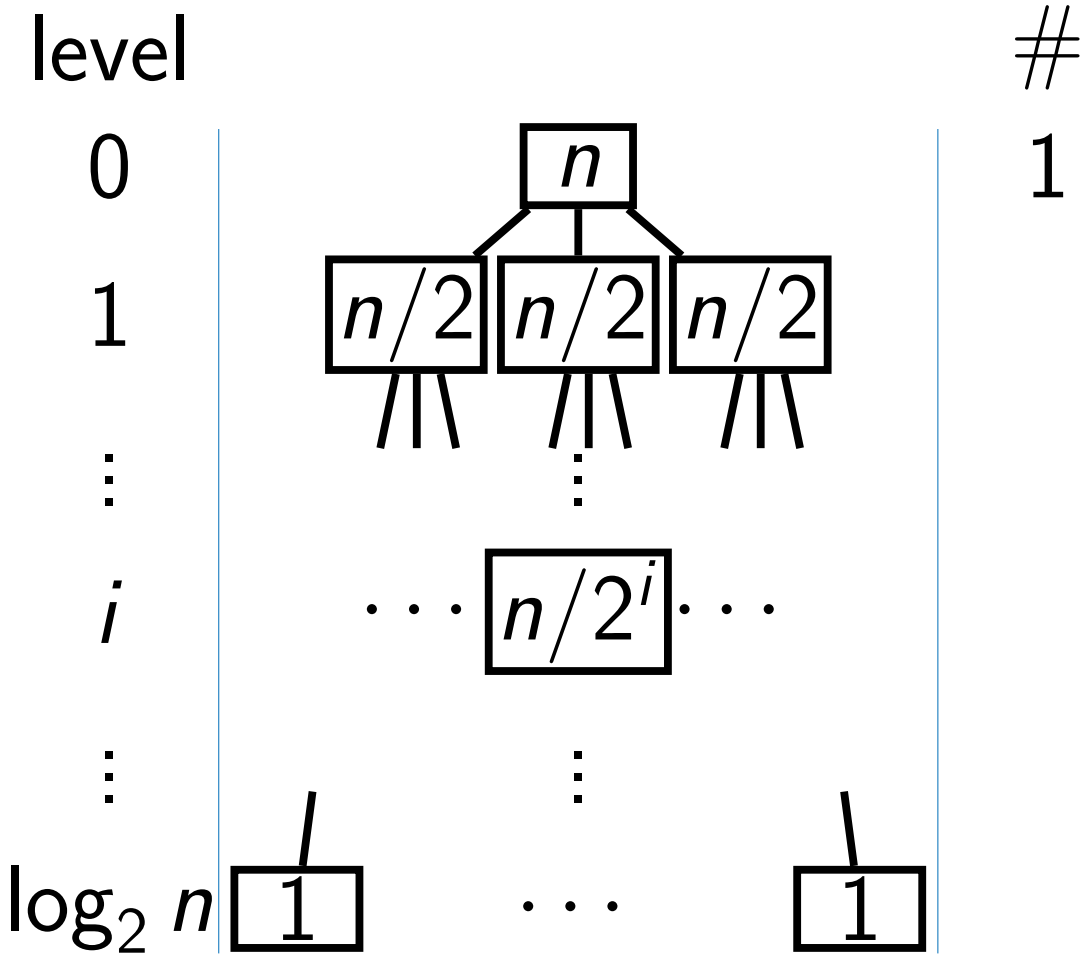
0

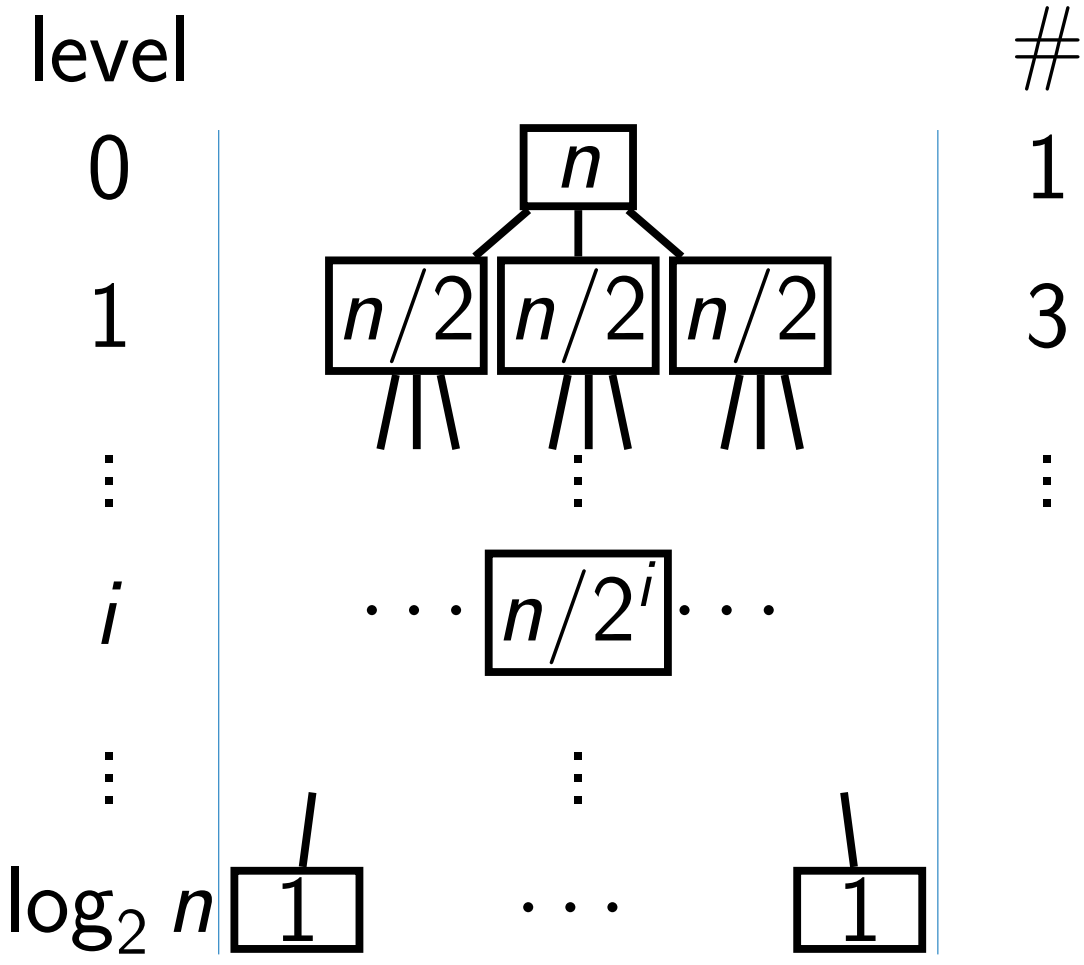
1

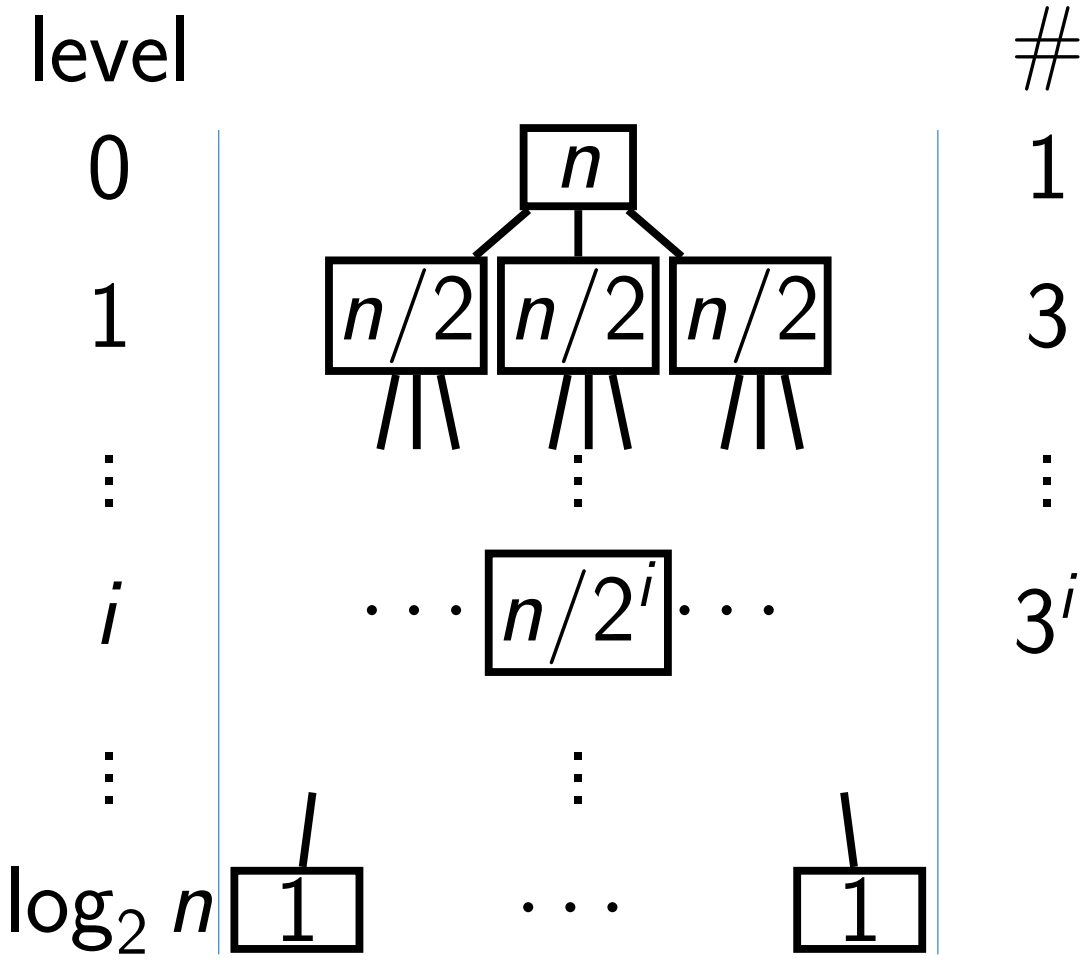


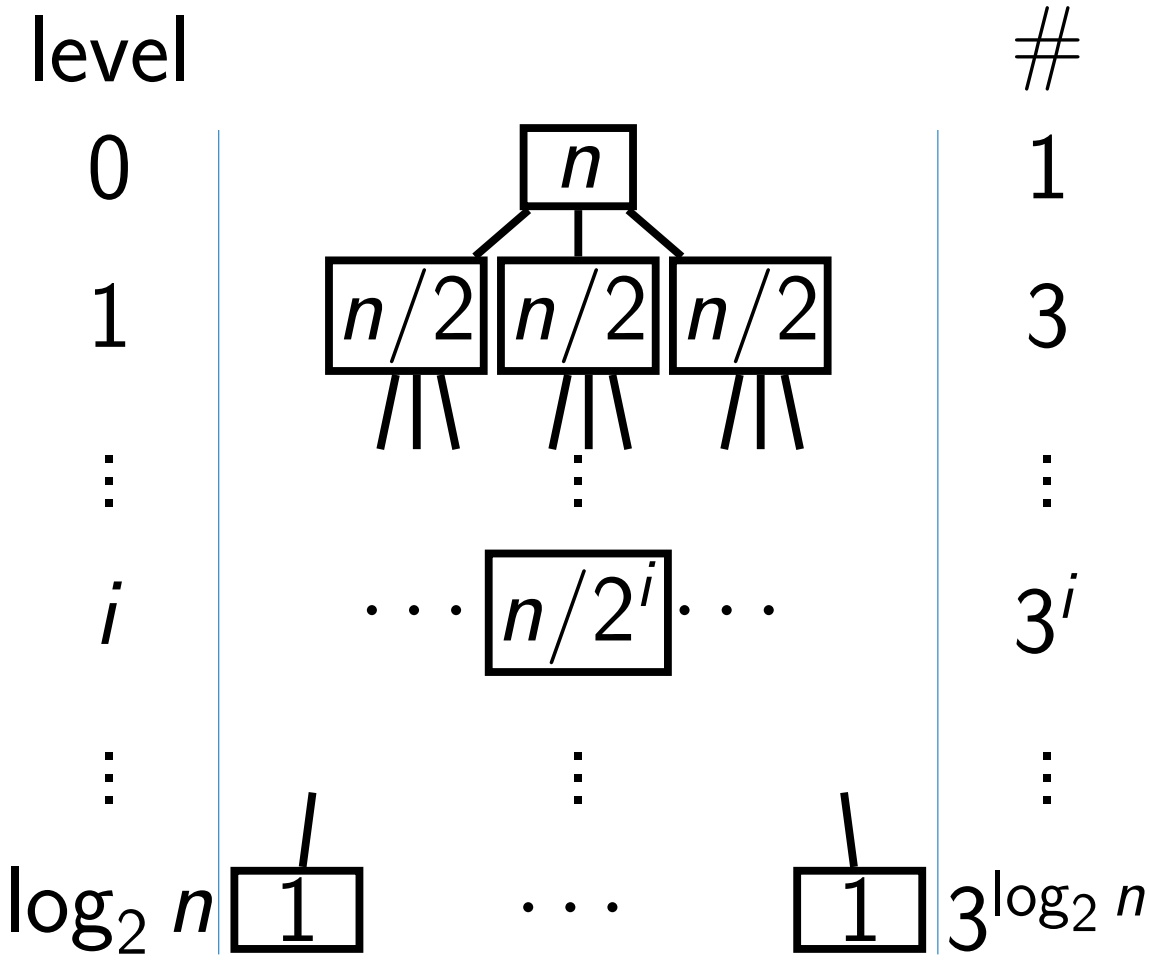

level
0
1
⋮
i
n
n/2
n/2
n/2
⋮
· · ·
n/2^i
· · ·

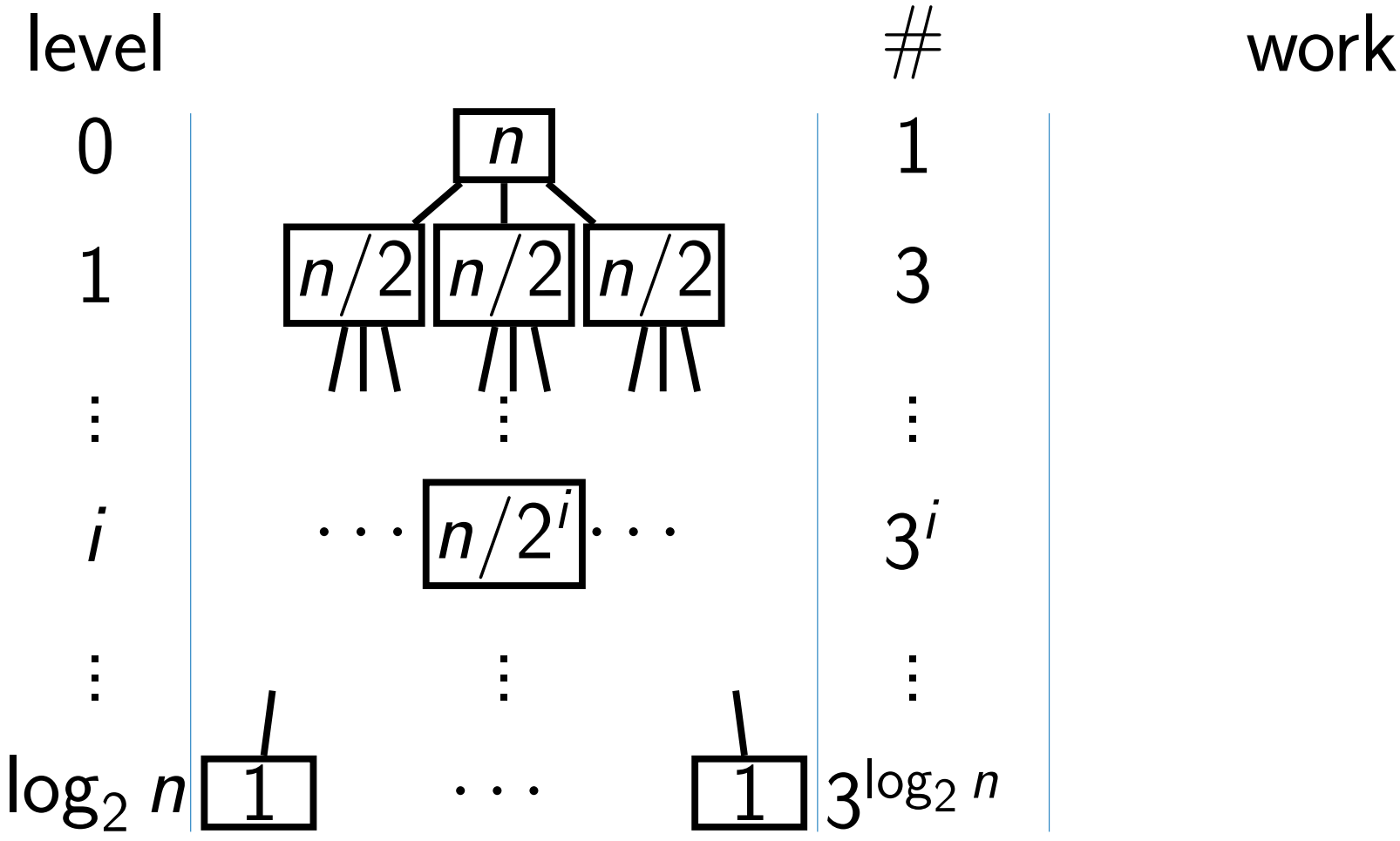

level
0
1
i
log2 n
n
n/2
n/2
n/2
n/2^i
1
1

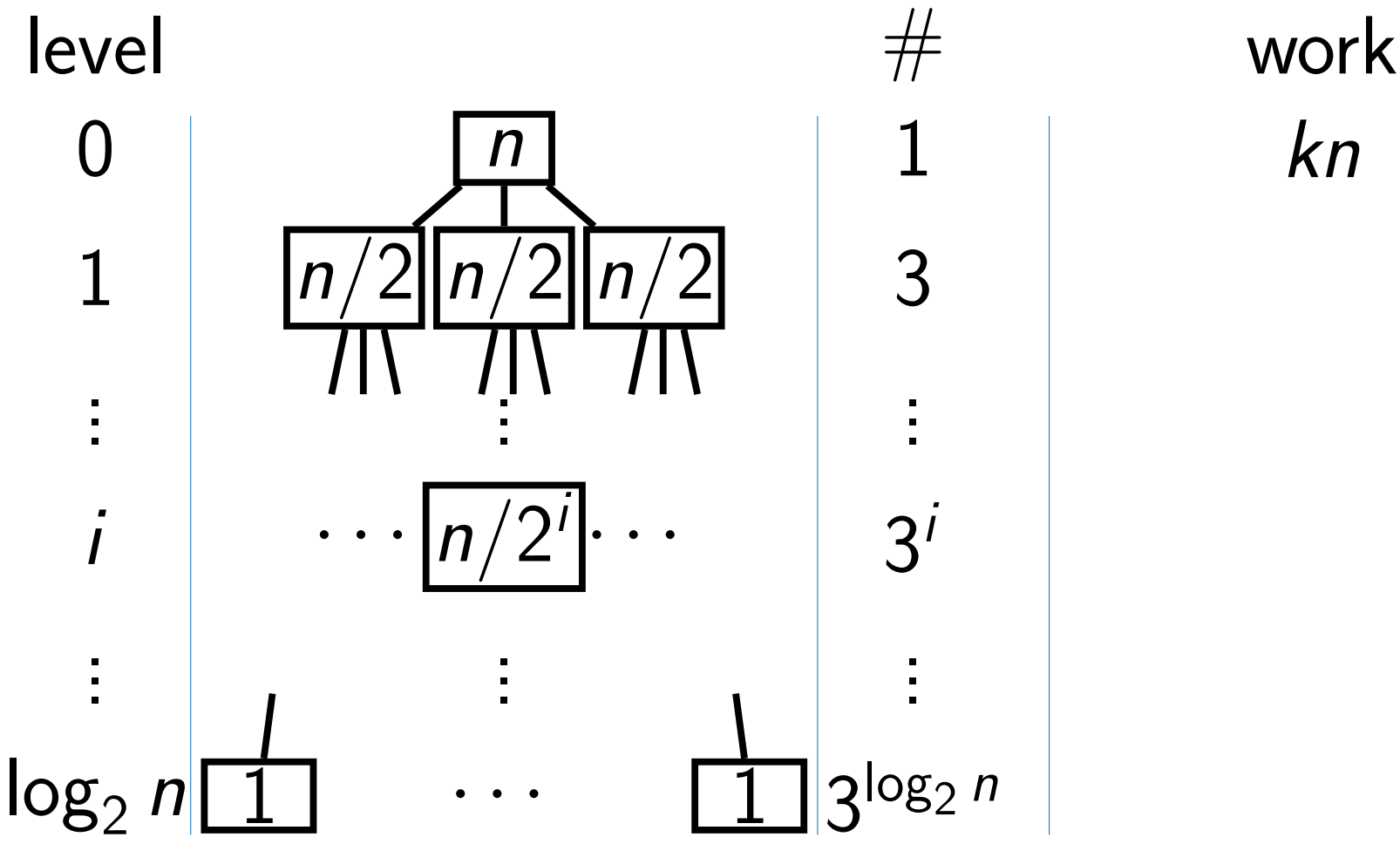

level
#
0
1
i
$\log_2 n$
n
$n/2$
$n/2$
$n/2$
$n/2^i$
1
1
1

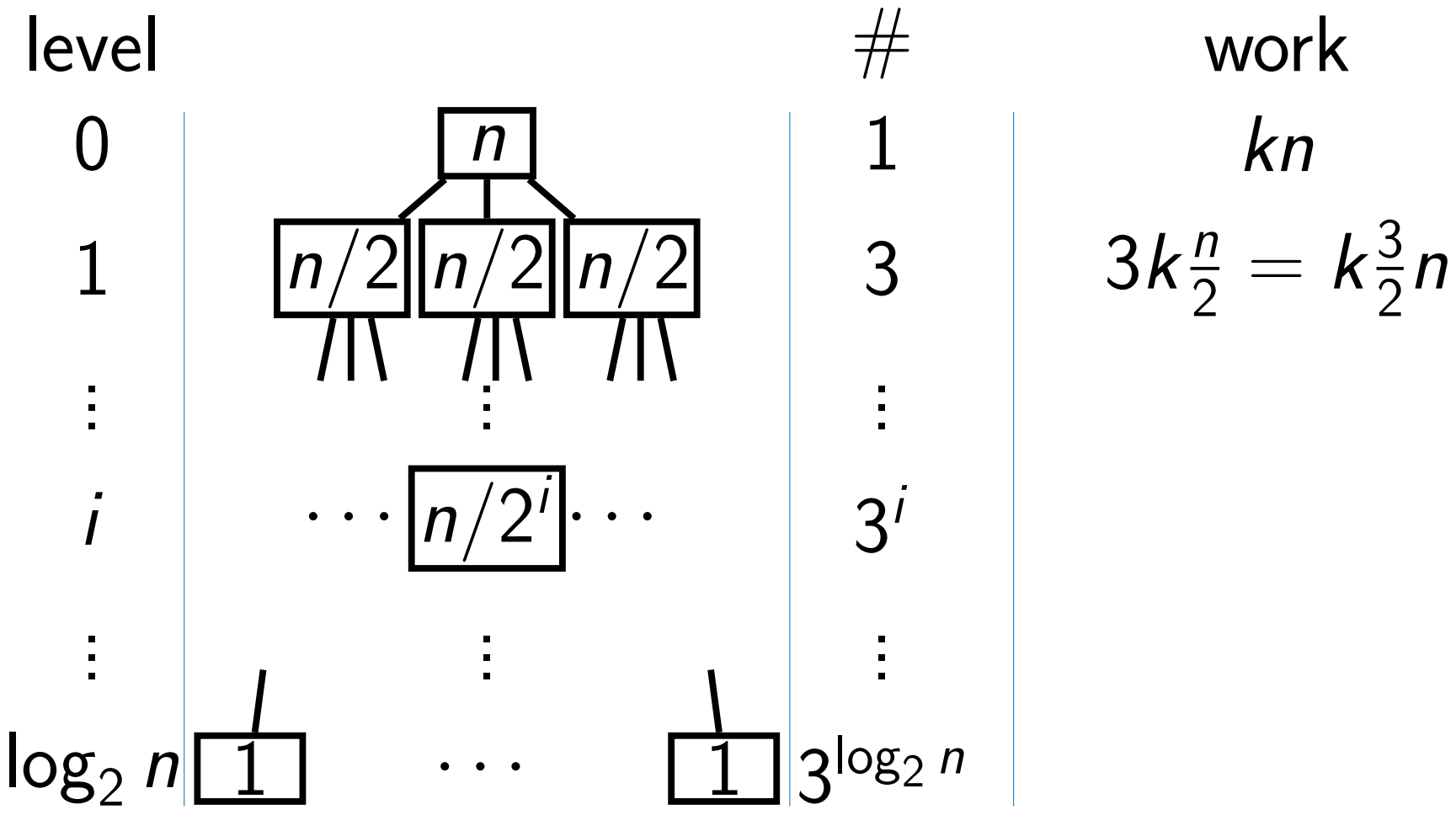

level
#
0
1
i
log2 n
n
n/2
n/2
n/2
n/2^i
1
1
1
3

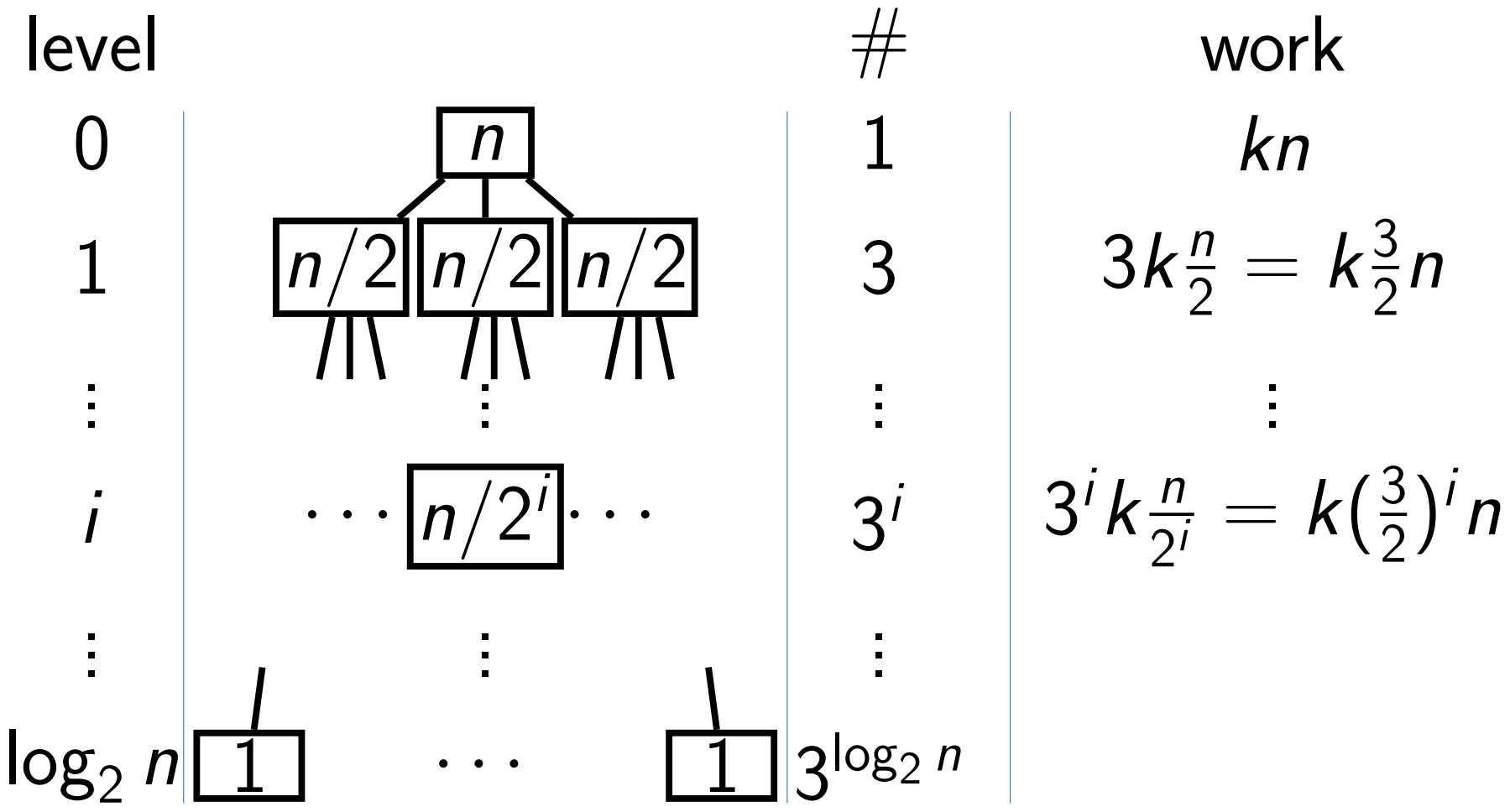

level
#
0
1
i
log2 n
n
n/2
n/2
n/2
n/2^i
1
1
1
3
3^i

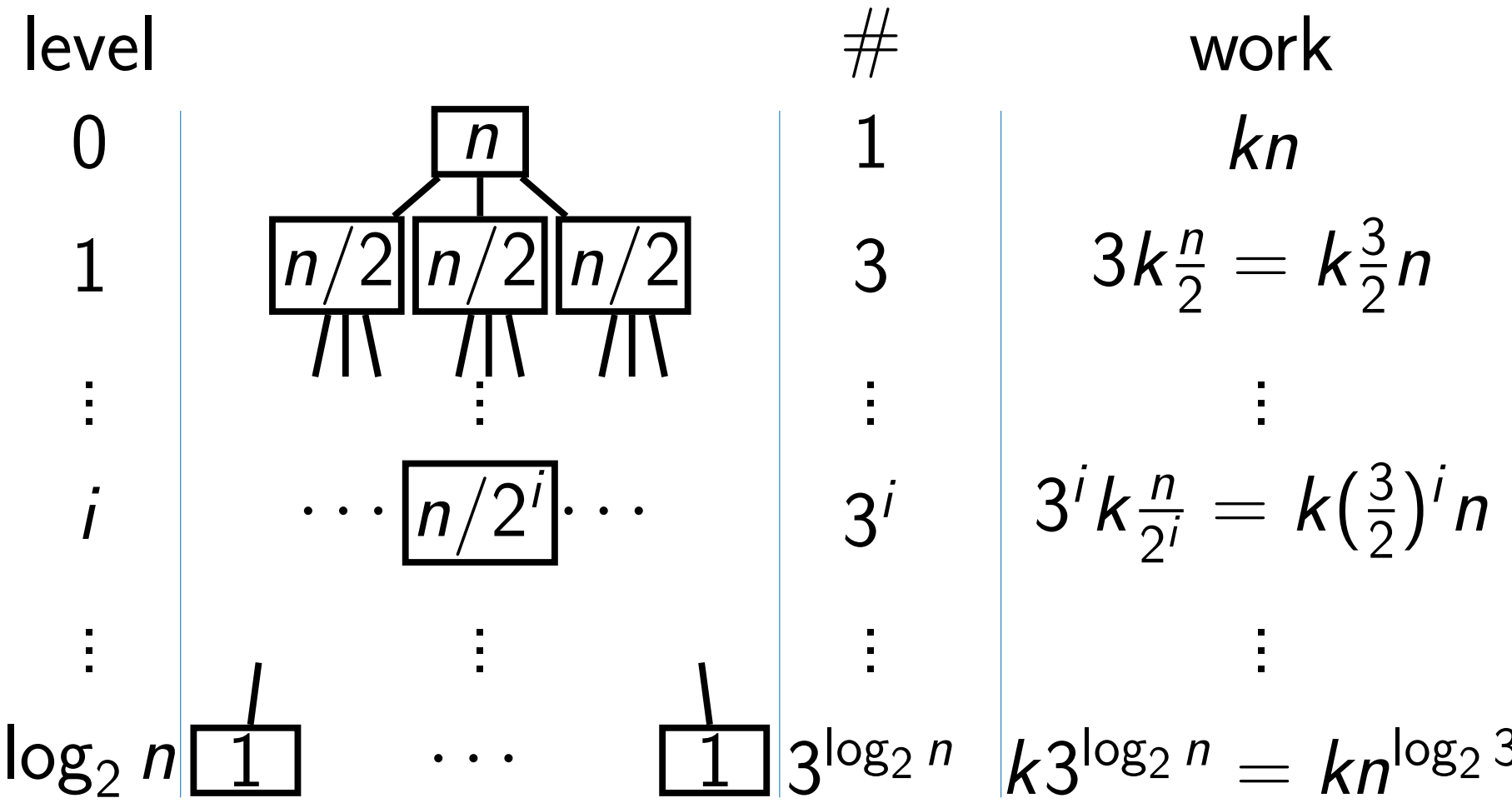

level
#
0
1
i
log2 n
n
n/2
n/2
n/2
n/2^i
1
1
1
3^log2 n


level
#
work
0
1
i
$\log_2 n$
n
$n/2$
$n/2$
$n/2$
$\cdots$ $n/2^i$ $\cdots$
1
$\cdots$
1
1
3
3^i
$3^{\log_2 n}$


level
#
work
0
1
i
$\log_2 n$
n
$n/2$
$n/2$
$n/2$
$\cdots$ $n/2^i$ $\cdots$
1
$\cdots$
1
1
3
3^i
$3^{\log_2 n}$
kn


level
#
work
0
1
i
$\log_2 n$
n
$n/2$
$n/2$
$n/2$
$\cdots$
$n/2^i$
$\cdots$
1
$\cdots$
1
1
3
3^i
$3^{\log_2 n}$
kn
$3k\frac{n}{2} = k\frac{3}{2}n$


level
#
work
0
1
kn
n
1
$n/2$
$n/2$
$n/2$
3
$3k\frac{n}{2} = k\frac{3}{2}n$
i
$\cdots$
$n/2^i$
$\cdots$
3^i
$3^i k\frac{n}{2^i} = k(\frac{3}{2})^i n$
$\log_2 n$
1
$\cdots$
1
$3^{\log_2 n}$


level
#
work
0
1
kn
n
n/2
n/2
n/2
1
3
3k n/2 = k 3/2 n
i
n/2^i
3^i
3^i k n/2^i = k(3/2)^i n
log_2 n
1
1
3^{log_2 n}
k3^{log_2 n} = kn^{log_2 3}

Outline

1. What is the Master Theorem
2. Proof of Master Theorem

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$



$$T(n) = O(\log n)$$

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$



$$T(n) = O(n^2)$$

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$



$$T(n) = O(n^{\log_2 3})$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$



$$T(n) = O(n \log n)$$

Master Theorem

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$

Master Theorem

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

Master Theorem

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases}$$

Master Theorem

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \end{cases}$$

Master Theorem

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ \end{cases}$$

Master Theorem

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

Master Theorem Example 1

$$T(n) = 4\,T\left(\frac{n}{2}\right) + O(n)$$

$$a = 4$$

Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

$$a = 4$$

$$b = 2$$

Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n^1)$$

$$a = 4$$

$$b = 2$$

$$d = 1$$

Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

$$a = 4$$

$$b = 2$$

$$d = 1$$

Since $d < \log_b a$, $T(n) = O(n^{\log_b a}) = O(n^2)$

Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$a = 3$$

Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$a = 3$$

$$b = 2$$

Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n^1)$$

$$a = 3$$

$$b = 2$$

$$d = 1$$

Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$a = 3$$

$$b = 2$$

$$d = 1$$

Since $d < \log_b a$,
$T(n) = O(n^{\log_b a}) = O(n^{\log_2 3})$

Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2$$

Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2$$

$$b = 2$$

Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^1)$$

$$a = 2$$

$$b = 2$$

$$d = 1$$

Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2$$

$$b = 2$$

$$d = 1$$

Since $d = \log_b a$,
$T(n) = O(n^d \log n) = O(n \log n)$

Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Master Theorem Example 4

$$T(n) = 1\,T\left(\frac{n}{2}\right) + O(1)$$

$$a = 1$$

Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$a = 1$$

$$b = 2$$

Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(n^0)$$

$$a = 1$$

$$b = 2$$

$$d = 0$$

Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$a = 1$$

$$b = 2$$

$$d = 0$$

Since $d = \log_b a$, $T(n) = O(n^d \log n) = O(n^0 \log n) = O(\log n)$

Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

$$b = 2$$

Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

$$b = 2$$

$$d = 2$$

Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

$$b = 2$$

$$d = 2$$

Since $d > \log_b a$, $T(n) = O(n^d) = O(n^2)$

Outline

1. What is the Master Theorem
2. Proof of Master Theorem

Master Theorem

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

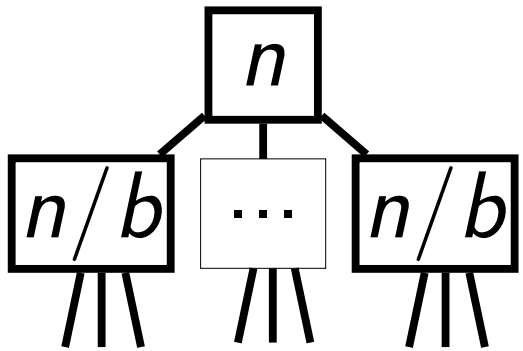
$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$

level

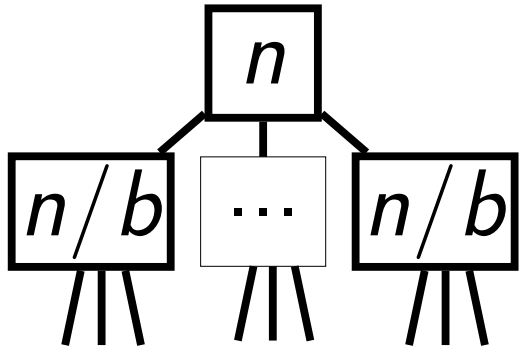


$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$

level

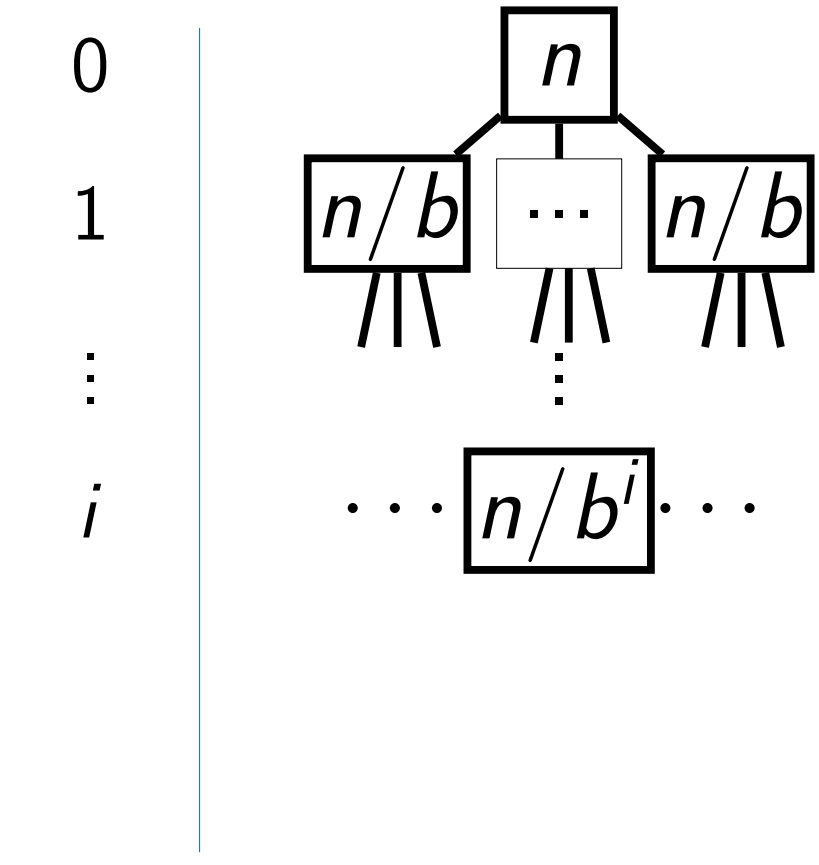
0

1

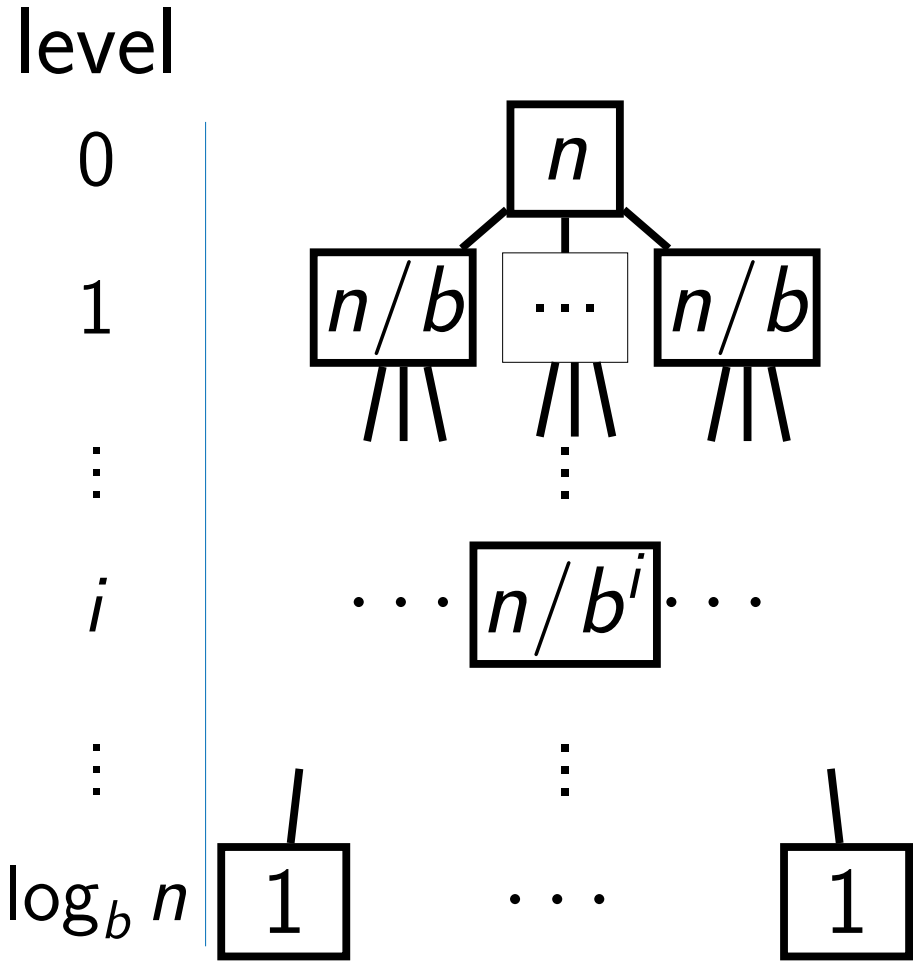


$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$

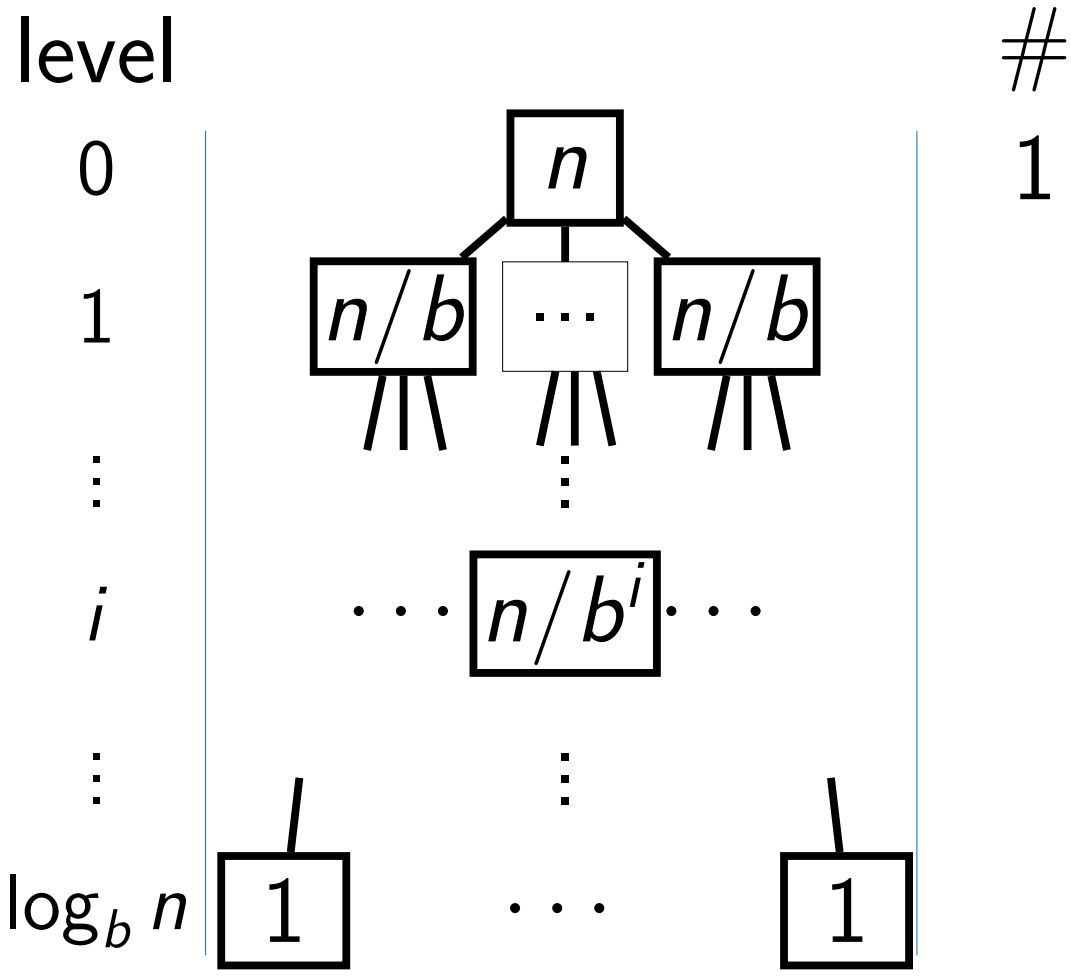
level



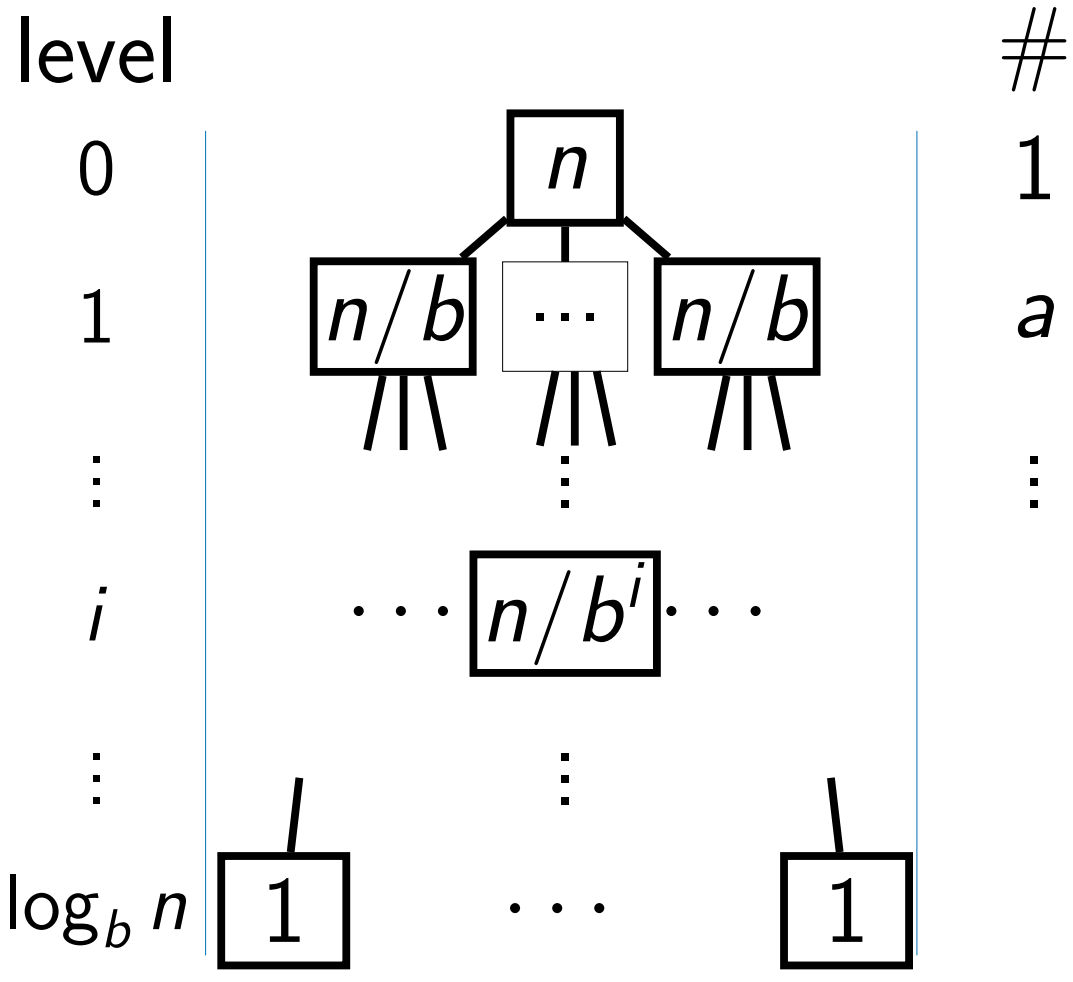
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



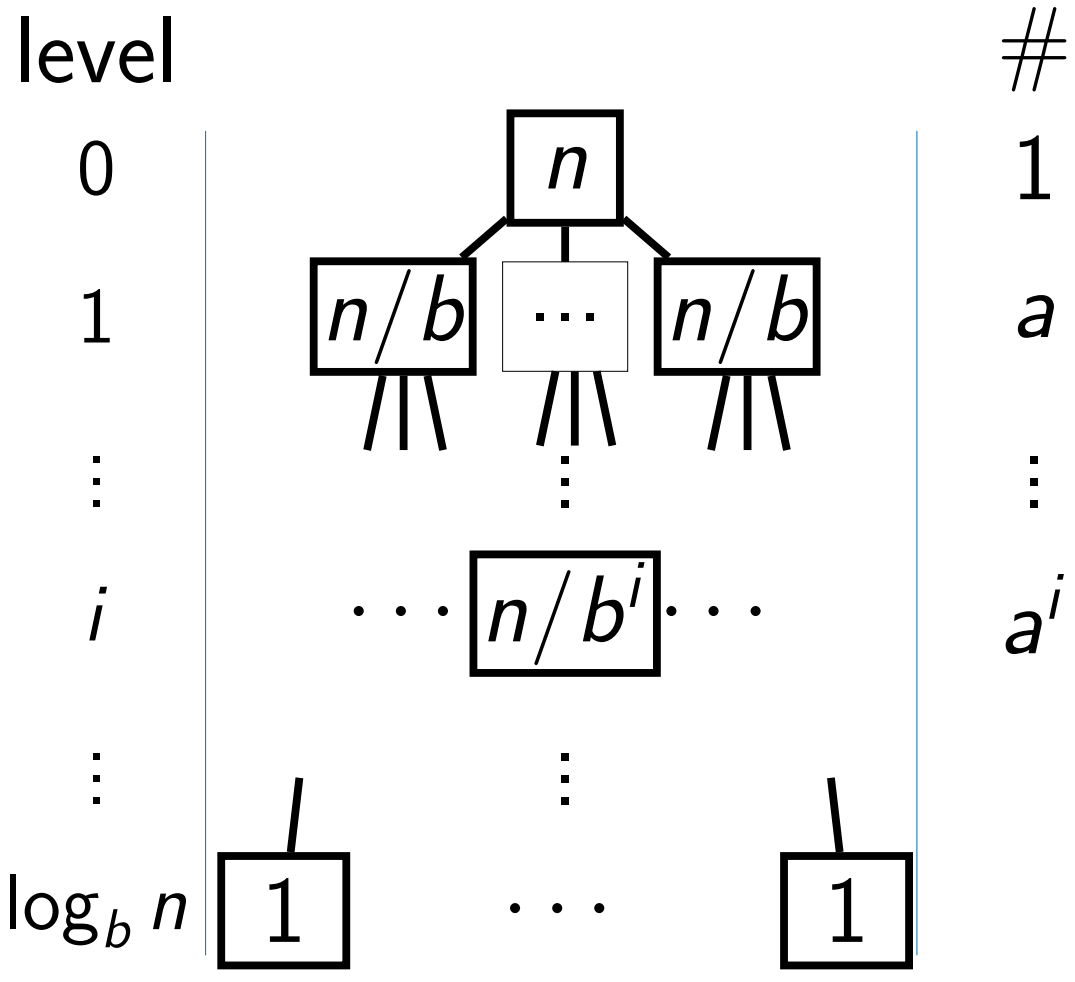
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



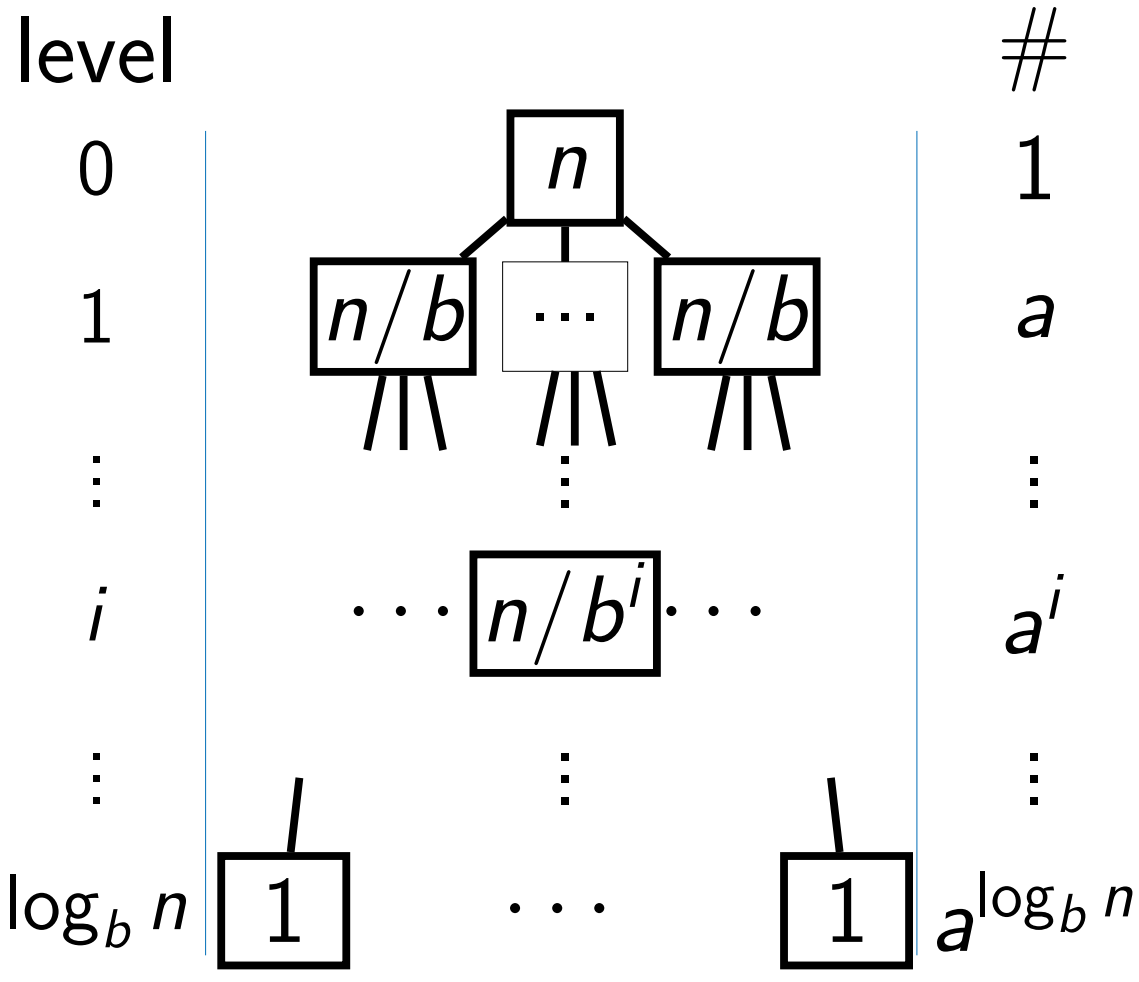
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



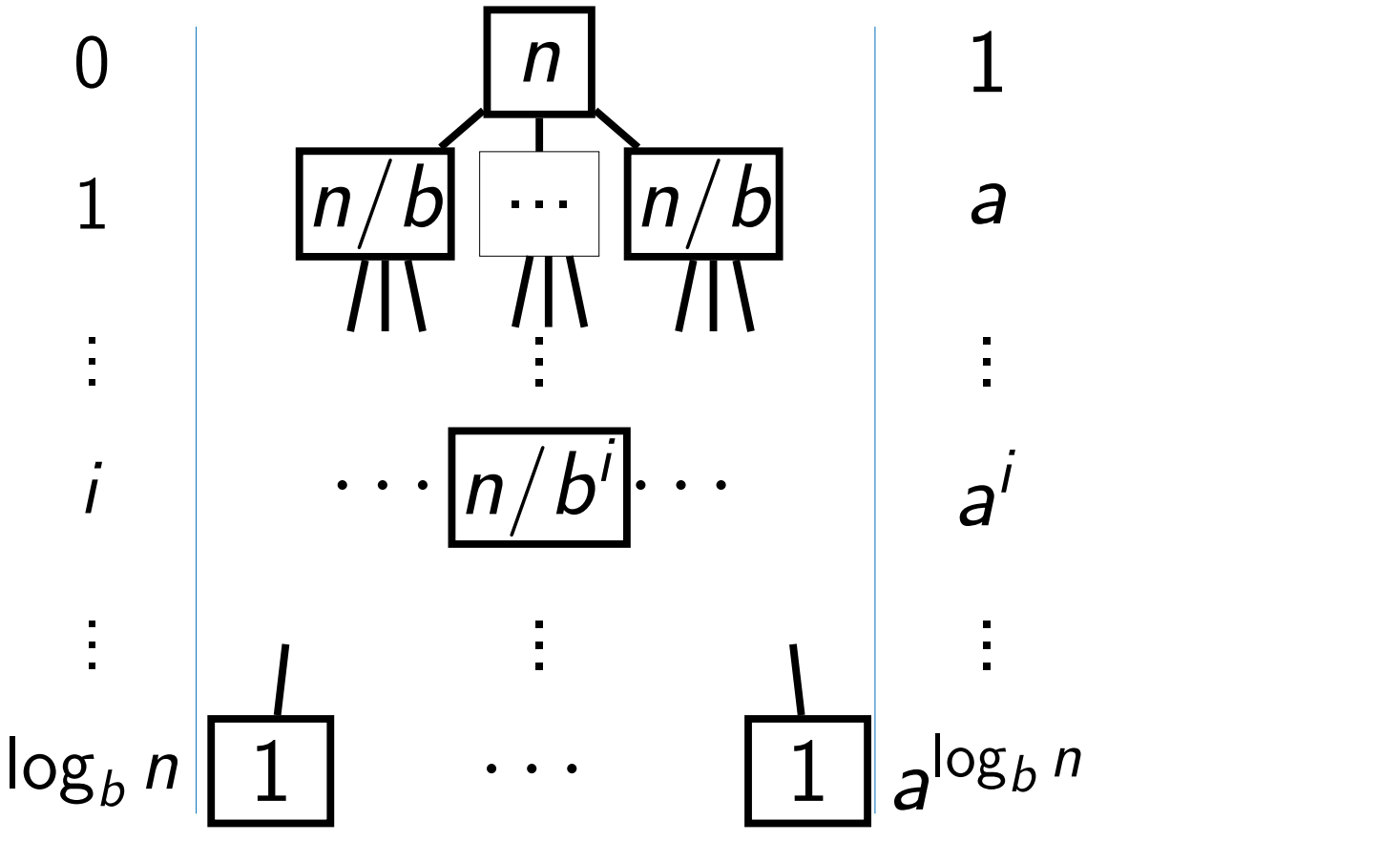
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



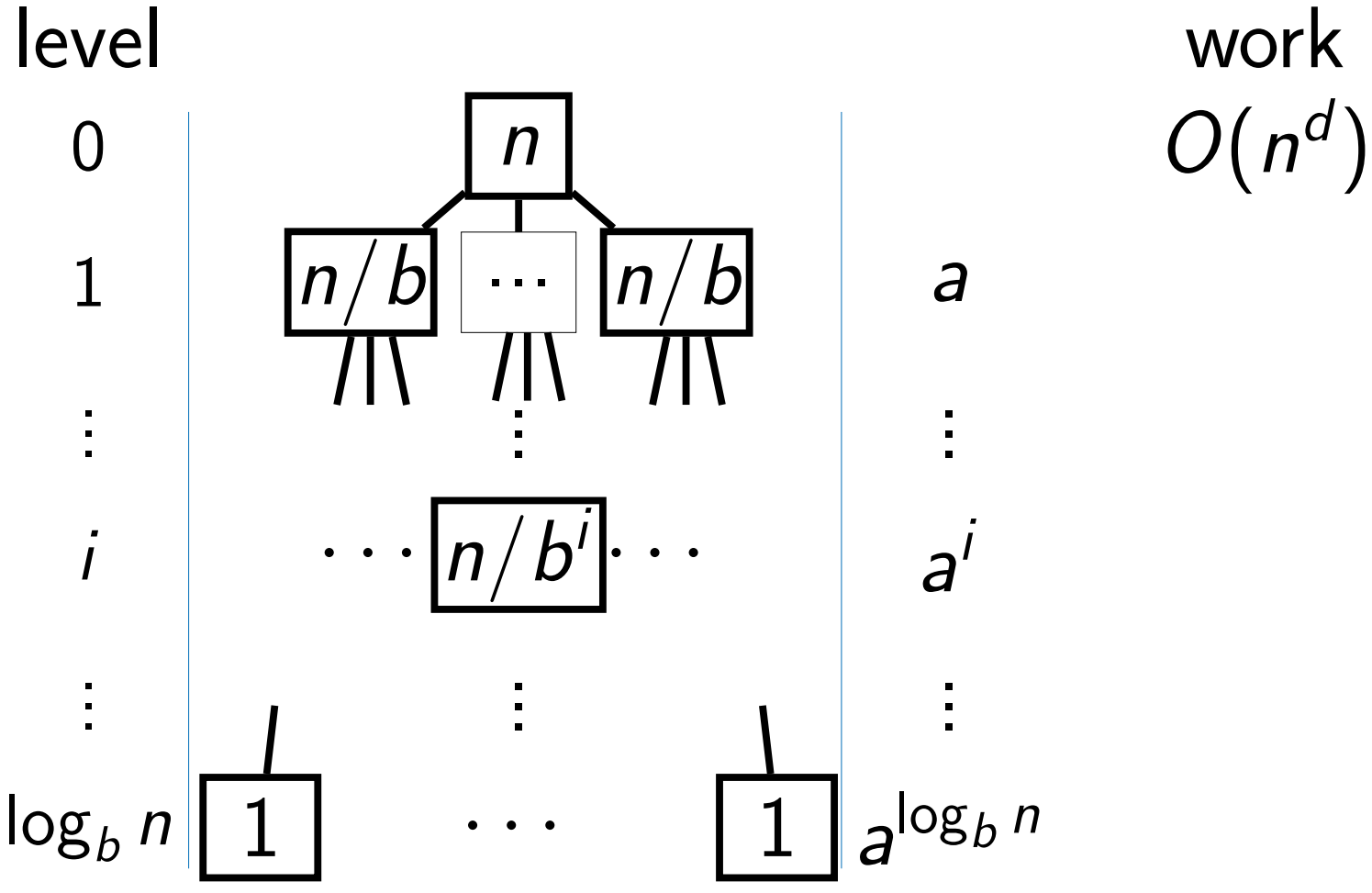
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



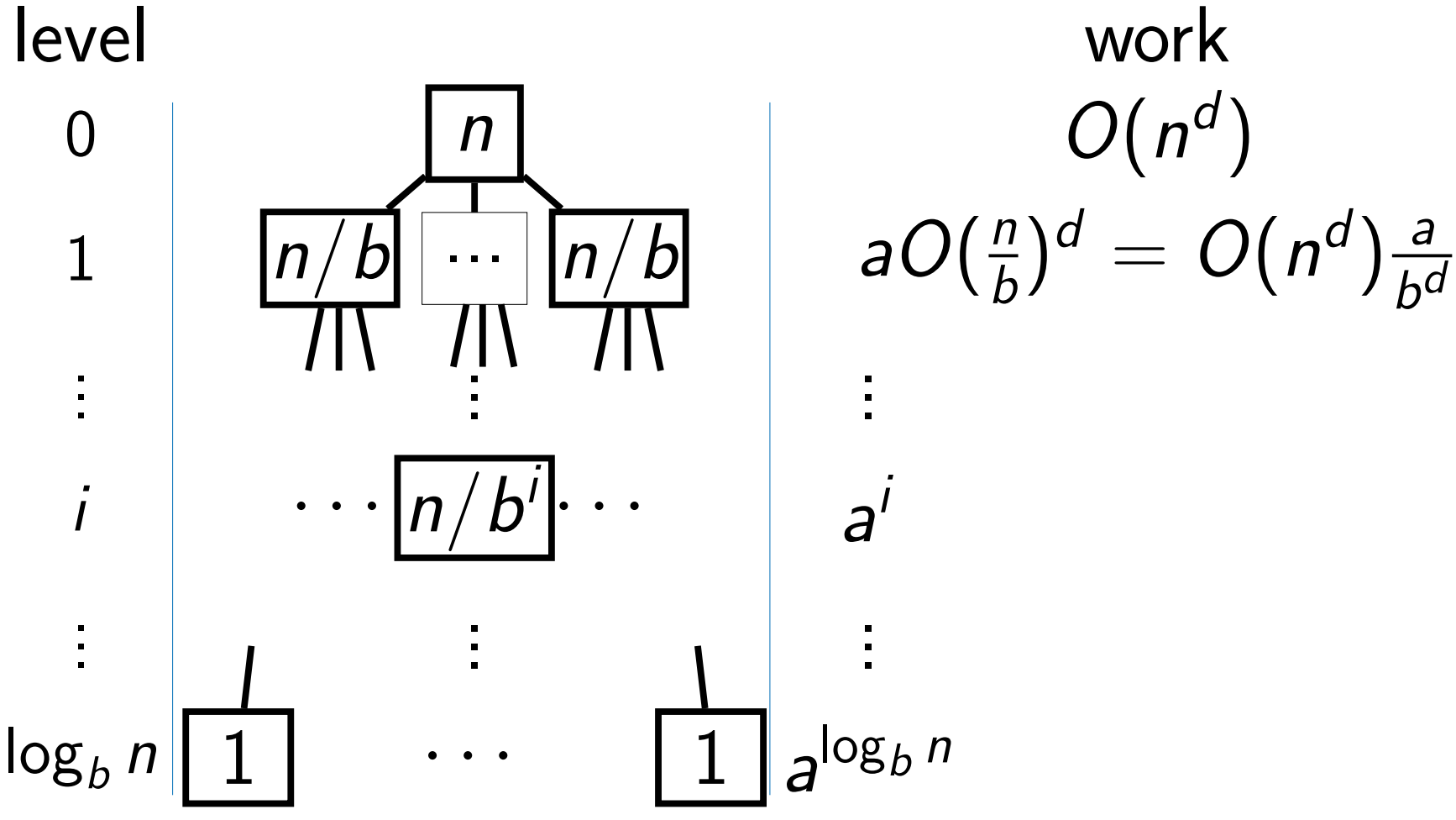
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



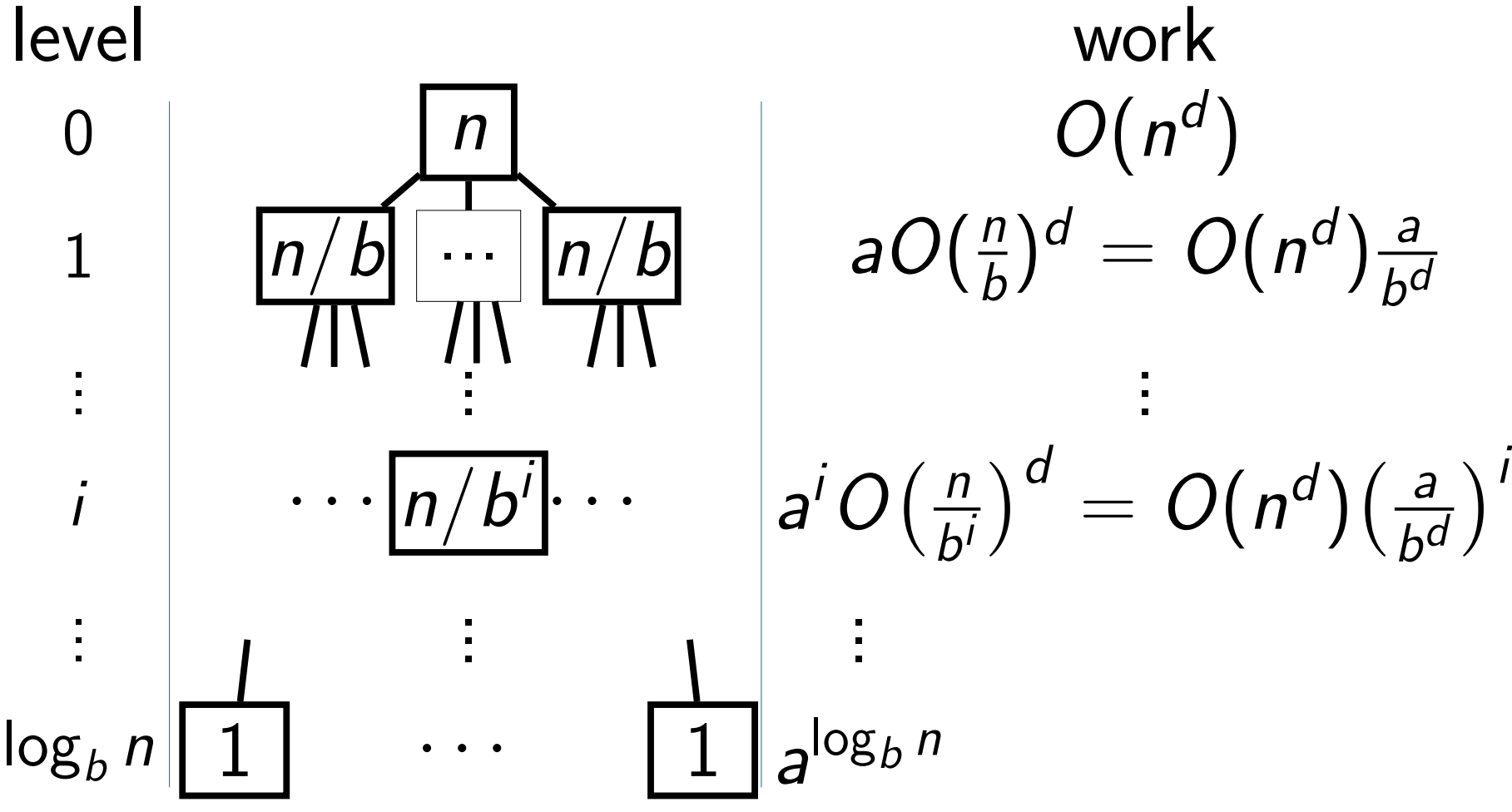
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



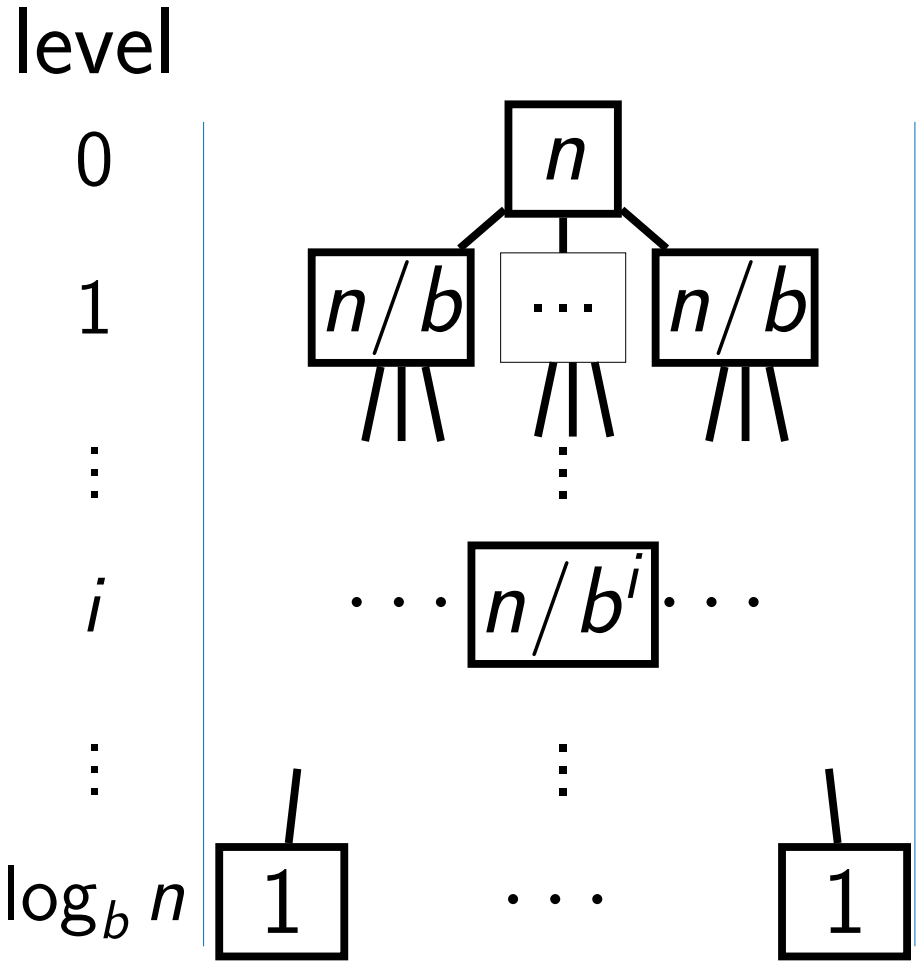
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



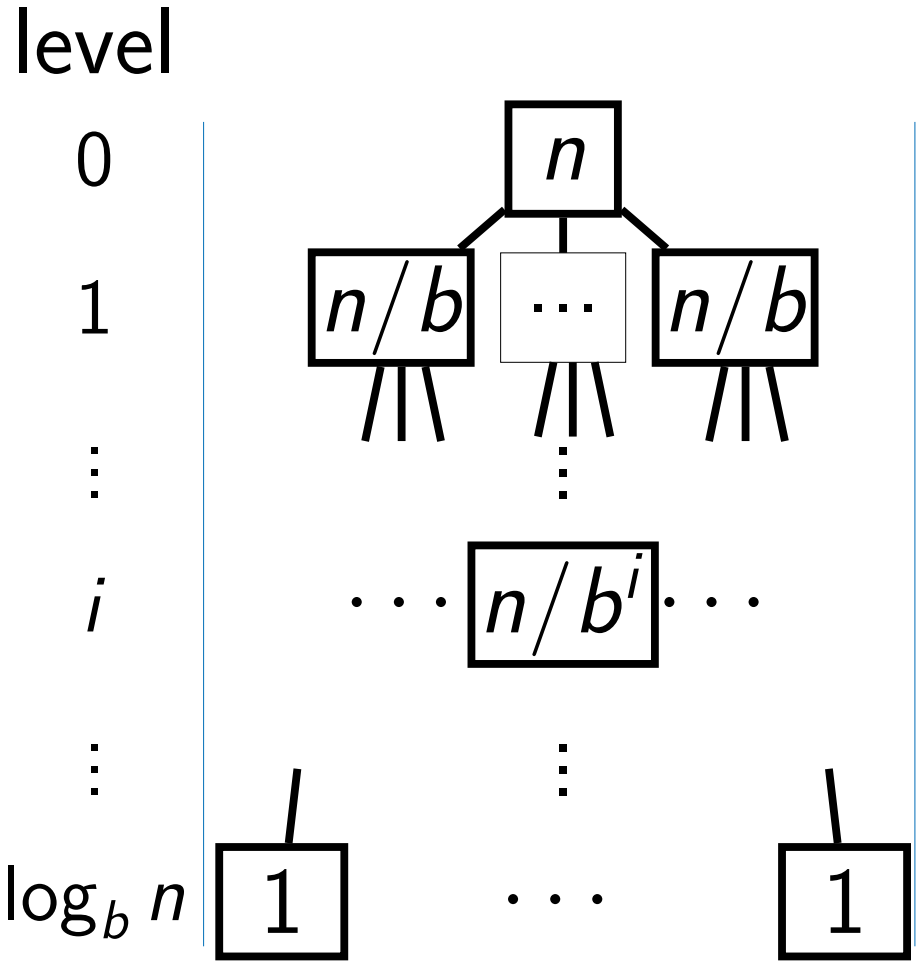
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



Geometric Series

For $r \neq 1$:

$$a + ar + ar^2 + ar^3 + \cdots + ar^{n-1}$$

Geometric Series

For $r \neq 1$:

$$\begin{aligned} & a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\ =& a\frac{1 - r^n}{1 - r} \end{aligned}$$

Geometric Series

For $r \neq 1$:

$$
\begin{aligned}
& a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\
=& a\frac{1 - r^n}{1 - r} \\
=& \begin{cases}
\end{cases}
\end{aligned}
$$

Geometric Series

For $r \neq 1$:

$$
\begin{aligned}
& a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\
=& a\frac{1 - r^n}{1 - r} \\
=& \begin{cases} O(a) & \text{if } r < 1 \\ \end{cases}
\end{aligned}
$$

Geometric Series

For $r \neq 1$:

$$
\begin{aligned}
& a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\
=& a\frac{1 - r^n}{1 - r} \\
=& \begin{cases} O(a) & \text{if } r < 1 \\ O(ar^{n-1}) & \text{if } r > 1 \end{cases}
\end{aligned}
$$

Case 1: $\frac{a}{b^d} < 1$ ($d > log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

Case 1: $\frac{a}{b^d} < 1$ ($d > log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O(n^d)$$

Case 2: $\frac{a}{b^d} = 1$ $(d = log_b a)$

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

Case 2: $\frac{a}{b^d} = 1$ $(d = log_b a)$

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= \sum_{i=0}^{\log_b n} O(n^d)$$

Case 2: $\frac{a}{b^d} = 1$ ($d = log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= \sum_{i=0}^{\log_b n} O(n^d)$$

$$= (1 + \log_b n) O(n^d)$$

Case 2: $\frac{a}{b^d} = 1$ ($d = log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= \sum_{i=0}^{\log_b n} O(n^d)$$

$$= (1 + \log_b n) O(n^d)$$

$$= O(n^d \log n)$$

Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

$$= O\left(O(n^d)\frac{a^{\log_b n}}{b^{d \log_b n}}\right)$$

Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

$$= O\left(O(n^d)\frac{a^{\log_b n}}{b^{d \log_b n}}\right)$$

$$= O\left(O(n^d)\frac{n^{\log_b a}}{n^d}\right)$$

Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

$$= O\left(O(n^d)\frac{a^{\log_b n}}{b^{d \log_b n}}\right)$$

$$= O\left(O(n^d)\frac{n^{\log_b a}}{n^d}\right)$$

$$= O(n^{\log_b a})$$

Summary

Master theorem is a shortcut:

Summary

Master theorem is a shortcut:

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

CORRECTNESS

What is Correctness?

- Simply, an algorithm is **correct** if for any **valid input** it produces the **result required** by the algorithm's **specification**

- For example, a sorting function: **sort(int[ ] in)**
  - We specify that for a **valid** integer array as input
  - The sort function will sort the input integer array into **ascending numerical order**
  - The result that is returned is a correctly sorted integer array, for any valid array of integers

Why not just test?

- Testing is a **critical skill** for a developer
- Provides no guarantees of correctness
  - Depends on generation of **good test-cases**
  - Generally these follow sensible heuristics
    - Test correct values, e.g. `sort({1,2,3})`
    - Test incorrect values, e.g. `sort({‘a’, ‘b’, ‘c’})`
    - Test boundaries, e.g. `sort({0, 1, ..., MAX_VALUE})`
  - However, this is by **no means exhaustive**, and there will be numerous missing tests
    - Trade-off in effort vs completeness
    - Let’s face it, developers have deadlines

Adopt more formalism

- We can express our algorithms as **mathematical entities** and use various strategies to prove correctness (rather than guess work)
- However, this is an expensive process...
  - That is, the cost up front is expensive
  - The benefits come late (**contradicts time to market**)
  - Yet, software estimation is more art than science
    - Money, time, developers
    - As a rule...take your best guess and multiply it by 3 ;-)
  - But can be useful for algorithms in particular
- Two approaches considered here:
  - **Loop Invariants**
  - **Mathematical Induction**

Invariants

- In English: *adjective* never changing

- IN CS: An invariant is a statement about the state of some variable(s) that **can be relied to hold true** during execution

- Two example types:
  - **Loop Invariants:** Invariants hold true before and after each iteration of the loop (e.g. for or while)
  - **Class invariants:** Invariants hold true throughout the life-span of an object instantiated from the class

Loop Invariants

- Non-trivial algorithms involve forms of looping
  - Iteration
  - Recursion
- Loop invariants allow **logical assertions** about the **behaviour** of the loop to be declared
  - Can be implicitly **documented** as comments
  - Can be explicitly **coded** as a condition or assertion
- By establishing the correctness of looping behaviour, we can move towards correctness of an algorithm
  - As well as understand better **the effects of the loop**

Loop Invariants: Sum

```
/ **
* Returns the sum 1 + 2 + ... + n, where n> = 1st
* /
public long sum (int n) {
    long sum = 0;
    int i = 1;
    while (i <= n) {
        // Invariant: sum = 1 + 2 + ... + (i - 1)
        sum+ = i;
        i ++;
    }
    return sum;
}
```

Loop Invariants: Good Properties

- Three characteristics ensure goodness:
  - **Initialisation**
    - The loop invariant must be true before we execute the loop
    - e.g. sum = 0 and i = 1
  - **Update**
    - If the loop invariant is true in one iteration, it must also be true in the next iteration of the loop
  - **Conclusion**
    - Finally, the loop invariant will tell us something useful about the conclusion of the loop, which should help us understand the behaviour
    - sum = 1 + 2 + ... + n-1 + n

Loop Invariants: In design of max( )

```
/ **
 * Returns the max value in the vector v.
 * /
public int max (int [] v) {
    int max = ...;
    for (int i = 0; i <v.length; i ++) {
        // Invariant: max = max (v[0], v[1], ..., v[i-1])
        ...
    }
    return max;
}
```

We can see that it satisfies the 'conclusion' characteristic, but there is an issue: Loop Invariant must be true at the start of the loop - what is the problem?

Loop Invariants: In design of max( )

```
/ **
 * Returns the max value in the vector v, where v.length > 0
 *
 *throws IllegalArgumentException if v.length = 0
 * /
public int max (int [] v) {
    if (v.length == 0)
    throw new IllegalArgumentException ("v.length = 0");

    int max = ...;
    for (int i = 1; i <v.length; i ++) {
        // Invariant: max = max (v[0], v[1], ..., v[i-1])
        ...
    }
    return max;
}
```

Loop Invariants: In design of max( )

```
/ **
 * Returns the max value in the vector v, where v.length > 0
 *
 *throws IllegalArgumentException if v.length = 0
 * /
public int max (int [] v) {
    if (v.length == 0)
    throw new IllegalArgumentException ("v.length = 0");

    int max = v [0];
    for (int i = 1; i <v.length; i ++) {
        // Invariant: max = max (v [0], v [1], ..., v [i-1])
        if (v [i]> max) {
            max = v [i];
        }
    }
    return max;
}
```